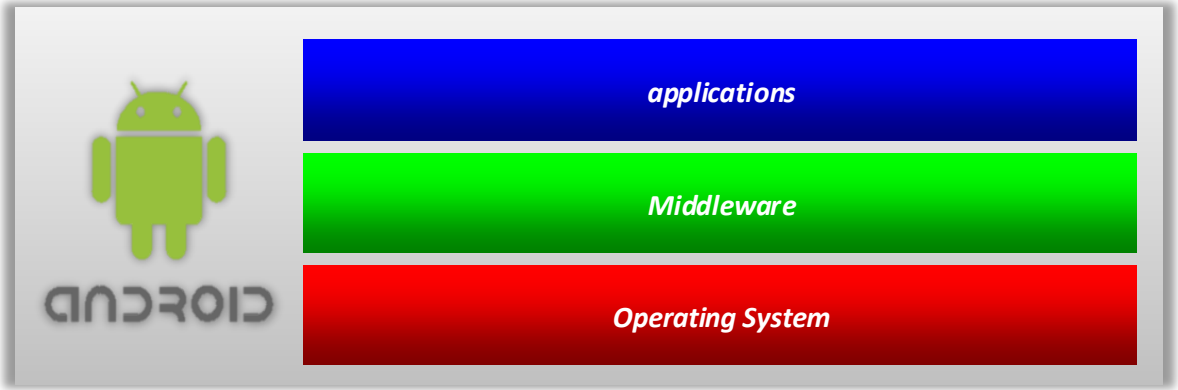
Android

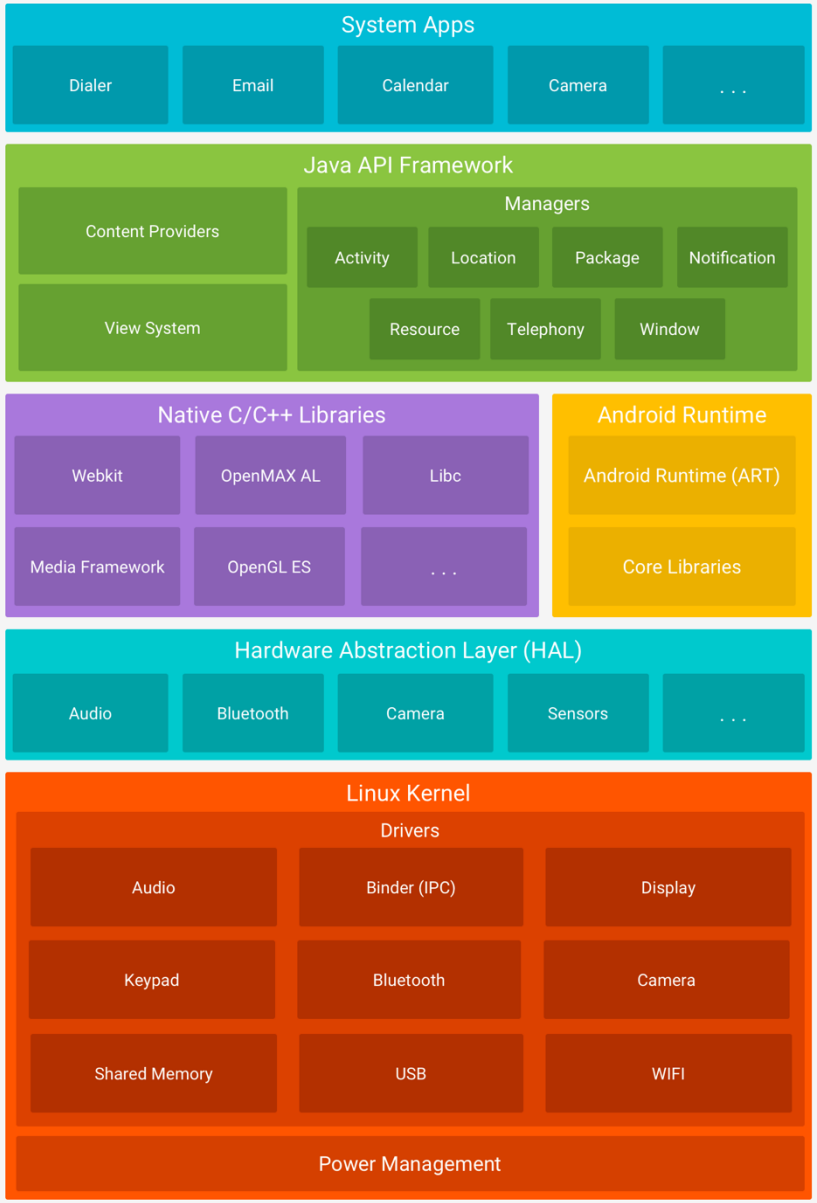
Android architecture and development

Android

- Software suite that includes the operating system, *middleware* and applications
  - Linux-based (initially, version 2.6)
    - From version 4 of *Android*, version 3 of Linux started to be used
    - From version 7 of *Android*, version 4 of Linux started to be used
    - From version 11 of *Android*, version 5 of Linux started to be used
    - From version 14 of *Android*, version 6 of Linux started to be used
  - Support for the latest applications and technologies in the area of mobile devices (photography, video, GPS, compass, accelerometer, games, OpenGL , SQLite, ... and cell phone 😀)



Architecture



- *System apps*
  - Set of applications that offer the basic functionalities expected by an end user: Calendar, SMS, Contacts, *Browser* , …
- *Java API Framework*
  - APIs that provide access to essential functionalities for developing applications for the *Android platform*
- *Native C/C++ Libraries*
  - Native C/C++ libraries that provide access to lower-level functionality, e.g. *OpenGL ES* , *SQLite*
  - Used by *Java libraries*
- *Android Runtime*
  - Virtual machine in the context of which applications run
  - *Java libraries (Java basic features)*
- *Hardware abstraction layer (HAL)*
  - Layer that normalises hardware specificities (*camera* , sensors, …)
- *Linux Kernel*
  - *Threading* , memory management, file system, …

Architecture – Android Runtime

- Constituted by:
  - Set of base libraries similar to the Java language support libraries
  - *Runtime Environment* ("Virtual machine")
    - *Dalvick Virtual Machine* (≤ API 19)
      - *Just-In-Time (JIT) compiler*
        - The code is compiled when it needs to be executed
    - ART (≥ API 21 – experimental on API 19)
      - *Ahead-Of-Time (AOT) compiler*
        - The code is compiled in advance
        - JIT is also available
      - *Garbage Collection* optimized

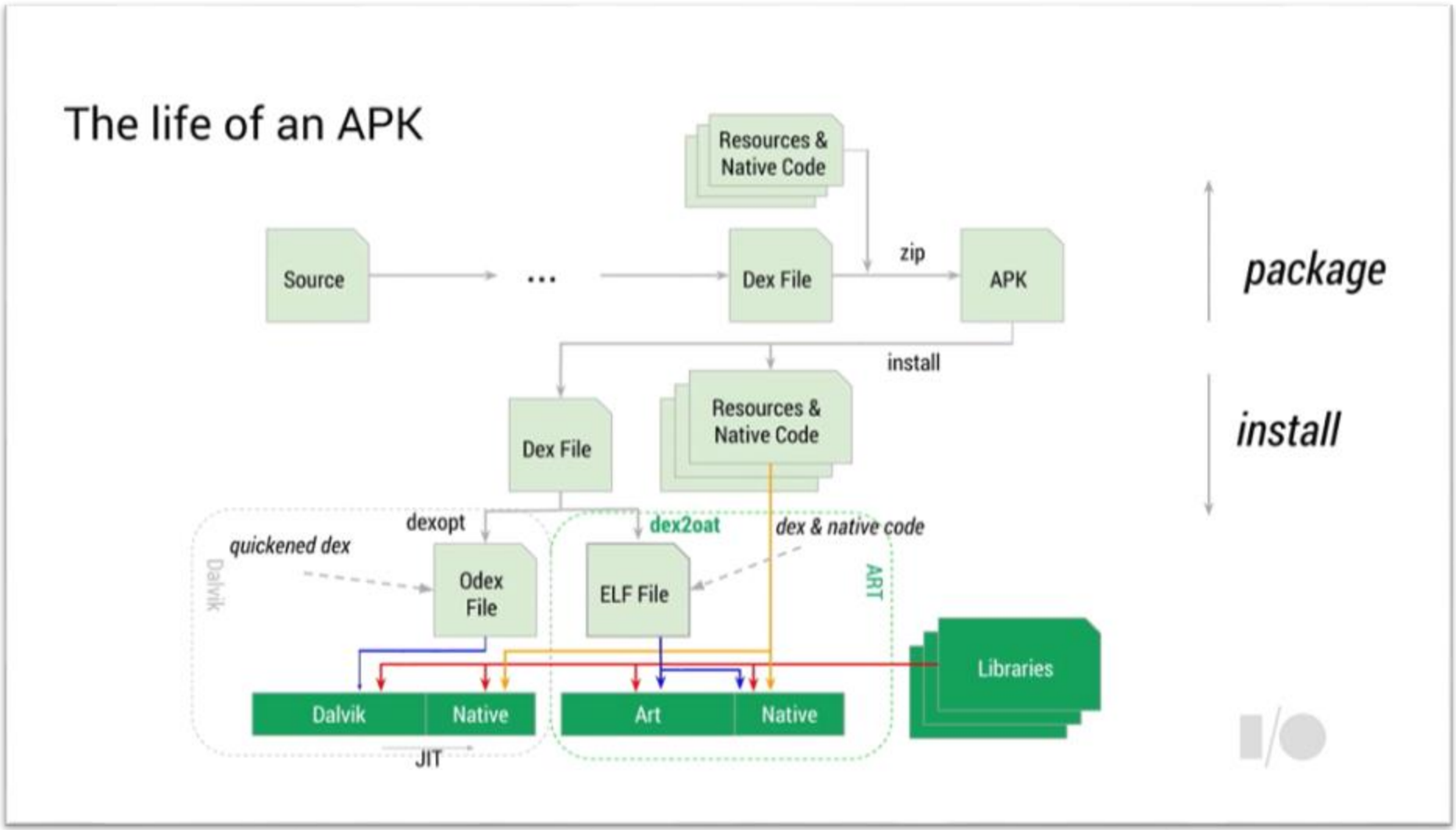
Java API Framework

- Modular system of components and services
  - *Content Providers*
    - Allows sharing or access to certain types of data from the system and other applications (e. g. Contacts)
  - *View System*
    - Management system for all views that make up the UI (buttons, lists, text boxes, …)
  - *Managers*
    - *Activity*
      - Responsible for managing the life cycle of activities/applications and *back stack*
    - *Location*
      - Allows access to GPS and other location systems
    - *Package*
      - Responsible for managing the various software packages installed on the device
    - *Notification*
      - Responsible for managing the notification bar, allowing different applications to present notifications to the user
    - *Resource*
      - Manages available resources. For example:
        - chooses the best layout for the application depending on the screen size, device orientation,…
        - adequacy of messages to the language defined on the device
    - *Telephony*
      - Implementation of all functionalities related to mobile communications
    - *Window*
      - Management of all application windows and related events
    - …

Android application development

- Main languages used: *Java* and *Kotlin*
- The code is compiled and a *package* with extension `.apk` is created
  - The *apk* also includes additional features
  - The assembly of all components in an *apk* is called an application
  - More recently, another form of application distribution called *Bundles* (`.aab`) has emerged and become mandatory
- Each application
  - Runs in an independent Linux process
    - However, it can be supported by several processes
  - It has an (instance of) virtual machine independent of other running applications

APK files


The life of an APK
Resources & Native Code
Source
...
Dex File
zip
APK
package
install
Dex File
Resources & Native Code
install
dexopt
dex2oat
dex & native code
quickened dex
Odex File
ELF File
Dalvik
ART
Libraries
Dalvik
Native
Art
Native
JIT

Components of an application

- Android applications can have multiple entry points
  - They have a set of components that can be activated whenever necessary (by another component of the same application or another application)

- Component types that can constitute an Android application
  - *Activity*
  - *Service*
  - *Broadcast Receiver*
  - *Content Provider*

Activity

- This type of component is characterized by having a visual interface

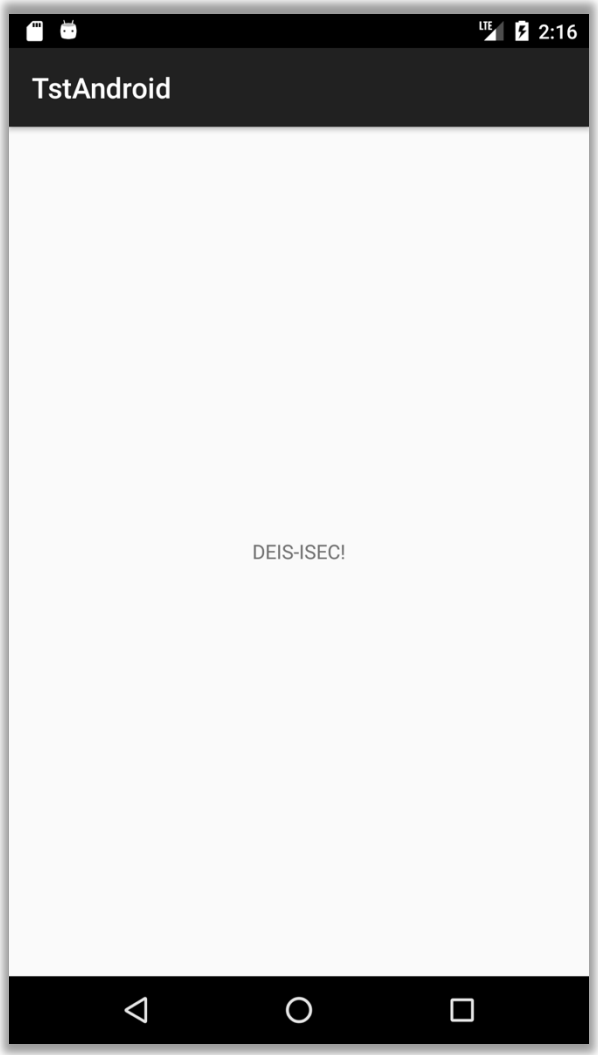
- An application can contain several *Activities*
  - classes derived from the `Activity` class
  - Activities are independent of each other
  - An *Activity* can be classified as being the first to be presented by the application

- Usually it occupies the entire screen, but there may be cases where it takes up less space or you can use additional windows to request or show some type of information

Activity

- To create an activity…
  - Derive a class from `Activity`
  - Define the visual components
  - Register the activity

```
class MainActivity : Activity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
}
```



Activity

- Visual contents are organized according to a hierarchy of objects derived from the `View` class
  - Each object is responsible for arranging the *layout* of its dependent objects (belonging to the hierarchy)
  - There are objects with different objectives
    - Arrange other objects
      - ~~`AbsoluteLayout`~~, `FrameLayout`, `LinearLayout`, `RelativeLayout`, `ConstraintLayout`, …
    - Interact with the user (presentation and request for information)
      - `Button`, `TextView`, `EditText`, `CheckBox`, `RadioButton`, `TimePicker`, …
  - The object hierarchy is assigned to an activity via the method:
    - `Activity.setContentView(<…>)`
- The organization of the object hierarchy is somewhat similar to what was already studied about *JavaFX* in the *Advanced Programming* subject (2nd year, 2nd semester)

Service

- A *Service* does not have a graphical interface
  - Used to implement functionalities that do not require asking or showing information to the user
    - Applications can use *Activity* components for this purpose
- Runs in *background*
- Classes must derive from the `Service` class
- Provides an interface (set of methods; "API") so that other components can interact with it

```
class MyService : Service() {

    override fun onBind(intent: Intent): IBinder {
        TODO("Not yet implemented")
    }
}
```

Broadcast Receiver

- Component that allows to wait for messages/broadcasts and react appropriately
  - Example of system-originated broadcasts
    - Battery Level
    - Language change
    - Changing the time zone
    - Receiving an SMS
    - …
  - Applications can also generate their own *broadcasts*

- Derive from the `BroadcastReceiver` class

Broadcast Receiver

- An application can contain several *Broadcast Receivers*

- Like services, they do not have a graphical interface although they can launch activities for this purpose
  - They can use the `NotificationManager` class to signal events (through vibration, sounds, blinking)
  - Can show messages through appropriate icons in the status bar

Broadcast Receiver

- Class derived from the `BroadcastReceiver` class

```
class MyReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context, intent: Intent) {
        // This method is called when the BroadcastReceiver
        // is receiving an Intent broadcast
        TODO("Not yet implemented")
    }
}
```

Content Provider

- Allows information to be made available to the various components of the application or other applications

- The information can be...
  - generated based on request
  - accessed from a suitable data source
    - Does not include its own data persistence mechanisms
    - Data can be stored in files, SQLite databases, accessed through means of communication, ...

Content Provider

- Derives from the `ContentProvider` class
  - Implements a set of methods that allow other components or applications to access and manage information
    - These methods are invoked by other applications through a `ContentResolver` object

- There are some *Content Providers* made available by the system that allow access to information managed by it
  - For example: Contacts, Tasks, Images, Videos, …

Content Provider

- Class derived from the `ContentProvider` class

```
class MyContentProvider : ContentProvider() {

    override fun onCreate(): Boolean {...}

    override fun delete(uri: Uri, selection: String?,
                                        selectionArgs: Array<String>?): Int {...}

    override fun getType(uri: Uri): String? {...}

    override fun insert(uri: Uri, values: ContentValues?): Uri? {...}

    override fun query(uri: Uri, projection: Array<String>?, selection: String?,
                    selectionArgs: Array<String>?, sortOrder: String?): Cursor? {...}

    override fun update(uri: Uri, values: ContentValues?, selection: String?,
                                        selectionArgs: Array<String>?): Int {...}
}
```

Component activation: *Intent*

- As already mentioned, the activation of a *Content Provider* is performed when a request is sent through a `ContentResolver` object

- The other types of components are activated through asynchronous messages called *Intent*
  - An `Intent` object
    - describes the purpose of the message
    - passes values/parameters between components
      - Functions `putExtra(key,value)` and `get<type>(key)` to associate attribute value pairs with the `Intent` or access previously saved values (similar to a *hashmap*)

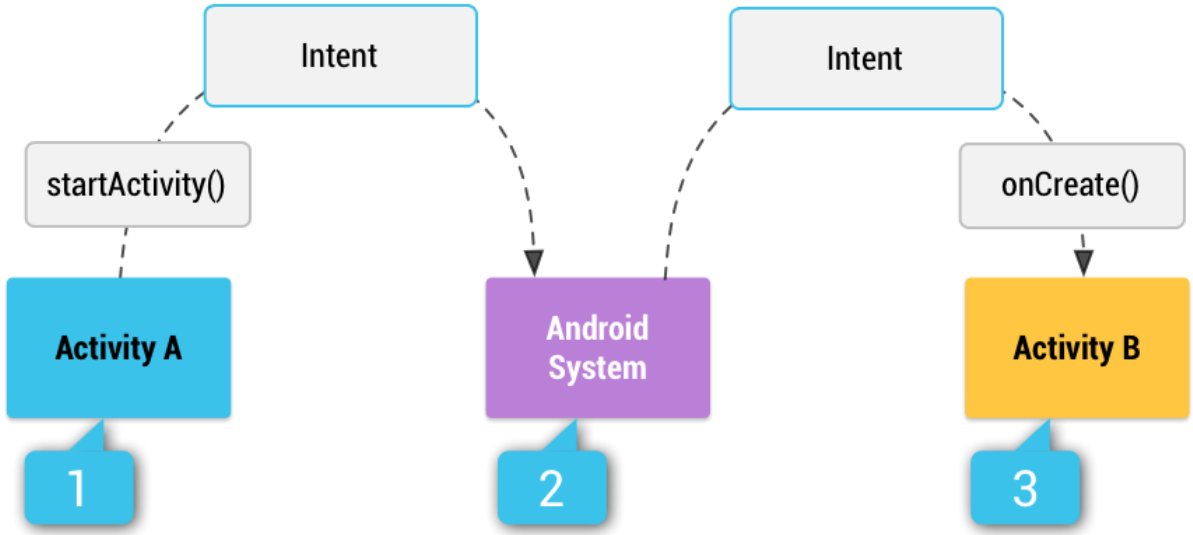
Intents

- Intents can be…
  - Explicit
    - When creating an `Intent`, the type of object class that implements the component to be activated is indicated
  - Implicit
    - When creating the `Intent`, the action to be executed is specified
      - Examples: “show a map”, “send a message”, “take a photo”, ...
    - There is no certainty about the application that will be used to fulfil the request
    - A list with the most common actions is available at

      https://developer.android.com/guide/components/intents-common.html

Intents

- It is the system that will create and start the execution of the component indicated through the *Intent*
  1. A component creates an `Intent` object with the action
  2. The *Android* system searches for the application and component that can satisfy the request
  3. The system starts the component (creates the object and calls the `onCreate` method)
     - The target component receives a reference to the `Intent`



Component activation: Intents

- *Activity*
  - Describe the intended activity through an `Intent` object
  - Pass the `Intent` object through the appropriate methods
    - `Context.startActivity()`, or
    - `Activity.startActivityForResult()`
      - In this form of activation, the eventual result can be accessed through the `onActivityResult` method
  - The launched activity can consult the `Intent` object, that is behind its activation, through the `intent` property
    - In Java the `getIntent()` method should be used

Component activation: Intents

- Examples of using *Intents* to launch activities
  - Example 1:

```
val intent = Intent(this,SomeActivity::class.java)
startActivity(intent)
```

  - Example 2:

```
// Create the text message with a string
val sendIntent = Intent()
sendIntent.action = Intent.ACTION_SEND
sendIntent.putExtra(Intent.EXTRA_TEXT, textMessage)
sendIntent.type = "text/plain"

// Verify that the intent will resolve to an activity
if (sendIntent.resolveActivity(packageManager) != null) {
    startActivity(sendIntent)
}
```

Intents – list of applications

- When there is more than one option to satisfy a request, a list of possible applications is shown
  - The choice is of the user's responsibility

- How to force the list to appear:

```
val sendIntent = Intent(Intent.ACTION_SEND)
val title = "Send to..."
// Create intent to show the chooser dialog
val chooser: Intent = Intent.createChooser(sendIntent, title)
// Verify the original intent will resolve to at least one activity
if (sendIntent.resolveActivity(packageManager) != null) {
    startActivity(chooser)
}
```

Component activation: Intents

- *Activity*
  - One activity can start another and wait for a result
  - Use: `startActivityForResult(<intent>,<intID>)`
    - The `intID` must be a unique *id* for that activity
    - When the second activity finishes, it returns to the first one, and the `onActivityResult` method is called
    - In the parameters of this function, the *id*, the error/success code and an `Intent` with additional data are received
    - *In the practical classes, an alternative method will be studied using Contracts*

Component activation: Intents

- *Service*
  - To activate a Service, an `Intent` object must be passed through the `Context.startService(<intent>)` method

- *Broadcast Receiver*
  - To activate a Broadcast Receiver, an `Intent` object must be passed through the `Context.sendBroadCast(...)`, `Context.sendOrderedBroadcast(...)` or ~~`Context.sendStickyBroadcast(...)`~~ methods

Finishing Android Components

- *Activity*
  - `finish()`
  - One activity can finish another (started through the `startActivityForResult` method) using the `finishActivity()` method
- *Service*
  - `stopSelf()`
  - `Context.stopService()`
- *Broadcast receiver*
  - No need to terminate as it is only active while responding to a message/broadcast
- *Content provider*
  - No need to terminate as it is only active while responding to a request

Manifest file

- File named `AndroidManifest.xml`
- It contains information (*meta-data*) essential to the system, to know…
  - "That the application exists" (follows a certain protocol)
  - The components that make it up
  - Required permissions
  - Minimum device characteristics, …

```
<?xml version="1.0" encoding="utf-8"?>
<manifest . . . >
    <application . . . >
        <activity android:name="com.example.project.FreneticActivity"
                  android:icon="@drawable/small_pic.png"
                  android:label="@string/freneticLabel"
                  . . .  >
        </activity>
        . . .
    </application>
</manifest>
```

Manifest file

- Example of Manifest file configurations
  - Set/request permissions

```
<uses-permission android:name="android.permission.RECEIVE_SMS" />
```

  - Define minimum characteristics

```
<uses-feature android:name="android.hardware.bluetooth" />
<uses-feature android:name="android.hardware.camera" />
```

- Definition of application components
  - *Activity*: `<activity ... />`
  - *Service/IntentService*: `<service ... />`
  - *Broadcast Receiver/Widget*: `<receiver ... />`
  - *Content Provider*: `<provider ... />`

intent-filter

- The *intent-filters* are usually defined in the manifest file, in the context of the component to which they apply
  - Can also be defined at *runtime*

- They are used to inform the system about the type of *intents* that are supported by the component
  - There can be multiple *intents* for each component
  - If a component does not define *intents* then it can only be activated explicitly

intent-filter

- The following example shows two *intent-filters*
  - The first *intent-filter* is common in all applications with the aim of indicating an activity to be activated first (*entry point*)
  - The second *intent-filter* declares a type of action that the activity can perform for a specific type of data

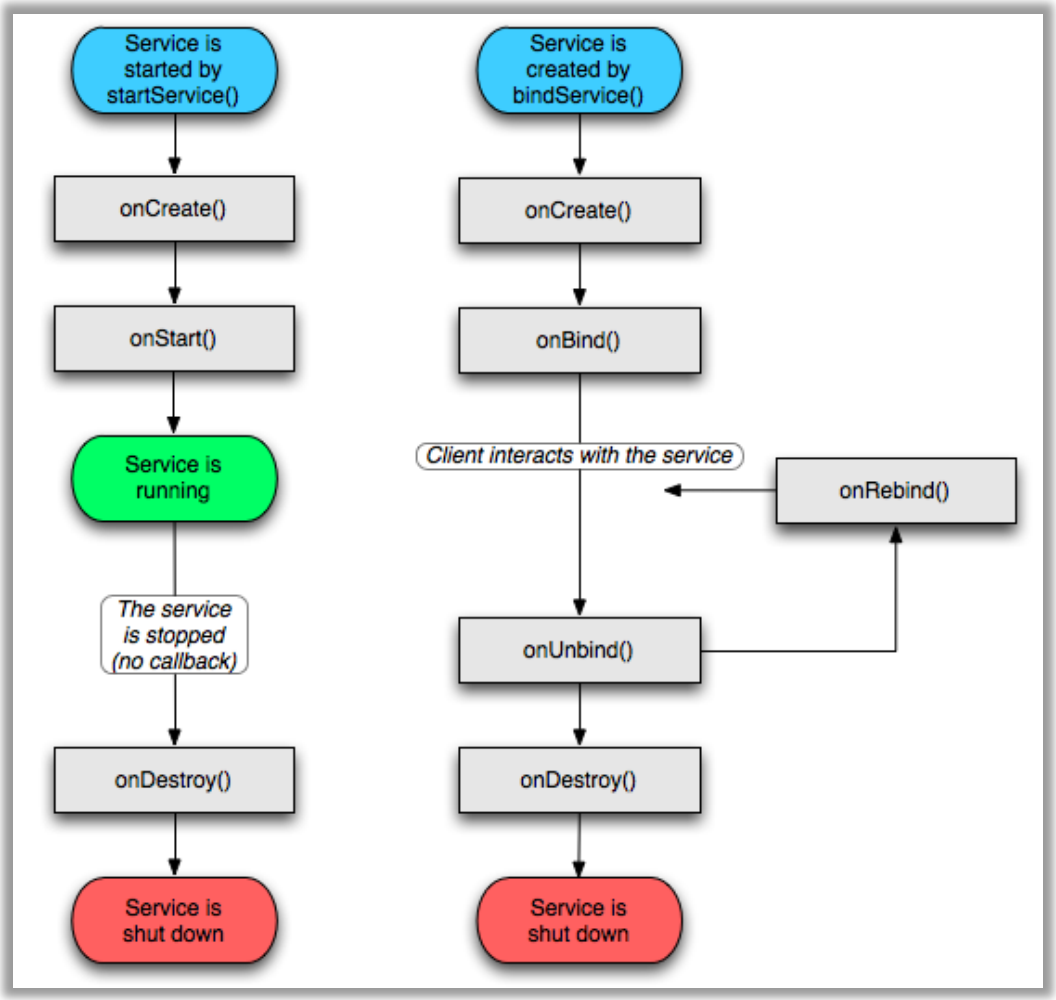
```
<?xml version="1.0" encoding="utf-8"?>
<manifest . . . >
    <application . . . >
        <activity android:name="com.example.project.FreneticActivity"
                  android:icon="@drawable/small_pic.png"
                  android:label="@string/freneticLabel"
                  . . .  >
            <intent-filter . . . >
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
            <intent-filter . . . >
                <action android:name="com.example.project.BOUNCE" />
                <data android:mimeType="image/jpeg" />
                <category android:name="android.intent.category.DEFAULT" />
            </intent-filter>
        </activity>
        . . .
    </application>
</manifest>
```

Lifecycle (BR and CP)

- The lifecycles of *Broadcast Receivers* (BR) and *Content Providers* (CP) correspond to the execution of methods, in response to messages/requests they received
  - They will be analyzed later when studying these components in more depth

Lifecycle (Service)

- The Service's lifecycle will also be studied in more detail later

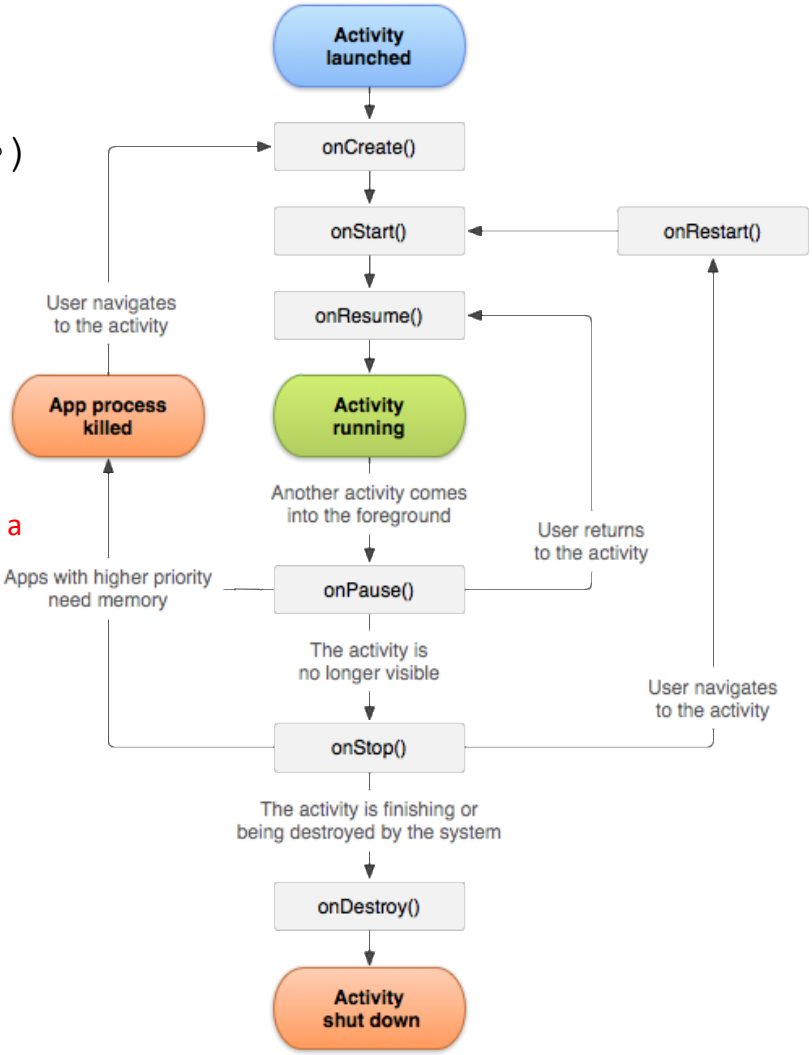


Lifecycle (Activity)

- Possible states:
  - *Created*
    - The activity is created and is available for initial settings
  - *Started*
    - The activity is visible but does not allow interaction
  - *Resumed, Active* or *Running*
    - The activity is in the foreground of the screen and has the focus of the user's commands
  - *Paused*
    - When *focus* is lost (may still be visible)
    - The activity can be terminated by the system if resources are needed
  - *Stopped*
    - Not visible
    - The information is still maintained

Lifecycle of an Activity

- The state change is signaled by calling several methods that can be redefined
  - `fun onCreate(saveInstanceState : Bundle?)`
  - `fun onRestart()`
  - `fun onStart()`
  - `fun onResume()`
  - `fun onPause()`
    - Data should be saved
      - This is the only method that is guaranteed to be called in a finalization process
    - High CPU-consumption processes should be stopped
  - `fun onStop()`
    - It is no longer visible
    - Save data
    - May not be called in case of abrupt termination
  - `fun onDestroy()`
    - May not be called in case of abrupt termination

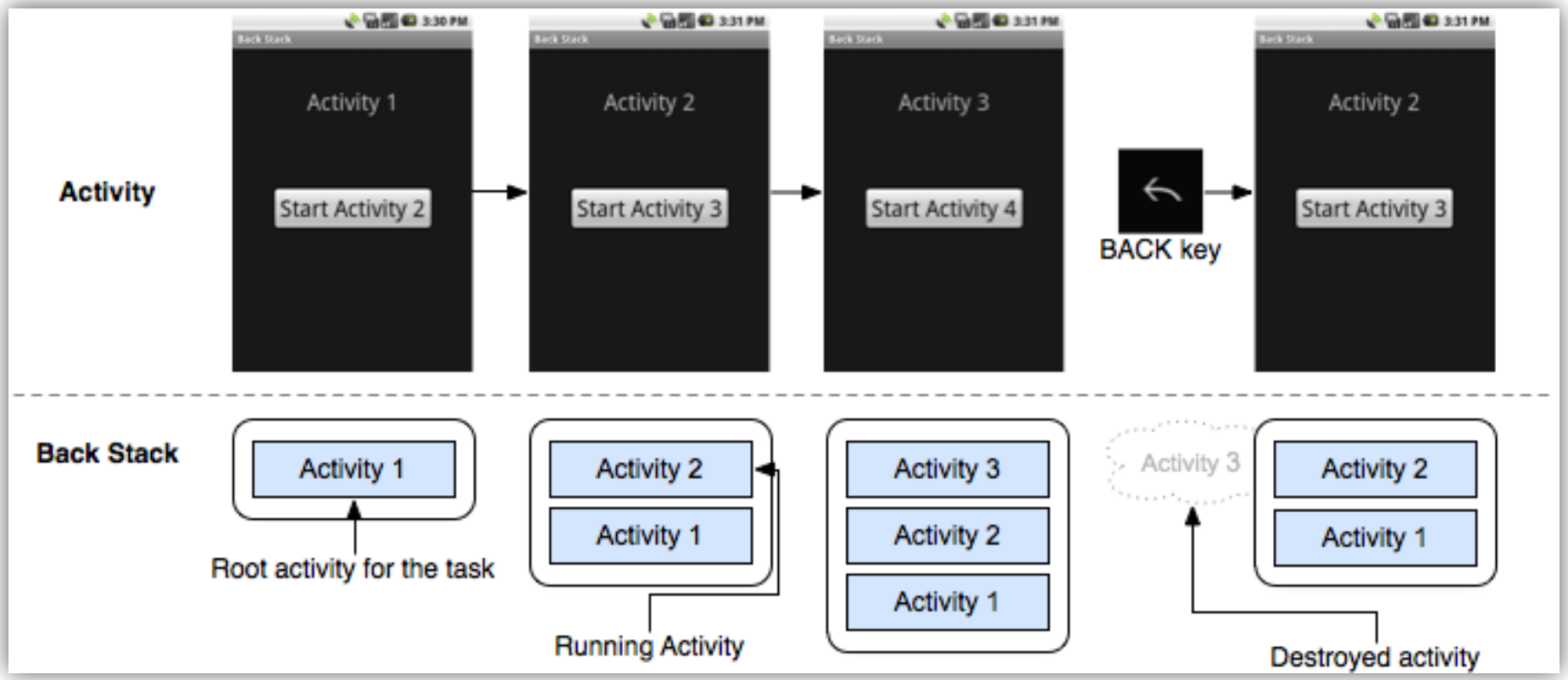


Activities: state persistence

- To save the state and data of an activity the `onSaveInstanceState()` method must be redefined
  - The correspondent base class method must always be called
  - `Bundle` argument is a reference to an object where relevant application data can be temporarily stored
    - Include a set of methods adapted to the primitive and more usual data types: `put<type>(String key,<type> data)`
  - The `Bundle` object is later passed in the `onCreate` parameter and also in the `onRestoresInstanceState` method, when the activity is restarted
  - It is normally used to maintain the state between application restarts by rotating the device, changing its orientation (*portrait vs landscape*)

Back Stack

- When several activities are activated sequentially, this internal system controls the return to the previous activity when the '*Back*' key is used

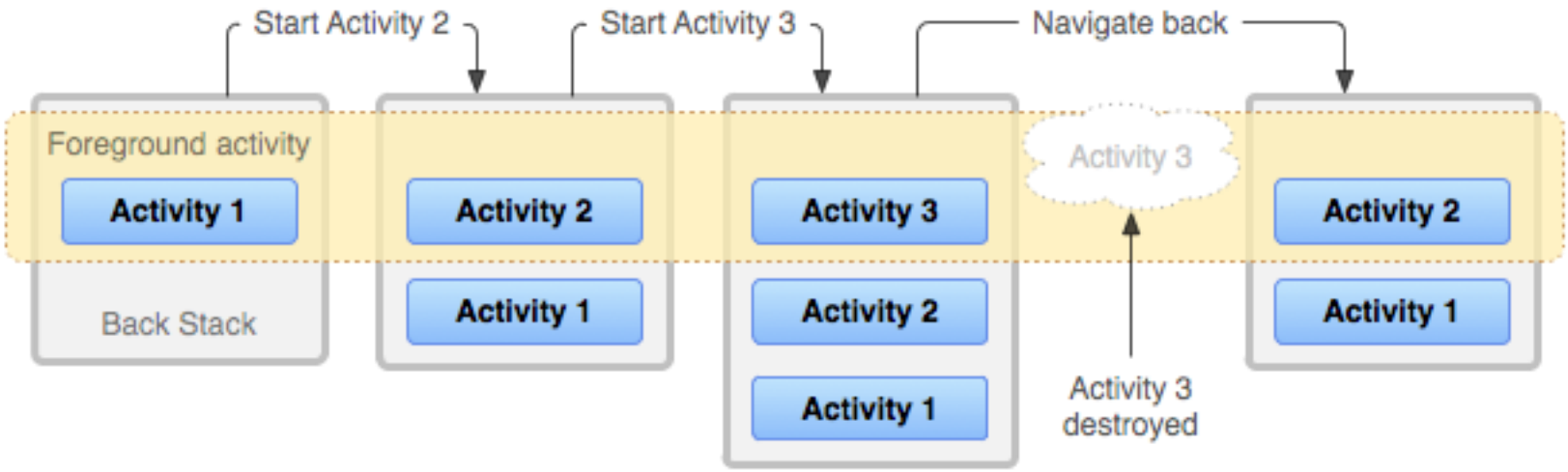


Back Stack and launchMode

- An activity can have the `launchMode` flag/attribute configured with values that will affect how activities are managed when they are started
  - "`standard`"
  - "`singleTop`" (`FLAG_ACTIVITY_SINGLE_TOP`)
    - If the activity is already on top of *Back Stack*, a new instance will not be created
      - The *Back Stack* will not change
  - "`singleTask`"
    - The activity is launched in a new *task*, if there is not already a *task* with the activity in question
    - The activity will be the root of the *task*
    - All activities that "stand in the way" of achieving the activity in question will be destroyed
  - "`singleInstance`"
    - Similar to the previous one, but the activity will be the only one in the *task*
  - "`singleInstancePerTask`"
    - Similar to the previous, but this activity can be started in multiple instances in different tasks
- When an activity is launched, but does not require its creation, instead of the `onCreate` method being called, the `onNewIntent(Intent)` method is called to signal this situation

Back Stack

- The list of recent applications and information about the *back stack* can be consulted using the command-line tool `adb`
- Example:
  - `adb shell dumpsys activity activities`



Processes and user tasks

- By default, all components of an *Android application* run in the context of
  - a process
    - Although the process may contain multiple *threads*, the components are executed within the scope of the main *thread*
  - a *user Task*
    - Set of activities with affinity between them
- An application can force
  - the creation of a given component in a separate process using the `android:process` attribute in the component registry
  - creating activities in different tasks using the `android:launchMode` attribute in the activity registration
  - creating activity groups using the `android:taskAffinity` attribute when registering the activity

Application object

- A process is represented in an *Android* application through an `Application` object
- This object remains valid throughout the lifetime of the application
- There is an `Application` object for each process

Application object

- An application is not required to redefine the `Application` class
  - In this situation, a base `Application` object is instantiated
- To redefine behavior
  - Redefine a new class from the `Application` class
    - Provide implementations for the intended methods (`onCreate`, `onLowMemory`, `onConfigurationChanged`, `onTrimMemory`, …)
  - Declare this new object type via the `android:name` attribute of the `<application .../>` structure in the manifest file
- To access this object in the project context, the `application` property must be used

Log system

- To monitor the execution evolution of different processes on a device the *Logcat* module can be activated…
  - As an additional view in the IDE
  - Through an open connection for this purpose using the `adb` command
    - `adb logcat`

Log message generation

- The task of generating *logs* to facilitate the development process can be carried out with the help of methods available in the `Log` class
  - *Debug*

    ```
    Log.d(tag:String , msg:String)
    ```
  - *Error*

    ```
    Log.e(tag:String , msg:String)
    ```
  - *Info*

    ```
    Log.i(tag:String , msg:String)
    ```
  - *Verbose*

    ```
    Log.v(tag:String , msg:String)
    ```
  - *Warning*

    ```
    Log.w(tag:String , msg:String)
    ```
  - "*What a Terrible Failure*"

    ```
    Log.wtf(tag:String , msg:String)
    ```

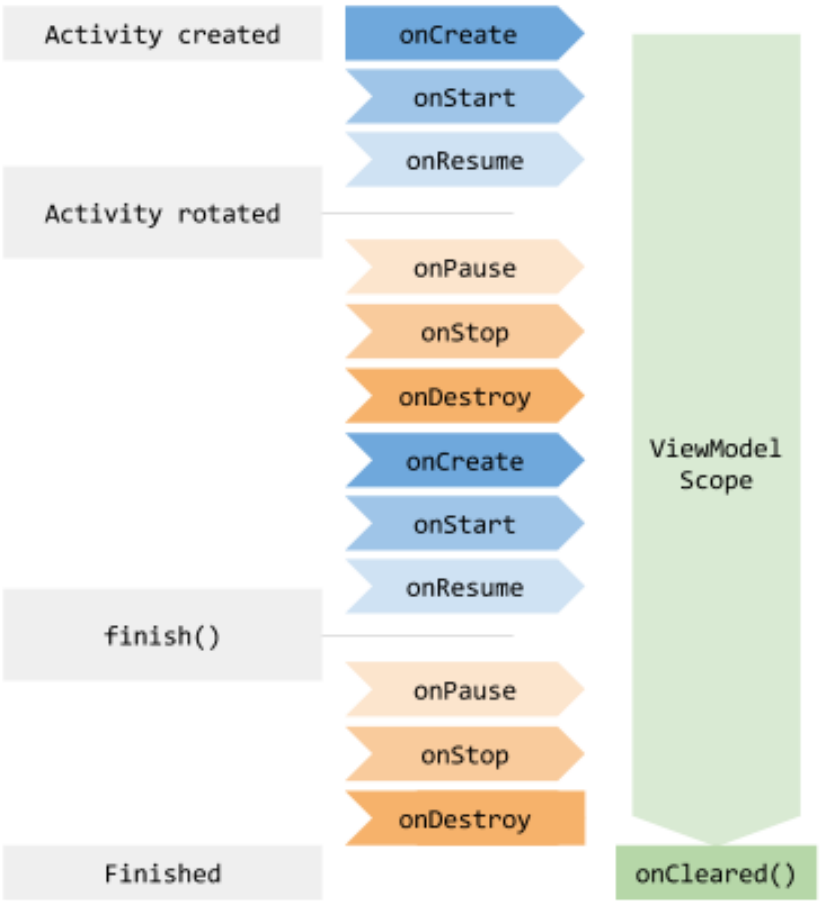
  - There are additional similar versions but with one more parameter corresponding to a `Throwable` object

Activity lifecycle revisited

- As already mentioned, the system can decide in certain situations to eliminate and then recreate components
  - E.g.: when the device is rotated (*Portrait* ⇔ *landscape*)
- Some solutions usually used:
  - Using `onSaveInstanceState` and `onRestoreInstanceState` methods, saving data in the `Bundle` made available for this purpose
  - **`ViewModel`** and **`LiveData`**
    - Will be discussed in upcoming slides
  - Other implementations based on data storage and its recovery
    - With guaranteed data persistence: files, databases, `SharedPreferences`, …
    - Without guaranteeing data persistence: `Application` object or other objects created to implement the data model (such as *singleton* objects)

ViewModel

- The `ViewModel` class allows to implement the *view-model* component of the MVVM *software design pattern*, already studied in *Advanced Programming* subject
- It can be used to temporarily store data of a related component (e. g., an activity)
  - Can be redefined to support implementation of an activity's entire data model
  - The object is not deleted with unexpected changes to the environment, such as screen rotations
  - Must not reference any *View*



ViewModel

- Creating a *View Model* object is done by extending a new class from the `ViewModel` class

```
class MyViewModel : ViewModel() { ... }
```

- The reference to an object of this type is obtained by delegating this functionality, managed internally, to a variable of the created type
  - In `build.gradle.kts (module)`:

```
implementation("androidx.fragment:fragment-ktx:1.6.1")
```

  - In the activity where the view model object will be used:

```
val model : MyViewModel by viewModels()
```

    - In the context of a *fragment*, if it is needed to access the activity's `ViewModel`, then "`by activityViewModels()`" should be used

ViewModel

```
class MyViewModel : ViewModel() {
    private var _value = 0
    val value : Int
        get() = _value++
}

class MainActivity : AppCompatActivity() {

    val viewModel : MyViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Log.i("Test", "onCreate: ${viewModel.value}")
    }

    override fun onResume() {
        super.onResume()
        Log.i("Test", "onResume: ${viewModel.value}")
    }

    override fun onPause() {
        super.onPause()
        Log.i("Test", "onPause: ${viewModel.value}")
    }
}
```

LiveData

- The `LiveData` objects allow to implement the *Observer software design pattern* taking into account the lifecycle of Android components
  - Provides methods for registering *Observers*
  - Only notifies active observers about updates
    - Active observers are those in the STARTED or RESUMED states
  - It has protected methods to change the respective value: `postValue` and `setValue`

- `MutableLiveData`
  - `LiveData` extension that exposes the `postValue` and `setValue` methods, changing them to `public`

ViewModel and MutableLiveData

```
class MyViewModel : ViewModel() {
    private var _value = 0
    val value : Int
        get() = _value++

    private val _state : MutableLiveData<String> by lazy { MutableLiveData<String>("none") }
    val state : LiveData<String>
        get() = _state

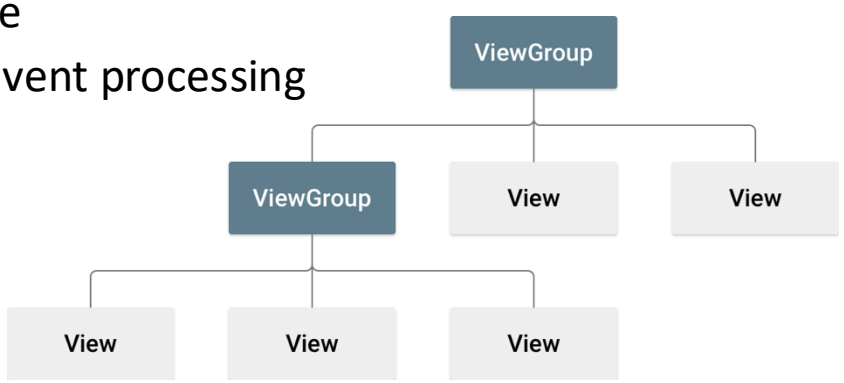
    fun changeState(newState: String) { Log.i("Test", "changeState: $newState $_value ${this.hashCode()}")
        _state.setValue("$newState $value")
    }
}
class MainActivity : AppCompatActivity() {
    val viewModel : MyViewModel by viewModels()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        viewModel.state.observe(this) {  Log.i("Test", it)  }
        viewModel.changeState("onCreate ${this.hashCode()}")
    }
    override fun onResume() {
        super.onResume()
        viewModel.changeState("onResume ${this.hashCode()}")
    }
    override fun onPause() {
        super.onPause()
        viewModel.changeState( "onPause ${this.hashCode()}")
    }
    override fun onStop() {
        super.onStop()
        viewModel.changeState( "onStop ${this.hashCode()}")
        //viewModel.state.removeObservers(this)
    }
}
```

```
changeState: onCreate 251924405 0 147911348
onCreate 251924405 0
changeState: onResume 251924405 1 147911348
onResume 251924405 1
changeState: onPause 251924405 2 147911348
onPause 251924405 2
changeState: onStop 251924405 3 147911348
changeState: onCreate 242710700 4 147911348
onCreate 242710700 4
changeState: onResume 242710700 5 147911348
onResume 242710700 5
changeState: onPause 242710700 6 147911348
onPause 242710700 6
changeState: onStop 242710700 7 147911348
changeState: onCreate 91623694 8 147911348
onCreate 91623694 8
changeState: onResume 91623694 9 147911348
onResume 91623694 9
```

User interface

- Android application UIs are created based on *views*, *view groups* and their derivatives
  - `View`
    - Basic objects that make up the interface, which allow:
      - To display information or object state
      - To accept user interaction through event processing
  - `ViewGroup`
    - Derive from `View`
    - Implement *container* features
    - Can contain `View` or `ViewGroup` objects
    - These are the base class for layout objects



- The objects constitute a hierarchy that will be assigned to an `Activity` through the `setContentView` method, called in the `onCreate` method

User Interface

- Objects can be created in two ways:
  - Programmatically
    - Creating `View` objects or their derivatives
    - Forming a hierarchy of *views* and assigning it to the activity
  - Built from XML files
    - The application layout is defined through a XML file
    - When associated with the activity, information is automatically read from the file, and objects are created and configured one by one through an 'inflate' operation

UI created programmatically

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    val ll = LinearLayout(this)
    val tv = TextView(this)
    tv.text = "Arq. Móveis"
    ll.addView(tv)
    val btn = Button(this)
    btn.text = "Ok"
    ll.addView(btn)
    btn.setOnClickListener {
        Toast.makeText(
            this,
            "Teste", Toast.LENGTH_LONG
        ).show()
    }
    setContentView(ll)
}
```

UI defined through XML

- Each structure/*tag* defined in XML will later lead to the creation of the corresponding *Android* object

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical" android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Arq. Móveis"
        android:id="@+id/textView"
        android:textSize="28sp"
        android:gravity="center_horizontal" />

    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="New Button"
        android:id="@+id/button"
        android:layout_gravity="center_horizontal" />
</LinearLayout>
```

XML Layouts

- The XML files which represent the layouts must be saved in the project directory: `res/layout`
- The development environment provides a user-friendly interface for building and editing layouts
- Later, the layouts are loaded using identifiers that are automatically assigned to the different resources

XML Layouts

- Assuming that a layout file is saved with the name `main_layout.xml` then the identifier `R.layout.main_layout` will be automatically created
  - To read the layout and to create the corresponding objects, the following command is used:

```
setContentView(R.layout.main_layout)
```

  - As mentioned, this operation is called an 'inflate' operation

Attributes

- Objects have a set of attributes for their configuration
- One of the most important attributes, to enable subsequent access to the created object, is the one that allows to assign an identifier to it
  - Although this identifier is an integer, in XML it is defined as follows
    - `android:id="@+id/objectID"`
      - The `@` indicates that it is a resource *ID*
      - The `+` indicates that it is a new *ID*
  - Using the created ID, we can access the corresponding object

```
<Button android:id="@+id/my_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/my_button_text"/>
```

```
val myButton: Button = findViewById(R.id.my_button)
```

Attributes

- There are two attributes that allow to indicate the height and width of a `View`, which are mandatory for most components
  - `android:layout_height`
  - `android:layout_width`

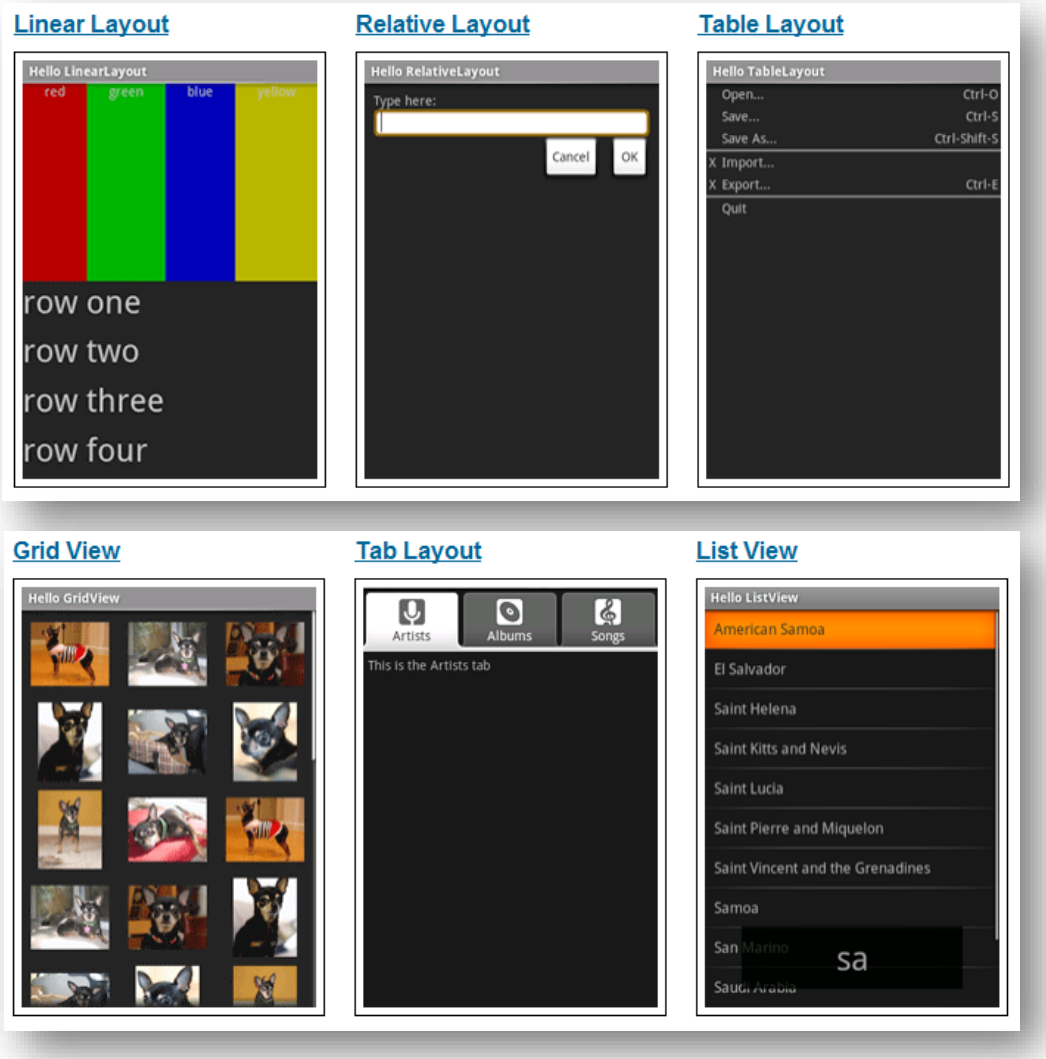
  - These attributes can take on the following values:
    - `wrap_content`
    - `match_parent` (previously: `fill_parent`)
    - A specific value for the height and/or width in the format <*value*> <unit>
      - The accepted units are as follows: *px (pixels), dp (density-independent pixels), sp (scaled pixels based on preferred font size), in (inches), mm (millimeters)*

```
<Button android:id="@+id/my_button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/my_button_text"/>
```

Layout objects

- The *layout* objects help to structure and arrange various elements on the screen
- There are various *layout* objects (`ViewGroup` object extensions) which allow different ways of organizing elements
    - `FrameLayout`
    - `LinearLayout`
    - `RelativeLayout`
    - `TableLayout`
    - `GridLayout`
    - ~~`AbsoluteLayout`~~
    - …
    - `ConstraintLayout` *(Android Jetpack)*

Layout objects



LinearLayout

- The `LinearLayout` enables the organization of objects in a horizontal or vertical sequence
- The alignment of objects on the screen can be configured in several aspects
  - Space occupation
    - `layout_width`
    - `layout_height`
  - Positioning
    - `layout_gravity`
      - `center, left, start, right, end, top, bottom, …`
      - Can be a combination of values (e. g., `top|end`)
  - Percentage of space occupied
    - Normalized distribution taking into account the values present in `layout_weight`

RelativeLayout

- Enables the organization of objects by defining their dependencies/relationships
  - Object A is aligned to the left of Object B
  - Object C is below Object D
  - Object E is aligned to the bottom margin
  - Object F is aligned to the right of Object G

TableLayout

- The `TableLayout` allows to organize objects in rows
  - Similar to the behavior of tables in HTML
- A `TableLayout` object is made up of one or more `TableRow`
- When used individually, the `TableRow` object has a behavior analogous to the horizontal `LinearLayout`
- The objects included in a `TableRow` do not need to have the `layout_width` and `layout_height` properties defined
  - They are automatically defined to adapt to the size of existing rows and columns
  - For each column, the largest width of the field in that column is used, along all lines

FrameLayout

- The `FrameLayout` is a *layout* designed to display just one item
  - Several items can be included in the `FrameLayout` as long as they are properly aligned to the different margins or to the center, using the `layout_gravity` property

- `ScrollView`
  - It derives from `FrameLayout` and allows to provide a larger space than is occupied on the screen, through the use of *scrollbars*
    - Inside it, only one item must be placed, although it can be a `ViewGroup` with several items

Processing view events

- In addition to the events generated on the activity (some of them were already tried while studying the lifecycle of an activity: `onCreate`, `onPause`, ...) the components that form the activity interface can be the target of actions, which generate events
  - Events are processed in the context of specific methods, for example:
    - `onDraw`
    - `onFocusChanged`
    - `onKeyDown`
    - `onKeyUp`
    - `onTouchEvent`

Processing view events

- There are events that are only processed if the application expresses interest in them
  - To register this “interest”, appropriate functions are used
    - Typical format: `setOn<event>Listener`
  - These functions allow to indicate the *Listeners* in the context of which events are processed
  - A *Listener* corresponds to one or more functions (depending on the group of events to be processed) declared through a suitable interface *Java*/*Kotlin*

View

- Example for processing a button press:

```
val btn : Button = findViewById(R.id.btnOk)
btn.setOnClickListener(ObjectOnClickListener)
```

  - The *`OnClickListener`* object can be implemented using:
    - Interface implementation in the class itself (*this*)
    - Implementation of the interface within the scope of a class created for this purpose
    - Instantiation of an object through an anonymous class
    - Inline implementation of an object from an anonymous class – *Lambda function*

Resources

- An Android app can include resources of several types
  - *Layouts*
  - *Drawable*
  - *Menus*
  - *Values*
  - …
- The same resource can be defined multiple times to adapt to the environment in which it will be viewed
  - Portrait/Landscape
  - Language
  - Screen resolution
  - …

Resources

- When including multiple versions of the same *resource* into the project, they must have the same name
  - Non-overlapping is guaranteed because each file is stored in a directory distinct from the others
  - Each directory includes a qualifier in its name that distinguishes it from the rest
    - The qualifier reflects the differentiating characteristics from the others (language, screen orientation, …)
- The choice of the file with the appropriate *resource* for a given context is carried out by the *Resource Manager*

Resources

- For example, the *drawables* can include *resources* with the same name, e.g. `icon.ico`, in several subfolders corresponding to possible resolutions
  - `drawable-ldpi`
    - Low point density
  - `drawable-mdpi`
    - Medium density
  - `drawable-hdpi`
    - High density
  - `drawable-xhdpi`, `drawable-xxhdpi`, …
    - Extra density
  - `drawable`
    - To be used when the characteristics of the execution environment don't match any of the more specific versions (default versions of the resources)

Resources

- In the same way that different *resources* can be made available according to the density of the pixels, appropriate resources can be defined based on
  - Language
    - For example, to translate an application to Portuguese…
      - make sure that all strings used in the app are defined in the `res/values/strings.xml` file
        - This is the default file, and the strings should be in English
      - additionally, define the same strings (with the same identifiers) in the `res/values-pt/strings.xml`, but in Portuguese
    - Strings are accessed using
      - In layout : `@string/<id_string>`
      - In Java/Kotlin : `Context.getString(R.string.<id_string>)`

      …and the *Resource Manager* will return the string in the appropriate language
  - Screen orientation
    - Example: defining suitable layouts for portrait or landscape in the folders `res/layout` and `res/layout-land`
  - …

Menus

- Menus are useful for providing access to common functions
- Menus must be created in the method:

```
fun onCreateOptionsMenu(menu:Menu?)
```

  - Dynamically, using methods from the `Menu` class
  - Defined through an XML file

- The chosen options must be processed in the `onOptionsItemSelected` method, which receives a reference to the selected item as a parameter



Menus

- Example of defining the menu options using an XML file

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">

    <item android:id="@+id/new_game"
          android:icon="@drawable/ic_new_game"
          android:title="@string/new_game" />

    <item android:id="@+id/help"
          android:icon="@drawable/ic_help"
          android:title="@string/help" />

</menu>
```

Menus

- Menu configuration

```
override fun onCreateOptionsMenu(menu: Menu?): Boolean {
    menuInflater.inflate(R.menu.mymenu,menu)
    return true
}
```

- Processing selected options

```
override fun onOptionsItemSelected(item: MenuItem): Boolean {
    when(item.itemId) {
        R.id.new_game -> newGame()
        R.id.help     -> showHelp()
        else -> return super.onOptionsItemSelected(item)
    }
    return true
}
```

Menus in the Action Bar

- From version 3 onwards, the title bar started to be used which is called *Action Bar*
  - includes the application *icon (1)*
  - allows to make some options available (2)
  - allows access to the menu (3)




```xml
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/new_game"
          android:icon="@drawable/ic_new_game"
          android:title="@string/new_game"
          android:showAsAction="ifRoom"/>
    <item android:id="@+id/help"
          android:icon="@drawable/ic_help"
          android:title="@string/help" />
</menu>
```

Force the Action Bar and title

- In the manifest file put:

```
<application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme=
      "@android:style/Theme.DeviceDefault.Light.DarkActionBar">

   ...

</application>
```

- Note: other values may be used for the *theme*
  - Example: `Theme.Holo.Light.DarkActionBar`
  - The `AppCompactActivity` also gives support to the *Action Bar*

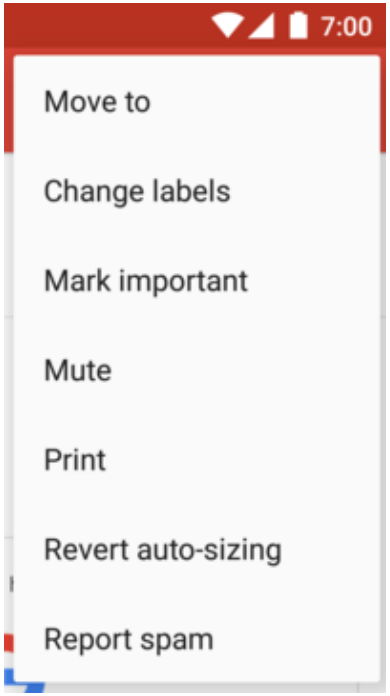
"Classic" context menus

- The context menus are the ones that are usually activated through a right-click in PC programs
- In Android apps, and in general mobile apps, these menus are activated with a “long press”, and will be adapted to the different objectives of the components
  - Process the `onCreateContextMenu` method
    - Similar to `onCreateOptionsMenu`
    - It has more parameters to provide information about the component to which it will be applied
  - Process the `onContextItemSelected` method
    - Similar to `OnOptionsItemSelected`
  - Register the *Views* for which context menus will be associated, using the `registerForContextMenu` method

Context menus and PopupMenu



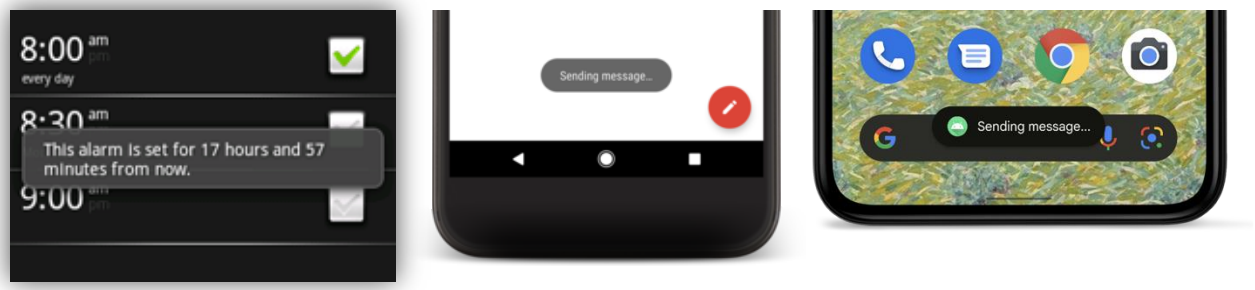






Toast message generation

- There is a UI element, called *toast*, that allows showing status evolution messages to the user, without interrupting the execution of other applications



- `Toast` class
  - It has a `makeText` method that allows the creation of this type of message
    - Messages can be viewed during two pre-defined periods in the system, short and long
    - Messages are displayed using the `show()` method

Toast message generation

- A more configurable toast can be created, which allows to define customizable *layouts*, by following these steps:
  - Define a new *layout resource* (e. g., `tstexemplo.xml`)
  - Use a `LayoutInflater` object to create instances of the objects defined in the layout

    ```
    val layout = layoutInflater.inflate(R.layout.tstexemplo, null)
    ```

  - Create a `Toast` object and associate it with the created object structure

    ```
    val toast = Toast(applicationContext)
    toast.setGravity(Gravity.CENTER_VERTICAL, 0, 0)
    toast.duration = Toast.LENGTH_LONG
    toast.view = layout
    ```

  - Display the created `Toast` object

    ```
    toast.show()
    ```

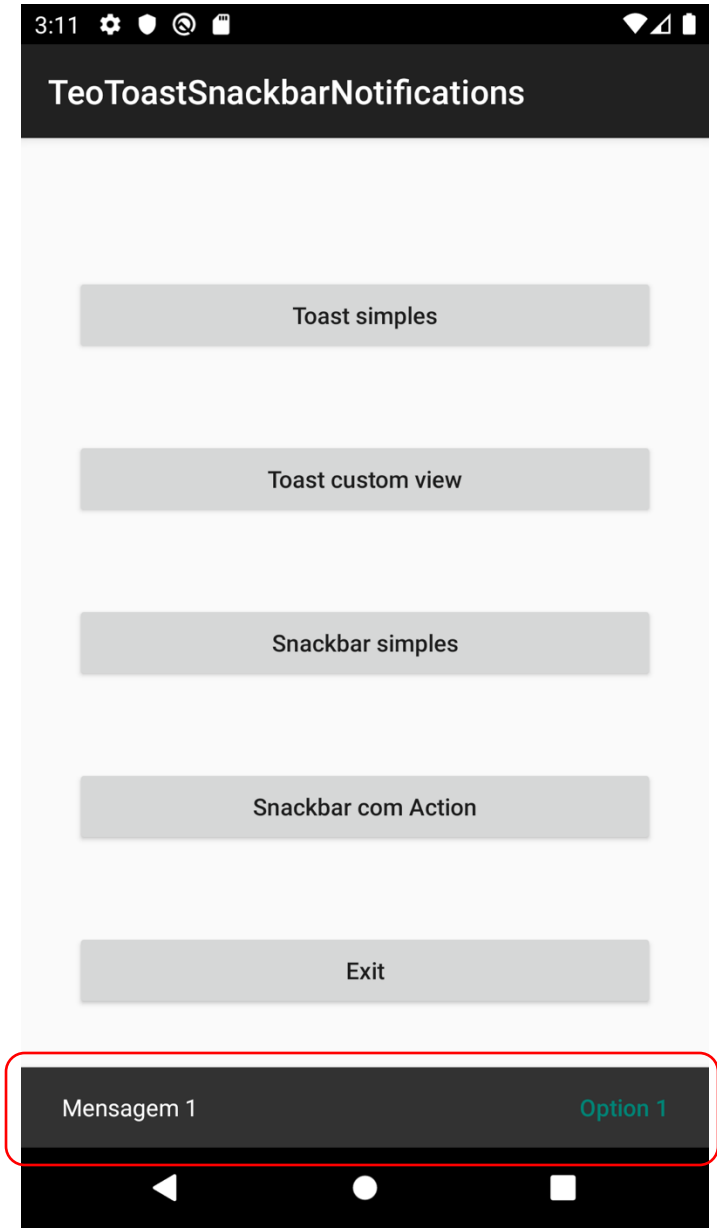
SnackBar

- A *snack bar* allows to display a text message at the bottom of the screen
- The message may have an action associated with it, which will be executed when pressed
- Like a *toast*, the *snack bar* will be shown on the screen for a short or long period (`Snackbar.LENGTH_SHORT` or `Snackbar.LENGTH_LONG`)
- It also allows it to remain permanent (`Snackbar.LENGTH_INDEFINITE`) which will be hidden after touching the action or through an explicit call to the `dismiss` function
- Component made available through the *Google Material Library*

```
implementation("com.google.android.material:material:1.12.0")
```

- Created with the `make` method

```
Snackbar.make(view,R.string.msg,Snackbar.LENGTH_LONG).show()
```

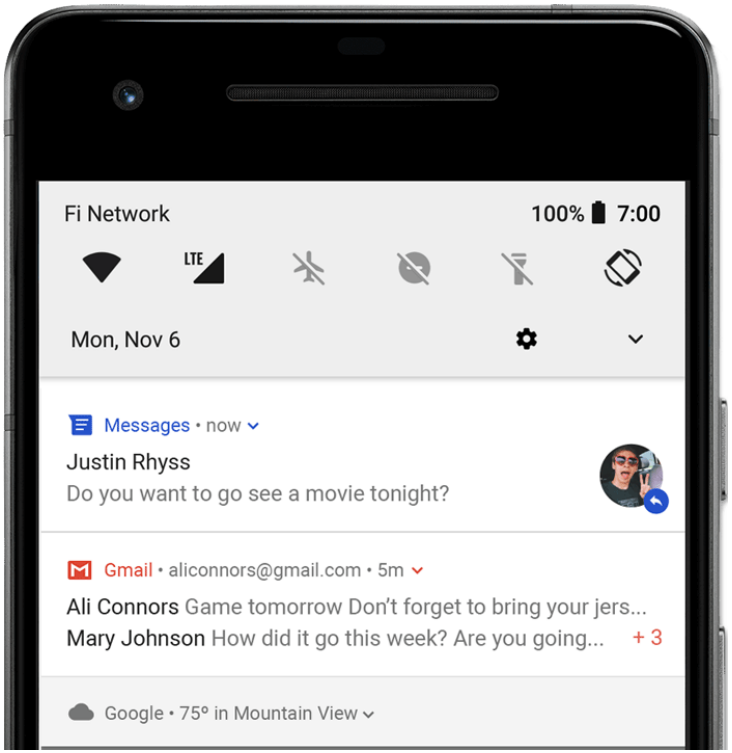


Status notification bar

- The status bar appears at the top of the screen

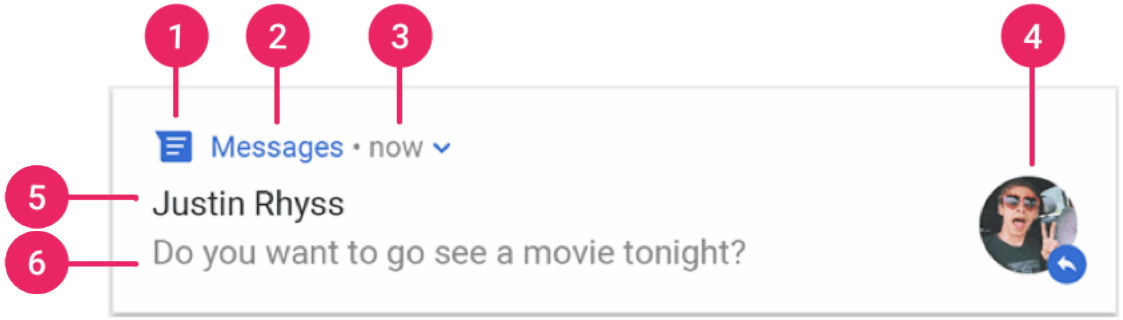


- Sliding the bar downwards brings up the notification window



Status notification bar

- Elements of a notification



1. Small icon
2. App name
3. Time stamp
4. Large icon
5. Title
6. Text

Notification channels

- Starting from *API 26*, it is mandatory to associate notifications with "notification channels" for categorization purposes
  - This allows the user to activate/deactivate notification channels, through the application settings
  - Channels are created by the application itself
    - Channel creation is not available in versions with API lower than 26, nor is it available via *Support Library*
    - The channel creation code must be protected so as not to run on older versions

Creating a channel

- Minimum information to create a channel

```
val channel_id = "amov_channel"
val channel_name = "AMov Channel"
val channel_importance = NotificationManager.IMPORTANCE_DEFAULT
```

- Channel creation

```
val notifMgr = getSystemService(NOTIFICATION_SERVICE) as NotificationManager
if (android.os.Build.VERSION.SDK_INT >= android.os.Build.VERSION_CODES.O) {
    val channel=NotificationChannel(channel_id,channel_name,channel_importance)
    notifMgr.createNotificationChannel(channel)
}
```

Creating notifications

- Example of basic information to create the notification

```
val notif_id = 1234
val title = "AMOV - Título"
val text  = "AMov - Texto"
val small_icon = android.R.drawable.ic_dialog_email
val priority = NotificationCompat.PRIORITY_DEFAULT
```

  …and the `channel_id` from the previous slide

- Notification creation

```
val notification = NotificationCompat.Builder(this,channel_id)
    .setSmallIcon(small_icon)
    .setContentTitle(title)
    .setContentText(text)
    .setPriority(priority)
    .setAutoCancel(true)
    .build()
```

- Display notification

```
notifMgr.notify(notif_id,notification)
```

Clicks on notifications

- To allow the application to be reactivated when a notification is clicked, a `PendingIntent` must be associated when it is created

```
val intent = Intent(this,MainActivity::class.java).apply {
    flags = Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK
}
val pendingIntent = PendingIntent.getActivity(this,0,intent,0)

val notification = NotificationCompat.Builder(this,channel_id)
    .setSmallIcon(small_icon)
    .setContentTitle(title)
    .setContentText(text)
    .setPriority(priority)
    .setContentIntent(pendingIntent)
    .setAutoCancel(true)
    .build()

notifMgr.notify(notif_id,notification)
```

Notification permissions

- Starting from *API 33*, it is mandatory to ask permission to post notifications
  - Manifest

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

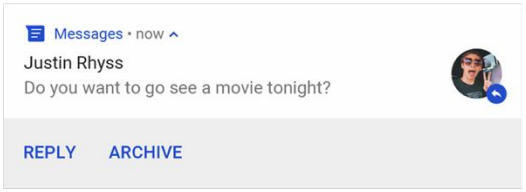
  - Activity

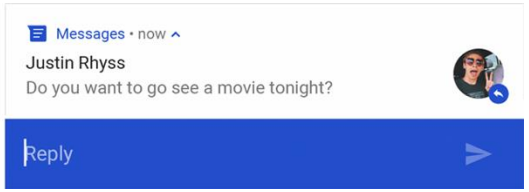
```
...
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
     requestNotificationPermission.launch(
         android.Manifest.permission.POST_NOTIFICATIONS
     )
}
...

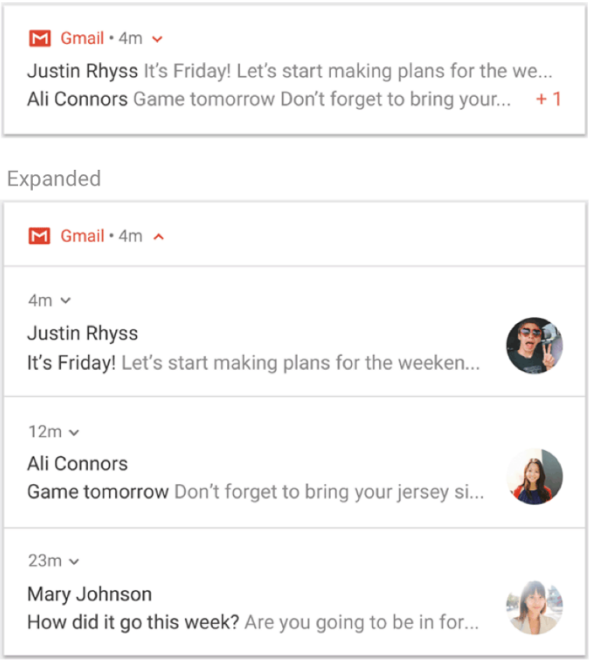
private val requestNotificationPermission = registerForActivityResult(
        ActivityResultContracts.RequestPermission(),
        { granted ->
            //some code
        }
)
```

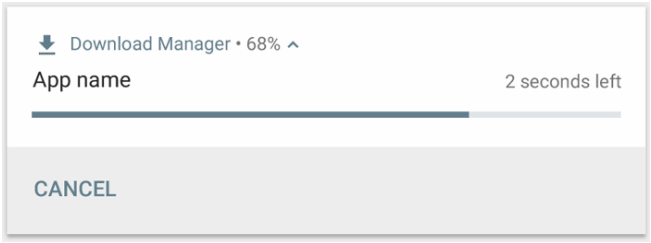
Other types of notification

- There are many other notification possibilities that can be configured: customizable actions, long/expandable notifications, notification accumulation, progress bars, lock screen notifications, sounds, vibration, ...









- See also:
  - https://developer.android.com/training/notify-user/build-notification
  - https://developer.android.com/training/notify-user/expanded
  - https://developer.android.com/training/notify-user/group

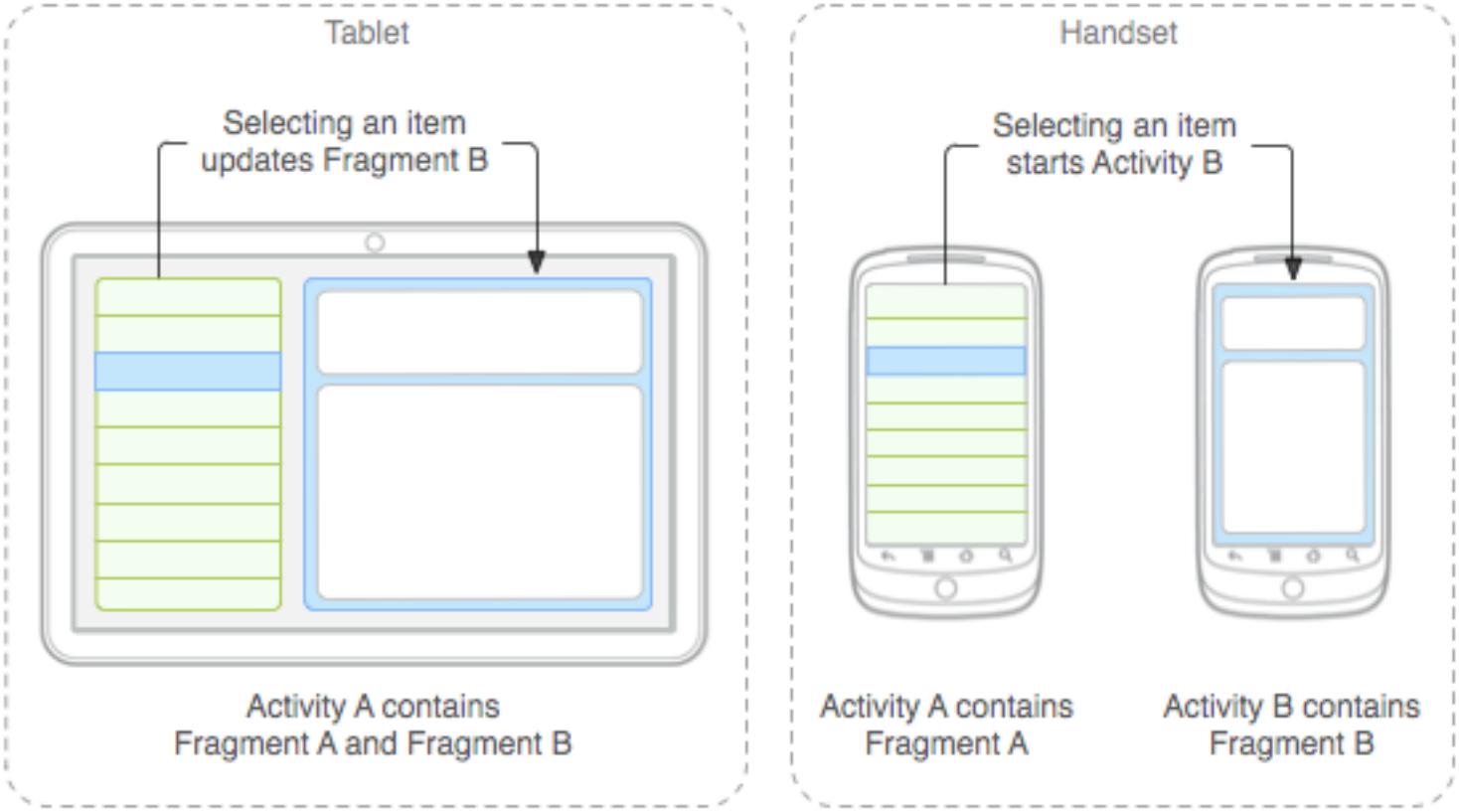
Fragment

- Fragments allow the UI definition by blocks
  - Fragments can be used in the context of different activities
    - To adapt to the size of the devices
    - To adapt to the needs of the application

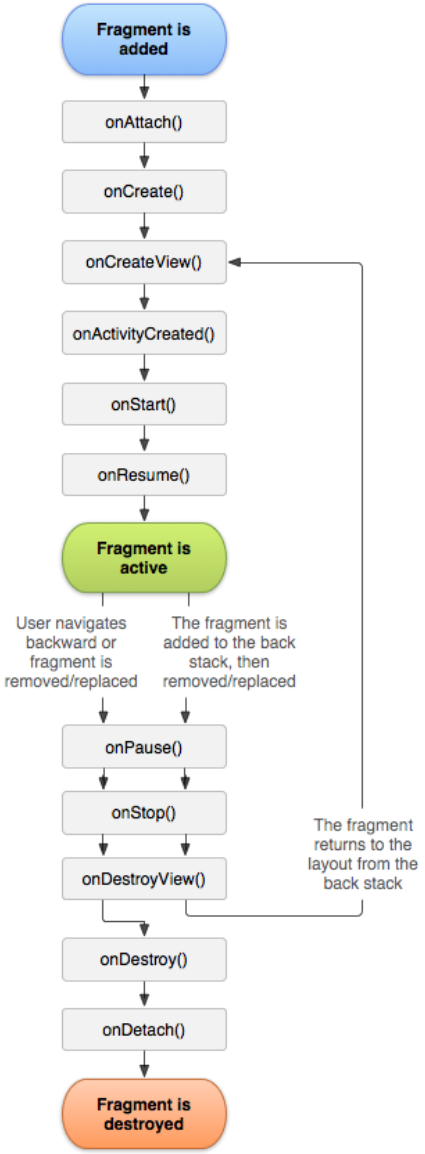
- They also allow to encapsulate the processing of the visual components of fragments

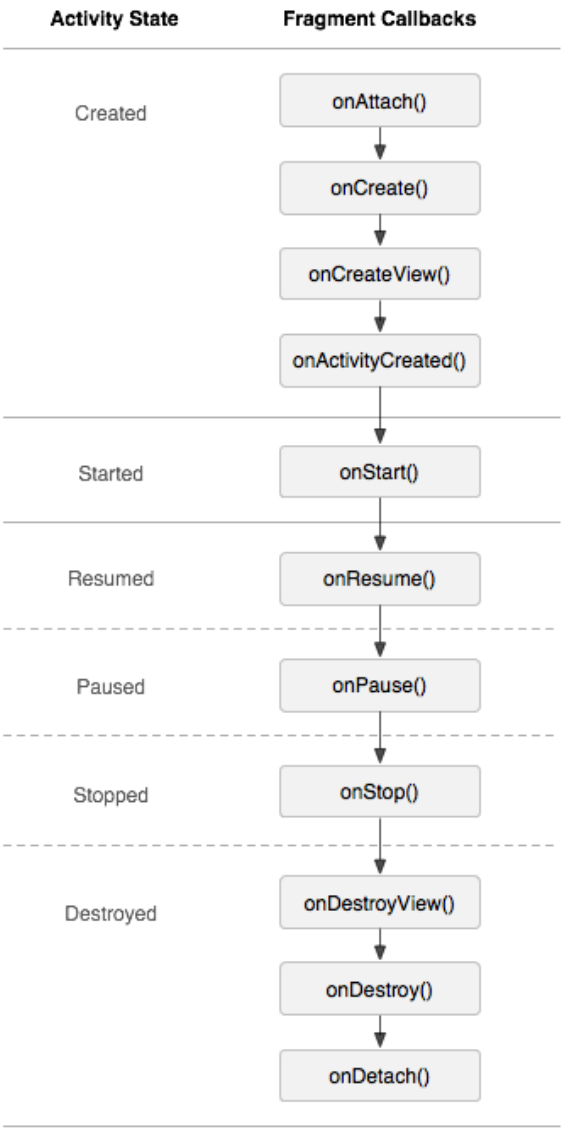
  - Note:
    - The activities layout can be defined including partial layouts that are described in independent layout files, using the `include` XML directive, however the event processing would have to be repeated in each activity

Fragment adaptation



Fragment Lifecycle





Definition of a Fragment

- The definition of a fragment is very similar to the definition of an activity, including…
  - the layout definition
    - Through an XML file
    - Through a component creation code (programmatically)
  - the definition of a new class as an extension of the `Fragment` class

Use of fragments

- Fragments are included in the activity context
  - For versions before *API 11*, `FragmentActivity` should be used as base class for the activities that will include fragments
  - To avoid API level checking, the `AppCompactActivity` available through *Support Library* can also be used

- There are two main ways to use/include fragments into activities
  - Defining them in the layout XML files
  - Programmatically

Example with XML

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <fragment android:name="com.example.news.ArticleListFragment"
            android:id="@+id/list"
            android:layout_weight="1"
            android:layout_width="0dp"
            android:layout_height="match_parent" />
    <fragment android:name="com.example.news.ArticleReaderFragment"
            android:id="@+id/viewer"
            android:layout_weight="2"
            android:layout_width="0dp"
            android:layout_height="match_parent" />
</LinearLayout>
```

Example of programmatic form

- Adding a new fragment

```
val fragmentManager = supportFragmentManager
val fragmentTransaction = fragmentManager.beginTransaction()
```

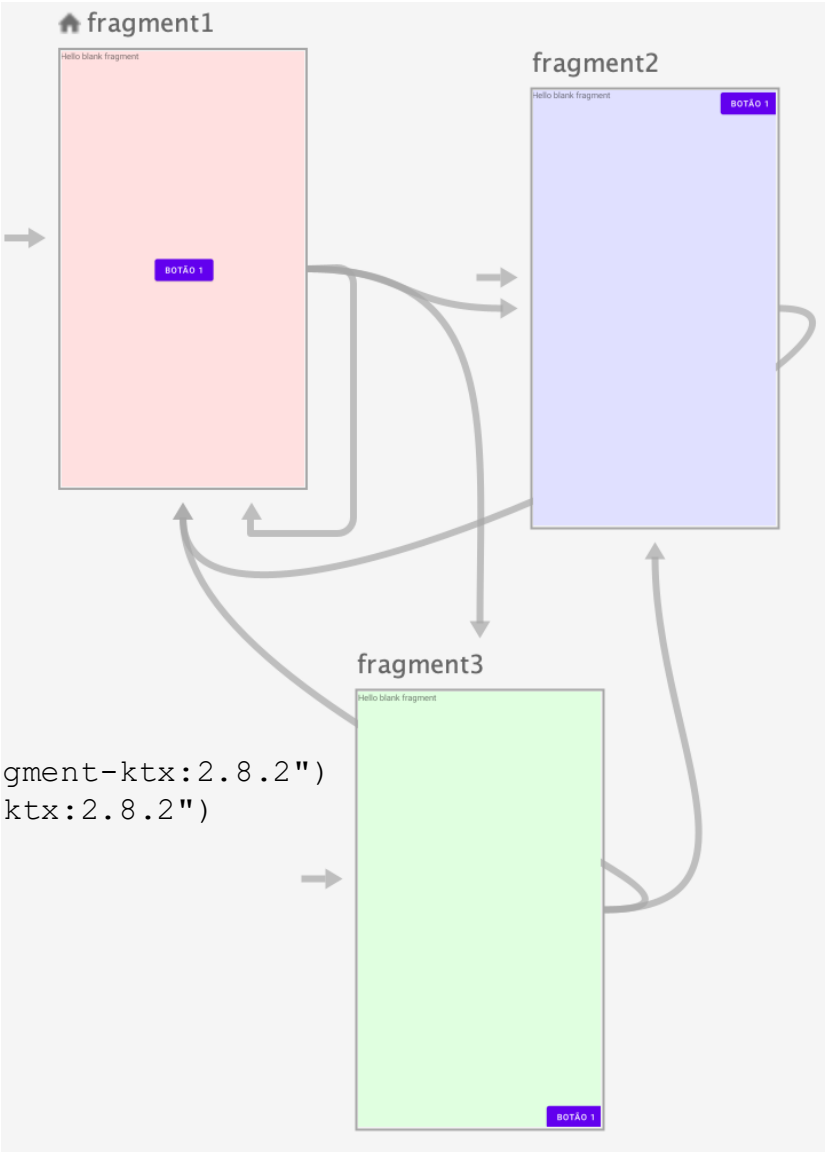
```
val fragment = ExampleFragment()
fragmentTransaction.add(R.id.fragment_container, fragment)
fragmentTransaction.commit()
```

- Change a fragment

```
val newFragment = ExampleFragment()
val transaction = supportFragmentManager.beginTransaction()
transaction.replace(R.id.fragment_container, newFragment)
transaction.addToBackStack(null)
transaction.commit()
```

Navigation Controller

- With the *Android Jetpack*, a *fragment controller* (which is also a fragment) is provided to assist in the task of managing the active fragment among a set of fragments, represented through a graph




```
dependencies {
   ...
   implementation("androidx.navigation:navigation-fragment-ktx:2.8.2")
   implementation("androidx.navigation:navigation-ui-ktx:2.8.2")
   ...
}
```

Navigation Controller

```
<!-- Activity with a NavHostFragment -->

<LinearLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  android:layout_width="match_parent"  android:layout_height="match_parent"
  android:orientation="vertical" >

  <TextView
    android:id="@+id/tvMsg1"
    android:gravity="center"
    android:layout_width="match_parent"    android:layout_height="wrap_content"
    android:text="Início" />

  <fragment
    android:layout_margin="16dp"
    android:layout_width="match_parent"    android:layout_height="0dp"
    android:layout_weight="1"
    android:id="@+id/fragment_base"
    android:name="androidx.navigation.fragment.NavHostFragment"
    app:defaultNavHost="true"
    app:navGraph="@navigation/base_navigation" />

  <TextView
    android:id="@+id/tvMsg2"
    android:gravity="center"
    android:layout_width="match_parent"    android:layout_height="wrap_content"
    android:text="Fim" />

</LinearLayout>
```

Navigation Controller

```
<!- Navigation resource example: base_navigation.xml -->
<navigation
  xmlns:android="http://schemas.android.com/apk/res/android"
  xmlns:app="http://schemas.android.com/apk/res-auto"
  xmlns:tools="http://schemas.android.com/tools"
  android:id="@+id/base_navigation"
  app:startDestination="@id/fragment1">

  <fragment android:id="@+id/fragment1"
    android:name="pt.isec.ans.teonavigationcontroller.Fragment1"
    android:label="fragment_1"
    tools:layout="@layout/fragment_1">
    <action android:id="@+id/action_fragment1_to_fragment2"  app:destination="@id/fragment2" />
    <action android:id="@+id/action_fragment1_to_fragment3"  app:destination="@id/fragment3" />
    <action android:id="@+id/action_fragment1_self"          app:destination="@id/fragment1" />
  </fragment>

  <fragment android:id="@+id/fragment2"
    android:name="pt.isec.ans.teonavigationcontroller.Fragment2"
    android:label="fragment_2"
    tools:layout="@layout/fragment_2">
    <action android:id="@+id/action_fragment2_to_fragment1"  app:destination="@id/fragment1" />
  </fragment>

  <fragment android:id="@+id/fragment3"
    android:name="pt.isec.ans.teonavigationcontroller.Fragment3"
    android:label="fragment_3"
    tools:layout="@layout/fragment_3">
    <action android:id="@+id/action_fragment3_to_fragment2"  app:destination="@id/fragment2" />
    <action android:id="@+id/action_fragment3_to_fragment1"  app:destination="@id/fragment1" />
  </fragment>

  <action android:id="@+id/action_global_fragment1"   app:destination="@id/fragment1" />
  <action android:id="@+id/action_global_fragment2"   app:destination="@id/fragment2" />
  <action android:id="@+id/action_global_fragment3"   app:destination="@id/fragment3" />
</navigation>
```

Navigation controller

- Access the `NavController` object
  - Activity
    - `findNavController(R.id.fragment_base)`
  - Fragment
    - `findNavController()`

- Control the `NavController`
  - Change the fragment
    - `navCtrl.navigate(resID)`
      - `resID` can be an action or a target (fragment)
  - Control the *back stack*
    - `navCtrl.popBackStack( )`
    - `navCtrl.clearBackStack(...)`
  - Monitor
    - `navCtrl.addOnDestinationChangedListener { }`
  - …

Dialogs

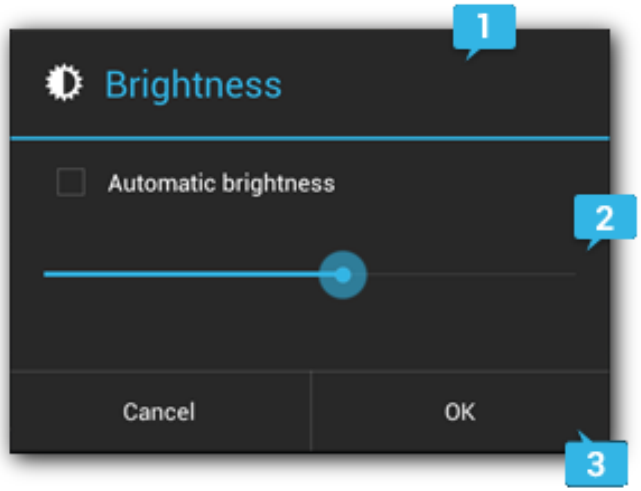
- A *dialog* is a window that allows interaction with the user, overlapping the screen and not occupying it in its entirety
  - Appears “on top” of the rest

- Dialogs are implementations of the `Dialog` class
  - There are defined types of *dialog* that facilitate their use, for example:
    - `AlertDialog`
    - ~~`ProgressDialog`~~
    - `DatePickerDialog`
    - `TimePickerDialog`

AlertDialog

- The creation of instances of the `AlertDialog` class must be done with the help of its nested class `Builder`

- Allows the creation of a dialog window consisting of 3 parts:
  - Title and icon
    - `setTitle, setIcon, setCustomTitle`
  - Body
    - message
      - `setMessage`
    - list of elements
      - `setItems, setSingleChoiceItems, …`
    - view/inflate or custom view
      - `setView`
  - Action buttons
    - action/confirmation buttons
      - positive button (ok, yes, …)
        - `setPositiveButton`
      - negative button (no, cancel, …)
        - `setNegativeButton`
      - neutral button (remind me later, …)
        - `setNeutralButton`



- Once configured…
  - must be created using the `Builder`'s `create()` method
  - displayed with the `show()` method

AlertDialog

```
val dlg = AlertDialog.Builder(this)
    .setTitle("Title")
    .setIcon(android.R.drawable.ic_dialog_alert)
    .setMessage("Message")
    .setPositiveButton("Yes",
      DialogInterface.OnClickListener { dialog, which -> . . . })
    .setNegativeButton("No",
      DialogInterface.OnClickListener { dialog, which -> . . . })
    .setCancelable(false)
    .create()

dlg.show()
```

DialogFragment

- For better internal management and adaptation to the lifecycle of activity components, dialog windows should be created in the context of a `DialogFragment` object and managed as fragments
  - Create an object derived from `DialogFragment` and return an `AlertDialog` in the `onCreateDialog` method

```
class MyDialog : DialogFragment() {
    override fun onCreateDialog(
                          saveInstState: Bundle?): Dialog {
        return AlertDialog.Builder(context!!)
            .setTitle("Titulo")
            ...
            .create()
    }
}
```

  - To show a dialog window created as fragment, do…

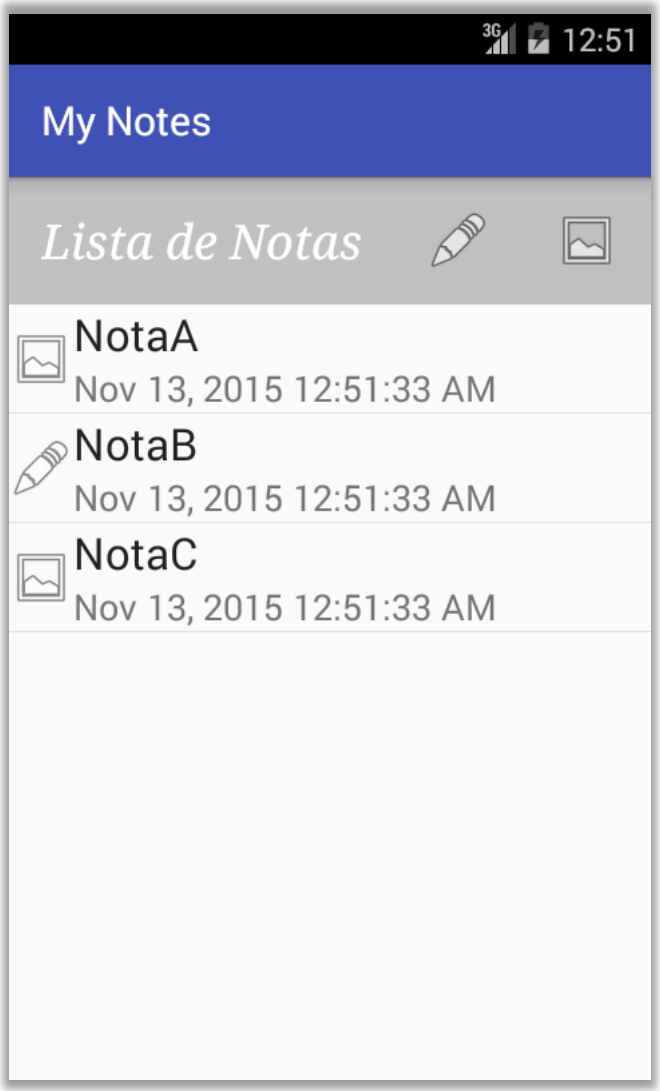
```
val dlg = MyDialog()
dlg.show(supportFragmentManager, "my_dialog")
```

Activities like Dialogs

- Another option for creating dialog windows is using activities
- For activities to behave like dialog windows, it is necessary to associate them with an appropriate theme, for example:
  - `@android:style/~~Theme.Holo.Dialog~~`
  - `@style/Theme.AppCompat.Light.Dialog`
- The activities will be presented in the same way as a regular activity, using an `Intent` object and calling the `startActivity` method

ListView and ListAdapter

- The ListView allows to display and select from lists of elements
- It is necessary to define a ListAdapter from which the data to be inserted into the ListView is obtained
  - ArrayAdapter<T>
    - The data is obtained from an Array to fill a single TextView, corresponding to each item in the list
  - SimpleAdapter
    - The data corresponding to each line of a list will be obtained from an ArrayList of Map objects, organized and displayed with the help of a *layout* XML file
  - CursorAdapter
  - SimpleCursorAdapter
  - Custom adapter
    - Class derived from BaseAdapter
    - Define the methods: getCount, getItem, getItemId, getView



RecyclerView

- Allows the implementation of lists similar to `ListView`, but with improved resource management
- To implement it, the following objects must be used:
  - `LayoutManager`
    - `LinearLayoutManager`
    - `GridLayoutManager`
    - `StaggeredGridLayoutManager`
  - `RecyclerView.Adapter`
    - Extend this class
    - It allows to configure the *recycler view* indicating: the number of items, a method to create an item and another method to update an item's data
  - `RecyclerView.ViewHolder`
    - Extend this class
    - It will represent the hierarchy of views corresponding to an item

Data storage

- As with other systems, there are several ways to store data and manage data persistence

- On Android systems :
  - Internal Storage
  - External Storage
  - SQLite Databases
  - Shared Preferences
  - Network Connection
    - `java.net.*`
    - `android.net.*`
  - Content Providers



Internal Storage

- Private application space to save files
  - When the application is uninstalled the files are deleted
  - `/data/data/<package_app>/*`
- Functions…
  - `openFileOutput(<name>,<mode>)`
    - returns a `FileOutputStream`
    - Mode: `MODE_PRIVATE`, `MODE_APPEND`, ~~`MODE_WORLD_READABLE`~~, ~~`MODE_WORLD_WRITEABLE`~~
  - `openFileInput (<name>)`
    - returns a `FileInputStream`
  - Auxiliary functions: `read, write, close, getDir, getFilesDir, getCacheDir, deleteFile, fileList, …`

External Storage

- Existing space on the device available for use by any application (e. g., *sd card*)
  - `/sdcard/Android/data/<package_app>/*`
- Files saved on *External Storage* of the application are removed when the application is uninstalled
- Before starting to use the applications, the state/existence must be checked using the function:

  `Environment.getExternalStorageState()`
  - This function allows to check whether the card has been *unmounted* or whether it is protected against writing operations

External Storage

- Functions available to obtain directories to store information:
  - On the application space
    - `getExternalFilesDir`
    - `getExternalCacheDir`
    - …
  - On shared space
    - `Environment.getExternalStorageDirectory`
    - `Environment.getExternalStoragePublicDirectory`
    - `Environment.getRootDir`
    - …

- The common Java classes available in the package `java.io.*` could be used to open and read/write files

External Storage

- Permissions needed in the manifest file

```
<uses-permission android:name="WRITE_EXTERNAL_STORAGE">
<uses-permission android:name="READ_EXTERNAL_STORAGE">
```

  - For API >= 23, permissions must also be requested at *runtime*
  - Write permission is not necessary if the application only needs to write in its own directories (obtained through the functions mentioned before)

- With API 30, a new permission was added to be able to use space managed by other applications: `MANAGE_EXTERNAL_STORAGE`
  - It should only be used as a last resort. Its use is subject to special verification when distributing the application through the *Google Playstore*

- With API 33, new permissions were added to be able to access audio, images and videos:
  - `READ_MEDIA_AUDIO`
  - `READ_MEDIA_IMAGES`
  - `READ_MEDIA_VIDEO`

SQLite Databases

- Database engine perfectly adapted to the *Android* system
- Offers the usual features in any SQL database system
- Objects to use:
  - `SQLiteOpenHelper`
    - By deriving a new class from this class, one of these objects can be used to create (overriding `onCreate`), open or upgrade the database defined
  - `SQLiteDatabase`
    - Allows normal operations: *query*, *insert*, *update*, transactions, …
  - `SQLiteQueryBuilder`
    - Advanced *queries* (e.g., support for table joining)
  - `Cursor`
    - The result of any *query* is represented through a cursor, which allows access to all the resulted data

SQLiteOpenHelper

```
const val DATABASE_NAME = "mydb.db"
const val DATABASE_VERSION = 1

class MyDBHelper(context : Context) :
        SQLiteOpenHelper(context, DATABASE_NAME, null, DATABASE_VERSION) {
    val createCmd = "CREATE TABLE people ( id NUMBER, name TEXT, age NUMBER );"

    override fun onCreate(db: SQLiteDatabase?) {
        db?.execSQL(createCmd)
    }

    override fun onUpgrade(db: SQLiteDatabase?, oldVersion: Int, newVersion: Int) { }
}
```

```
val dbh = MyDBHelper(context!!)
val db = dbh.writableDatabase

val values = ContentValues()
values.put("id", 1234)
values.put("name", "Zeferino")
values.put("age", 73)

db.insert("people", null, values)
```

```
val dbHelper = MyDBHelper(context!!)
val db = dbHelper.readableDatabase

val cursor: Cursor = db.query("people",
             null, null, null, null, null, null)
cursor.moveToFirst()
while (!cursor.isAfterLast) {
  val id = cursor.getInt(0)
  val name = cursor.getString(1)
  val age = cursor.getInt(2)
  cursor.moveToNext()
}
cursor.close()
```

Room

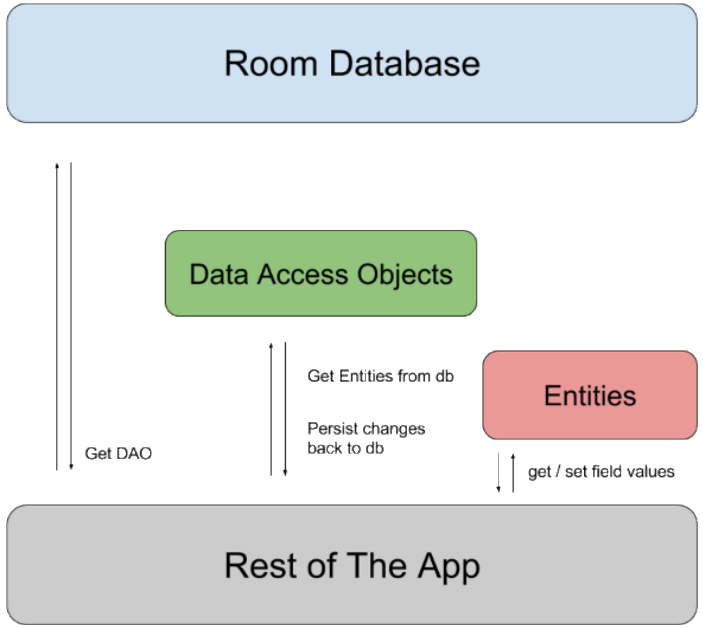
- Room is a *framework* introduced on *Android* to facilitate the use of SQLite

```
plugin{
    ...
    id("kotlin-kapt")
}

dependencies {
    ...
    implementation("androidx.room:room-runtime:2.6.1")
    annotationProcessor("androidx.room:room-compiler:2.6.1")
    kapt("androidx.room:room-compiler:2.6.1")
}
```

Room

- Room is based on 3 types of elements:
  - *Database*
    - Object that enables database management and access to information
  - *Entity*
    - Defines each database entity/table
  - *DAO (Data Access Objects)*
    - Database access methods



Example Room

```
@Database(entities = [Person::class], version = 1)
abstract class MyRoomDB : RoomDatabase() {
    abstract fun personDao() : PersonDao
}
```

```
@Entity(tableName = "person")
data class Person (
    @PrimaryKey(autoGenerate = true) val id : Int = 0,
    @ColumnInfo(name = "full_name") var name : String,
    var age : Int
)
```

Example Room

```
@Dao
interface PersonDao {
    @Query("SELECT * FROM person")
    fun getAll(): List<Person>

    @Query("SELECT * FROM person WHERE id IN (:personIds)")
    fun loadAllByIds(personIds: IntArray): List<Person>

    @Query("SELECT * FROM person WHERE age >= :minAge AND age<= :maxAge LIMIT 1")
    fun findOneByAge(minAge: Int, maxAge: Int): Person?

    @Insert
    fun insertAll(vararg people: Person)

    @Delete
    fun delete(person: Person)
}
```

Example Room

```
val db = Room.databaseBuilder(
                        context!!,
                        MyRoomDB::class.java,
                        "my_room_db.db"
        ).build()

val person = Person(
    //id = 1234,
    name = "Zeferino",
    age = 73
)

db.personDao().insertAll(person)
```

Example Room

```
val db = Room.databaseBuilder(
                        context!!,
                        MyRoomDB::class.java,
                        "my_room_db.db"
        ).build()

println(">>> all\n")
for (p in db.personDao().getAll())
    println(" - [${p.id}] ${p.name} {age = ${p.age}\n")

println("\n>>> One with 30..40 years old\n")
db.personDao().findOneByAge(30, 40)?.apply {
    println(" - [$id] $name {age = $age\n")
}
```

Room – Database Views

- *Database Views* can be specified to obtain data from relational databases

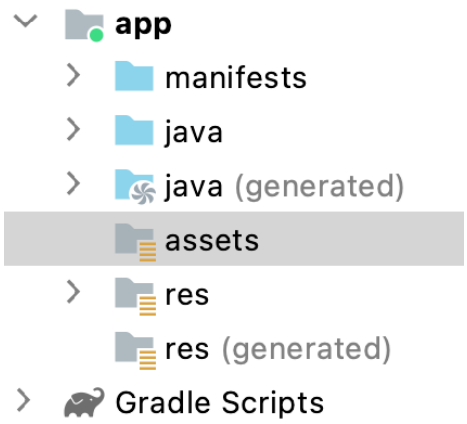
```
@DatabaseView("SELECT user.id, user.name, user.departmentId," +
        "department.name AS departmentName FROM user " +
        "INNER JOIN department ON user.departmentId = department.id")
data class UserDetail(
    val id: Long,
    val name: String?,
    val departmentId: Long,
    val departmentName: String?
)

@Database(entities = [User::class], views =[UserDetail::class],
        version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun userDao(): UserDao
}
```

Assets

- *Assets* correspond to auxiliary files that are included in the application package for distribution
- They must be stored in an appropriate directory hierarchy

- *Assets* base folder:
  - configured at module level
  - name: `assets`



- The Context of an application or activity provides the `assets` property that allows to obtain the `AssetManager` reference, from which the files can be accessed

Files and images in Assets

- Given the file name...

```
fun getFileFromAsset(assetManager: AssetManager, strName: String): InputStream? {
    var istr: InputStream? = null
    try {
        istr = assetManager.open(strName)
    } catch (e: IOException) {
        e.printStackTrace()
    }
    return istr
}
```

- Example to read an image and assign it to an `ImageView`

```
fun setImgFromAsset(mImageView: ImageView, strName: String) {
    val assetManager = mImageView.context.assets
    try {
        val istr = assetManager.open(strName)
        val d = Drawable.createFromStream(istr, null)
        mImageView.setImageDrawable(d)
    } catch (e: IOException) {
        e.printStackTrace()
    }
}
```

SharedPreferences

- The `SharedPreferences` objects allow to save settings and other data between application runs

- The data corresponds to attribute/value pairs (similar to *Bundle* , *Intent* , *Map* , ...) that are stored through an XML file in the *Internal Storage*
  - Since the `SharedPreferences` entity corresponds to an interface, there may be other implementations that, for example, do not use XML files

SharedPreferences

- Object creation is carried out with the help of the following methods
  - `getSharedPreferences(name : String, mode : Int)`
  - `getPreferences(mode : Int)`
    - Similar to the previous one but using the name of the activity as the name
  - `PreferenceManager.getDefaultSharedPreferences(c : Context)`
    - To obtain an object with the data managed by `PreferenceScreen` screens

SharedPreferences

- Access to data is carried out using *get* methods, similar to bundles, intents, etc.
  - These methods allow to indicate a default value if there is no value for a given attribute
- To edit the attribute/value pairs of a `SharedPreferences` object it is necessary to use a `SharedPreferences.Editor` object
  - Created with the method: *`SharedPreferences`*`.edit()`
  - Provides the essential methods for managing data (*put* methods)
  - After making the changes, it is necessary to call the `commit()` or `apply()` methods to confirm the changes, which respectively work in a synchronous and asynchronous form
  - Other methods: `clear()`, `remove(`***attribute***`)`

PreferenceScreen

- The management of an application's settings, saved through `SharedPreferences`, can be carried out with the help of an Android subsystem that facilitates the provision of standard screens
- The definition of settings screens are made through `PreferenceScreen` resources
  - These resources replace the usual *layouts* of an activity or fragment
  - Are presented in the context of a `PreferenceFragment`
  - Allow to include: `CheckBoxPreference`, `EditTextPreference`, `ListPreference`, `MultiSelectListPreference`, `PreferenceCategory`, `Preference`, `RingtonePreference`, `SwitchPreference` or other `PreferenceScreen`
    - Each of these elements allow the configuration of a '`key`' attribute that will receive the value configured within the elements

PreferenceScreen (API29+)

- The aforementioned preference model is *deprecated* in Android 10 and up (API Level 29+)
  - In new implementations *AndroidX Preference Library* must be used
    - https://developer.android.com/guide/topics/ui/settings
  - Operation is maintained through fragments, but within the scope of the *Support Library*, the fragment should be derived from the `PreferenceFragmentCompat` class
  - Inserting a *Settings* activity through the *Android Studio activities gallery* allows the testing of this new way of implementation
  - There are additional specific elements that can be added to these screens, that are implemented in the *Support Library*

Content Providers

- The Content Provider components allow:
  - to manage information to be used by one or more applications
  - to access and update information
  - to share information between applications

- The system provides several Content Providers for the most common data
  - Audio, Video, Images, Contacts, … (see *android.provider*)

Content Providers

- All Content Providers provide a common interface
- New Content Providers can be defined
  - by deriving a new class from `ContentProvider`
  - by defining the abstract members: `query`, `insert`, `update`, `delete`, `getType`, `onCreate`
- Data is stored according to the programmer's preferences

ContentProvider

- Class derived from the `ContentProvider`

```
class MyContentProvider : ContentProvider() {
    override fun delete(uri: Uri, selection: String?, selectionArgs: Array<String>?): Int {
        TODO("Implement this to handle requests to delete one or more rows")
    }
    override fun getType(uri: Uri): String? {
        TODO("Implement this to handle requests for the MIME type of the data at the given URI")
    }
    override fun insert(uri: Uri, values: ContentValues?): Uri? {
        TODO("Implement this to handle requests to insert a new row.")
    }
    override fun onCreate(): Boolean {
        TODO("Implement this to initialize your content provider on startup.")
    }
    override fun query(uri: Uri, projection: Array<String>?, selection: String?,
                       selectionArgs: Array<String>?, sortOrder: String?): Cursor? {
        TODO("Implement this to handle query requests from clients.")
    }
    override fun update(uri: Uri, values: ContentValues?, selection: String?,
                        selectionArgs: Array<String>?): Int {
        TODO("Implement this to handle requests to update one or more rows.")
    }
}
```

Content Providers

- The *Content Provider* must be registered in the manifest file
  - Define the "`authorities`" attribute with the *Content Provider identification* (using a URI)

```
<provider
        android:name=".MyContentProvider"
        android:authorities="pt.isec.ans.amovcp.provider"
        android:enabled="true"
        android:exported="true" />
```

  - A `multiprocess` attribute can be added to allow the *Content Provider* to be executed within the scope of client processes

  - It is also possible to associate permissions to restrict its use

Content Providers

- The access to a *Content Provider* is carried out with the help of a *Content Resolver* object
  - The `Context` objects provide a `contentResolver` property that allows to trigger different operations on Content Providers

    `contentResolver.query(...)`

- From API 30 onwards, it is necessary to include in the manifest file the *packages* and/or *providers* that are intended to be accessed within the scope of the application

```
<queries>
   <provider android:authorities="pt.isec.ans.amovcp.provider"/>
</queries>
```

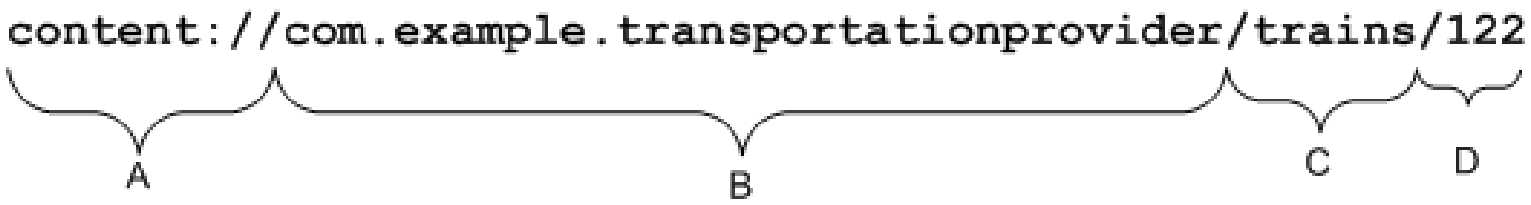
Content Providers

- Data is available as simple tables
  - A line corresponds to a record having several columns corresponding to the various data fields
  - Each record must have an `_id` field that identifies it and allows data relationships with other tables

Content Providers – URI

- One *Content Provider* provides a unique URI (*Uniform Resource Identifier*) that allows its identification

- Format:



  - A – Mandatory prefix
  - B – *Content Provider* identification
  - C – Type/Table of data to be accessed (optional)
  - D – Identification of the record to be accessed (optional)

Content Providers – query

- To preform a *query* to the *Content Provider* the following information must be provided:
  - URI (mandatory)
  - Name of data fields (optional)
  - Data type of the fields in question (optional)

  - If the objective is to consult just one record, it will be necessary to have its `_id` (if the Content Provider allows this type of query)

Content Providers – query

- Method used to perform a query
  - `ContentResolver.query(…)`
  - Support Library `CursorLoader`

  - The method to perform the *query* has the following parameters
    - Uri (e. g., `"content://contacts" ; "content ://…/45"`)
    - `Array<String>` with the desired field names or `null` to get all fields
    - Filter (*String*) corresponding to the SQL `WHERE` clause or `null` not to impose *restrictions*
    - `Array<String>` with the arguments for the previous filter (corresponding to the '?' placed in the filter *string)*
    - Ordering (*String*) corresponding to the SQL `ORDER BY` clause or `null` not to order

Content Providers – query

```
// Sets the columns to retrieve for the user profile
projection = arrayOf(
        ContactsContract.Profile._ID,
        ContactsContract.Profile.DISPLAY_NAME_PRIMARY,
        ContactsContract.Profile.LOOKUP_KEY,
        ContactsContract.Profile.PHOTO_THUMBNAIL_URI
)

// Retrieves the profile from the Contacts Provider
profileCursor = contentResolver.query(
        ContactsContract.Profile.CONTENT_URI,
        projection,
        null,
        null,
        null
)
```

Content Providers – other operations

- Add records (example)

```
val uriInsert = Uri.parse(CONTENTPROVIDER_URI)

val contentValues = ContentValues().apply {
    put("type",1)
    put("code",6543)
    put("name","Antanhol")
}

val uriresult = contentResolver.insert(uriInsert,
                                       contentValues)
```

Background tasks

- All 4 component types (*activity*, *content provider*, *service* and *broadcast receiver*) of an *Android* application execute in the context of the same thread – the *Main Thread*

- Any task that takes some time to perform can affect the expected behavior of the user interface

- The longest tasks must be executed in the context of background threads

Threads

(This topic is taught using examples explained in class)

- Some topics covered
  - *Thread* creation
    - Class `Thread`
    - Interface `Runnable`
  - Executing interface-related tasks on appropriate threads
    - ~~`Handler`~~
    - `View.post`
    - `runOnUIThread`
  - The `AsyncTask` class
  - `Executor` and `ThreadPool`

coroutines

- The coroutines concept was introduced in *kotlin* to offer functionalities similar to *threads* but with a more efficient management, including :
  - Management similar to `ThreadPool`
  - Possibility of suspending threads
    - It allows the use of a suspended thread for the execution of a new task
    - When the suspended task/thread resumes, its execution may occur in a *thread* different from the previous one

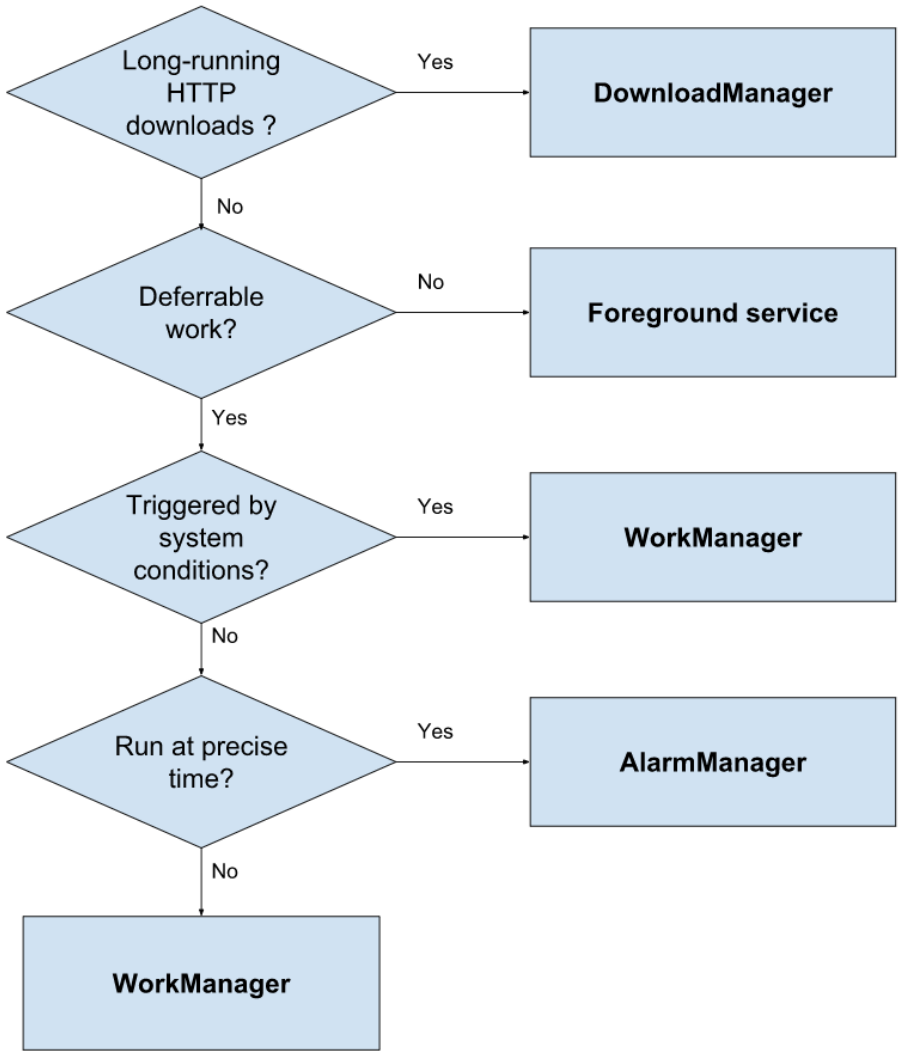
- Include in `build.gradle`

```
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.9.0")
implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.9.0")
```

- ***See example shown in the classes***

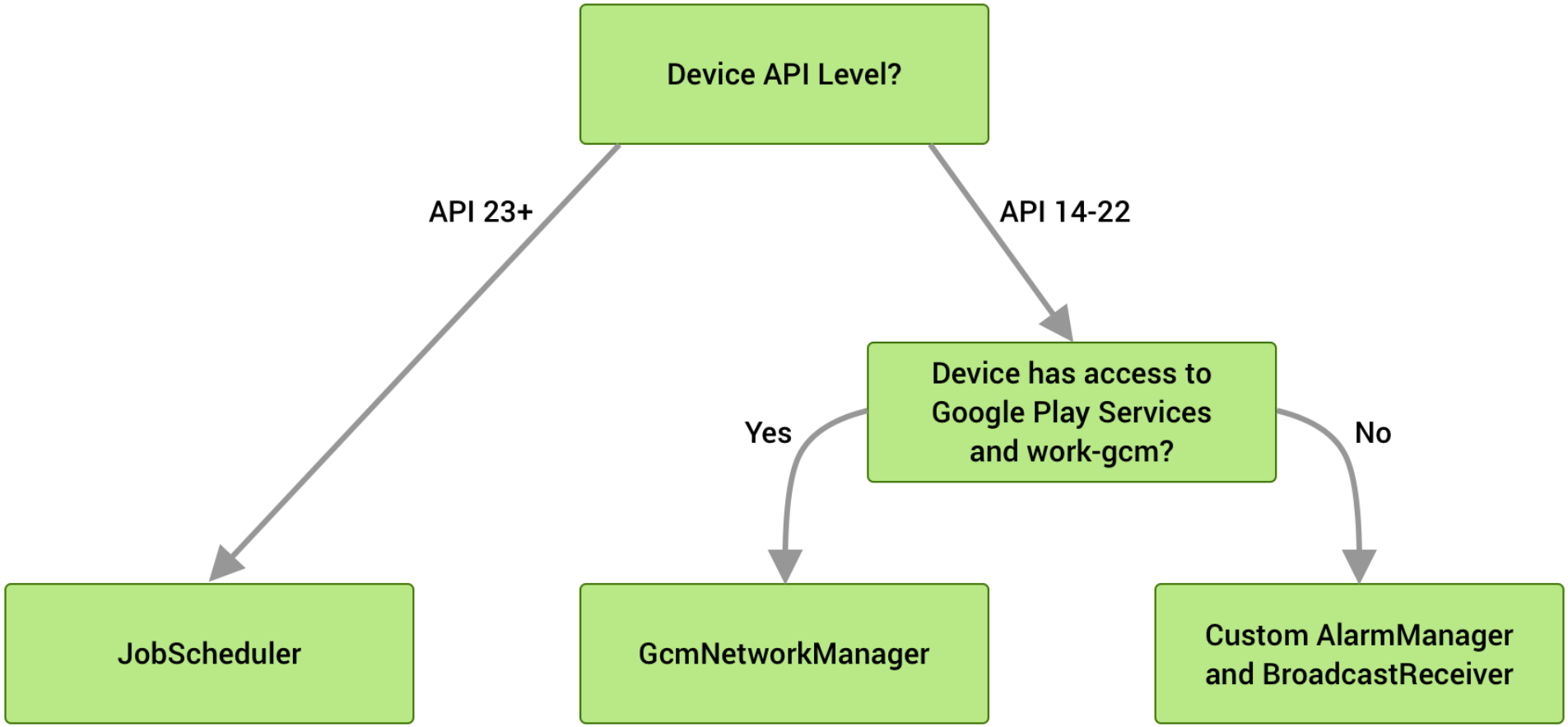
Background tasks

- In addition to traditional *threads* and *coroutines* (*kotlin*), other solutions can be chosen, which directly or indirectly use *threads*
  - `DownloadManager`
    - Download files in the background
  - `AlarmManager`
    - Run a specific task at a configured time
  - `WorkManager` (*Android Jetpack*)
    - Object that allows the execution of tasks as soon as the system has the capacity to do so
  - *Foreground service*
    - Service used to perform operations that the system should not interrupt
    - It must have a “non-disposable” notification associated with it



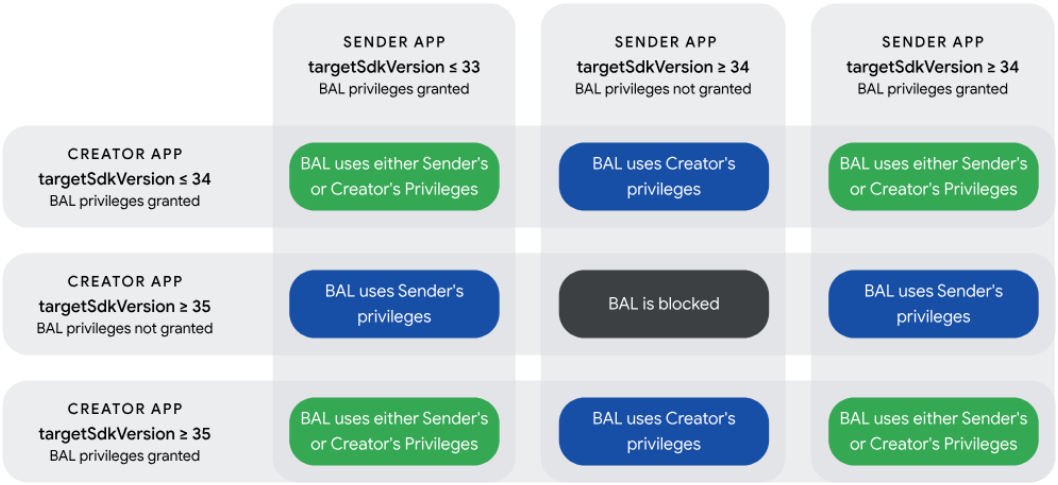
WorkManager

- `implementation("androidx.work:work-runtime-ktx:2.10.0")`



Background task restrictions

- Some restrictions on background tasks have been introduced in the latest versions
  - Since Android 10 (API 29), a background task cannot directly open an activity
  - An alternative is to generate a notification or run a foreground service to indirectly open the activity.
  - *Apps running on Android 10 or higher can start activities when one or more of the following conditions are met:*
    - *The app has a visible window, such as an activity in the foreground*
    - *The app has an activity in the back stack of the foreground task*
    - *The app has an activity in the back stack of an existing task on the Recents screen*
    - *The app has an activity that started very recently*
  - *Decision flow for Background Activity Launches (BAL):*



JobService and JobScheduler

- It implements a service that will only be activated if certain conditions are met (e. g., the device is charging, the battery is not too low, it is not connected to *roaming*, ...)

- Create class that derives from the `JobService` class
  - Register in the manifest file as if it were a normal service
  - Registration must include `BIND_JOB_SERVICE` permission

- The service is started as follows
  - Create a `ComponentName` object to indicate the service
  - Create a `JobInfo` object to describe conditions
  - Schedule startup in the context of a `JobScheduler` object using the `schedule` method
    - A `JobScheduler` object can be obtained with the method: `getSystemService`

JobService

AndroidManifest.xml

```
<service android:name=".MyJobService”
                        android:permission="android.permission.BIND_JOB_SERVICE" />
```

MyJobService.java

```
class MyJobService : JobService() {
    override fun onStartJob(params: JobParameters?): Boolean {
        // TODO: launch thread to execute intended code
        return true
    }

    override fun onStopJob(params: JobParameters?): Boolean {
        return true
    }
}
```

JobScheduler

```
val jobScheduler = getSystemService(JOB_SCHEDULER_SERVICE)
                                                    as JobScheduler

val cn = ComponentName(this, MyJobService::class.java)

val jobInfo = JobInfo.Builder(1234, cn)
    .setRequiresCharging(true)
    //.setPeriodic(3600000)
    //.setPersisted(true) // restart after reboot
                        //needs permission: RECEIVE_BOOT_COMPLETED
    .build()

jobScheduler.schedule(jobInfo)
```

Services

- A service allows the execution of procedures in the background

- Implemented through instances of `Service` objects
  - To define a new service, define a new subclass of `Service` class
    - Must be registered in the manifest file in a `service` structure

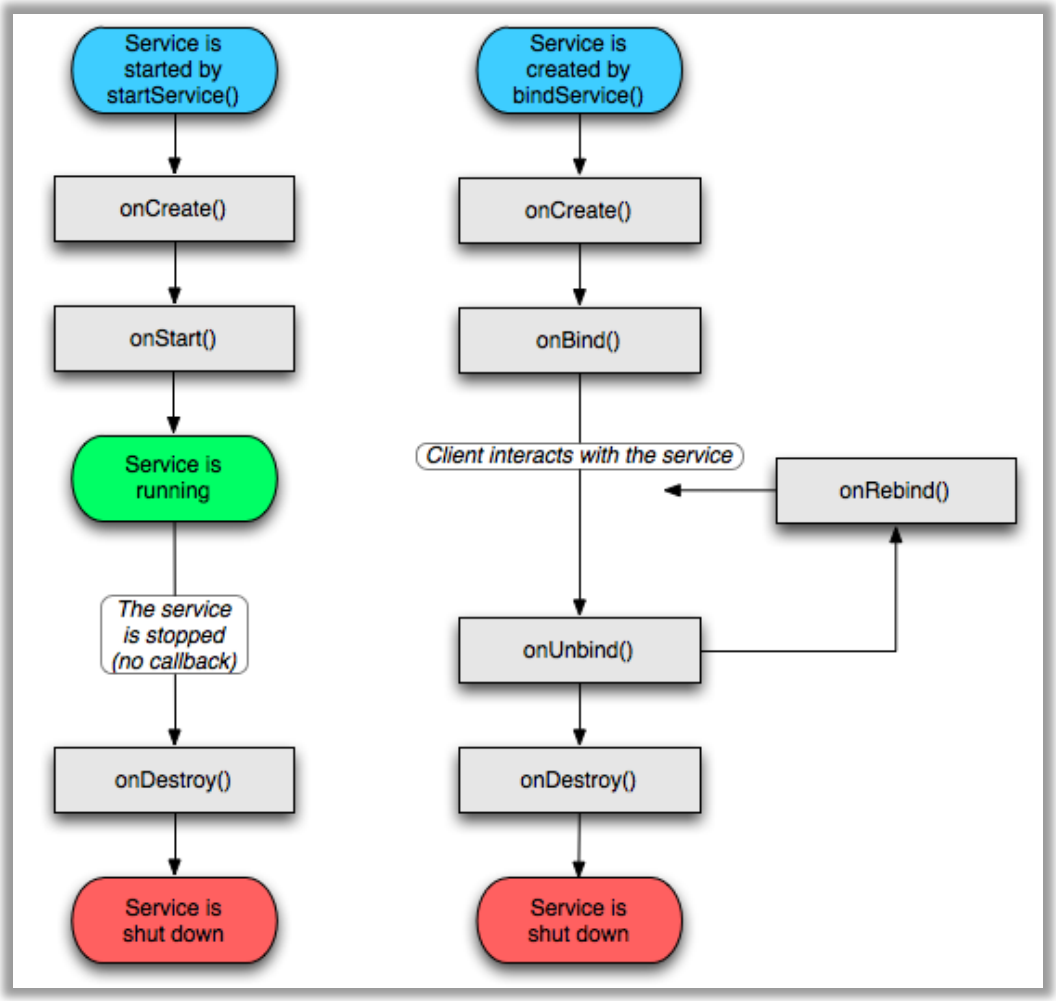
```
<service android:name=".MyService" />
```

  - A service is executed on the *main thread*

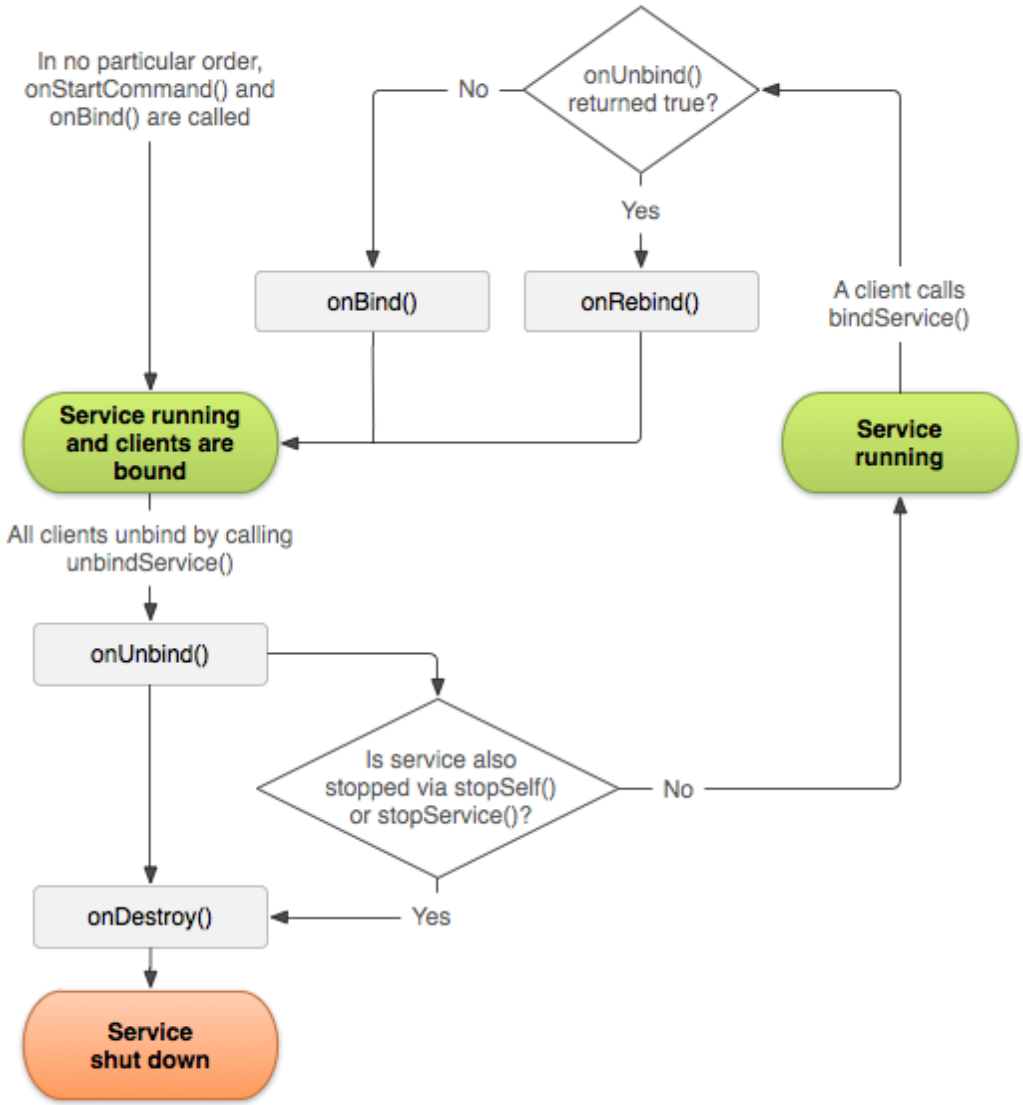
Services

- Lifecycle of a service
  - When started with `startService` the following methods are executed:
    - `onCreate`, `onStartCommand`, ~~`onStart`~~, `onDestroy`
  - When used in *bind* operations
    - A `Binder` class must be defined
      - Even if the service is not used in bind operations, a `Binder` class must be defined (eventually, without any functionality)
      - Creating an instance of the created `Binder` class, returning the reference of this instance in the `onBind` method
        - This object is received in the context of a `ServiceConnection` object that is associated with a service during a bind operation (calling the `bindService` methods)
        - The interaction with the service will be done using the methods and variables provided in the `Binder` class
    - If the service hasn't already been started it will be started
      - Depending on the flags of bind operation
    - The methods `onBind`, `onUnbind` and `onRebind` will be used to manage and signaling the bind operation phases

Lifecycle (Service)



Lifecycle (Service)



Foreground Service

- Service that has a notification associated with it
  - The notification is intended to make it clear that the user is aware of its execution

- For this kind of service to be used
  - ask permission `FOREGROUND_SERVICE`
  - start the foreground service in the same way as a regular service
  - call `startForeground` function in the first 5 seconds of execution, in which a `Notification` object must be passed/generated

- This type of service is the solution to the restrictions introduced in new Android versions regarding continued execution of services when the application goes into the background

Foreground Service – version differences

- Since API 31, it is no longer possible to launch *foreground services* when the application is in the *background*
  - Some situations are exempt from this restriction. See the official documentation:
    - https://developer.android.com/guide/components/foreground-services#background-start-restriction-exemptions
- From API 29 onwards, a service that uses location must declare this in the manifest file

```
<service ... android:foregroundServiceType="location" />
```

  - Ensure that the app request permission to use the location in the background
- From API 30 onwards, the use of a microphone and camera in a foreground service must also be explicitly expressed

```
<service ... android:foregroundServiceType="location|camera|microphone" />
```

- From API 34 onwards:
  - Setting the `foregroundServiceType` is mandatory for all foreground services.
  - The specific permission associated with the `foregroundServiceType` must be requested, e.g., `android.permission.FOREGROUND_SERVICE_LOCATION`

IntentService

- The `IntentService` is a subclass of the `Service` class
- To create a service of this type, a new class is derived from the `IntentService`
  - Create a constructor that calls the base class constructor, passing a *tag* that identifies the service
  - Redefine the `onHandleIntent` method
- Facilitates the processing of successive orders
  - Automatic queue management
    - Services are performed one at a time, in order of request
  - Execution is done in a *thread* created for this purpose
    - Does not run in the *main thread*
- This service is started in the same way as a regular service
- *Deprecated* as of API 30

IntentService

AndroidManifest.xml

```
<service android:name =".MyIntentService" />
```

MyIntentService.kt

```
class MyIntentService : IntentService("MyIntentService")
{

    override fun onHandleIntent(intent: Intent?) {
        //Code to execute
    }

}
```

JobIntentService

- The functioning of `JobIntentService` is similar to `IntentService` (which is now *deprecated*), but with more efficient management by the system
  - It presents a behavior similar to the *sticky* mode of services, and can be automatically restarted by the system if the process is terminated

- `JobIntentService` is also now *deprecated*
  - It is suggested to use `WorkManager`

JobIntentService

AndroidManifest.xml

```
<service android:name=".MyJobIntentService"
 android:permission="android.permission.BIND_JOB_SERVICE"/>
```

MyJobIntentService.kt

```
class MyJobIntentService : JobIntentService() {

    override fun onHandleWork(intent: Intent) {
        //Code to execute
    }
}
```

Broadcast Receivers

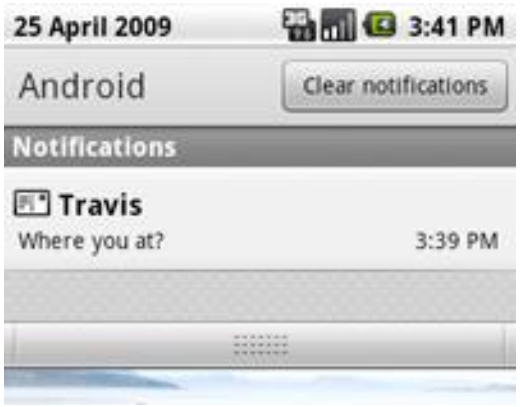
- The *Broadcast Receivers* (BR) allow an application to react to certain events that occur in the system, for example:
  - Receiving an SMS
  - Low battery level
  - Boot sequence has been completed
  - ...

- BRs can also be used for...
  - Status signaling, and/or...
  - Sending information between applications

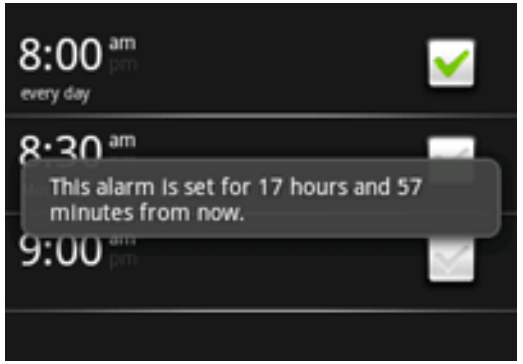
Broadcast Receivers

- BRs do not have a direct graphical interface
  - They can generate results for the user...
    - Using Android resources and subsystems

Status bar and notifications



Toast Notifications

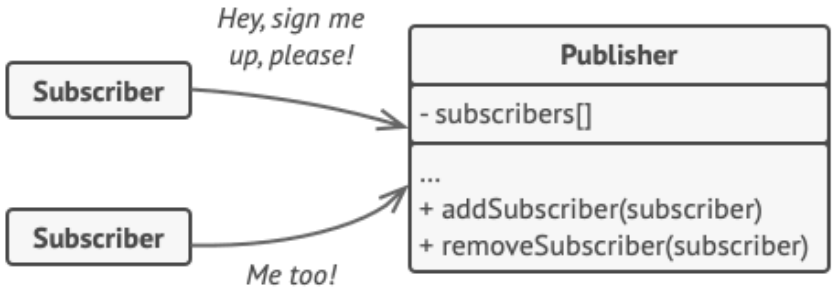


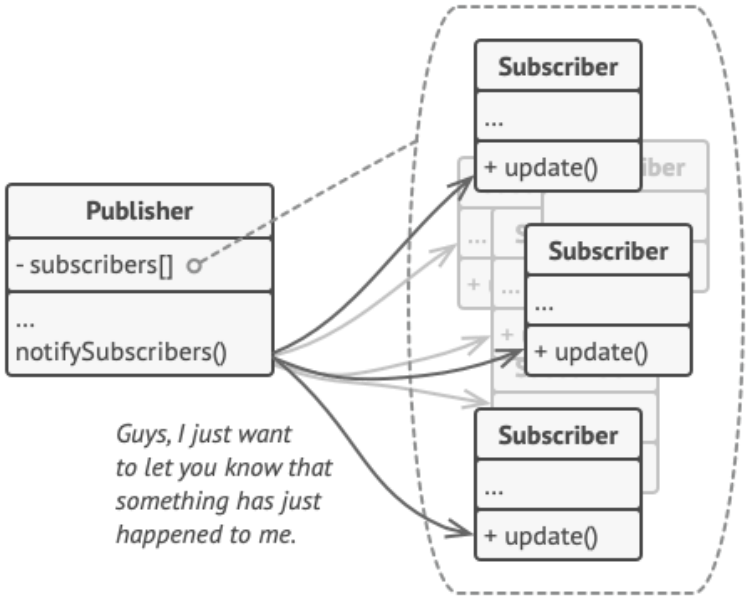
- ~~Launching other components (e. g., activities)~~
  - ~~This option should be avoided~~ (prohibited for API 29+)

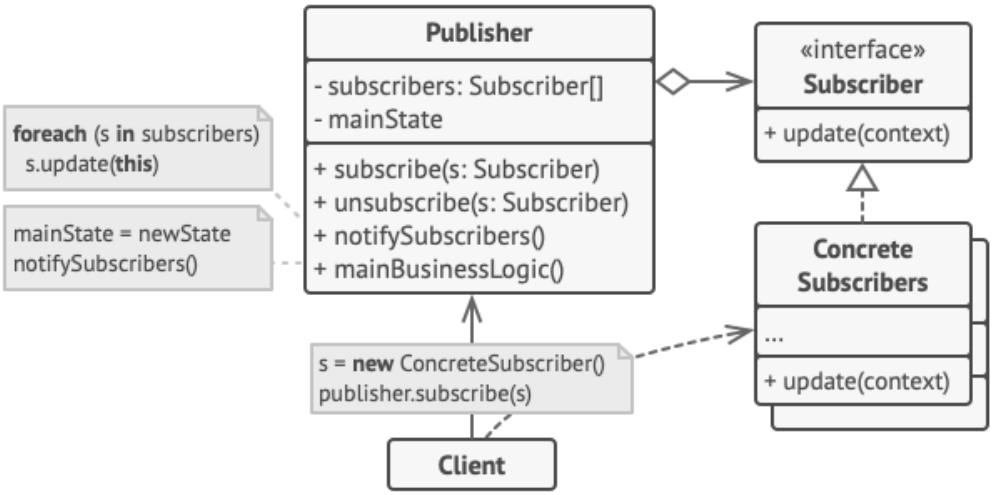
https://developer.android.com/develop/background-work/background-tasks/broadcasts
https://developer.android.com/guide/components/activities/background-starts

Observer

- BR consists of an implementation of the Publish-Subscribe software design pattern



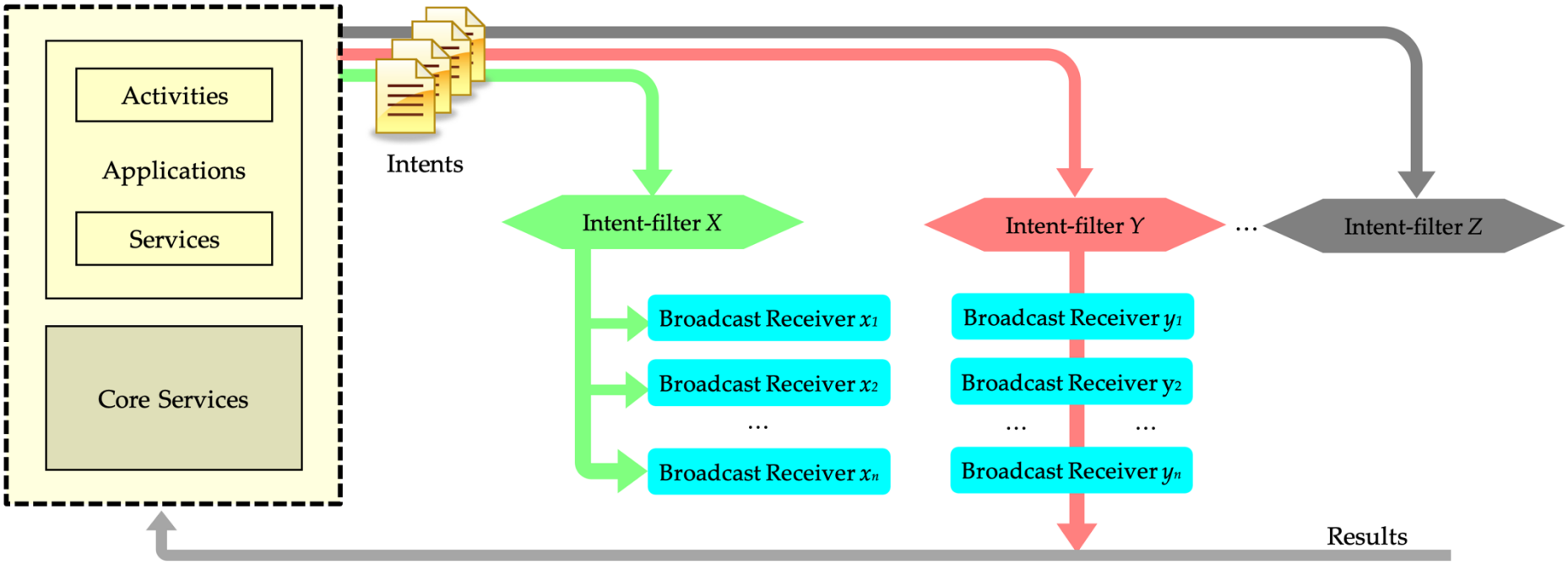




https://refactoring.guru/design-patterns/observer

General operation

- When a certain event occurs, a message is generated (represented by an *Intent*) which is sent to all BRs who have registered their interest in the event in question



Implementation

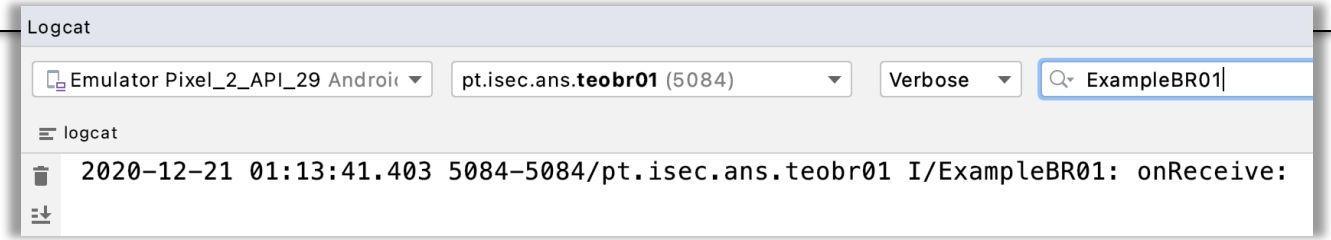
- Implementing a BR involves two steps
  - Definition
    - Definition of the code that will be executed when a specific broadcast occurs
  - Registration
    - Expression of interest in receiving the broadcast in question
    - Association of this interest with the defined code

Definition of a Broadcast Receiver

- To define a BR, a new class must be derived from the `BroadcastReceiver` class
  - This new class must implement the abstract method

```
override fun onReceive(context:Context, intent:Intent)
```

```
class ExampleBR01 : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        Log.i("ExampleBR01", "onReceive: ")
    }
}
```



Broadcast Receiver Registration

- Registering a BR can be done in two ways:
  - *Manifest-declared receivers* (statically)
    - Registration is carried out through the manifest file
    - From API 26, …
      - the number of *Broadcasts* that allow this registration has been reduced

        https://developer.android.com/develop/background-work/background-tasks/broadcasts/broadcast-exceptions
      - to receive statically registered *intents*, an application needs to have an activity and be executed at least once
  - *Context-registered receivers* (dynamically)
    - Registration is done at *runtime*

Manifest-declared receivers

- Static registration is performed by embedding a `<receiver ... />` structure into the `<application ... />` structure in the `AndroidManifest.xml` file

```
<receiver android:enabled=["true" | "false"]
          android:exported=["true" | "false"]
          android:icon="drawable resource"
          android:label="string resource"
          android:name="string"
          android:permission="string"
          android:process="string" >
    . . .

</receiver>
```

Manifest-declared receivers

- In the context of the `<receiver ... />` structure, the following must be declared:
  - `BroadcastReceiver` class – The class that defines the BR
  - Export status – Indicates whether the receiver is exported or not
    - Mandatory for API 33+
      - Use the `android:exported` attribute to explicitly define whether the receiver can receive broadcasts from outside the app
  - One or more Intent filters – Represent the events for which the BR expresses interest in being notified

Manifest-declared receivers

- The enumeration of the events in which the BR is interested in is carried out through `<intent-filter ... />` structures
  - These structures can list one or more actions that are intended to be processed

```
<action android:name="<action>" />
```

Example manifest file

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="pt.isec.ans.teobr01">

    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED" />

    <application
        android:allowBackup="true" android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name" android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true" android:theme="@style/Theme.TeoBR01">

        <receiver android:name=".ExampleBR01" android:enabled="true" android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED" />
            </intent-filter>
        </receiver>

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```



Context-registered receivers

- An application can register a BR at *runtime*
  - This is called a "Context-registered receiver"
- It can be done through the `registerReceiver` method

```
fun registerReceiver(receiver: BroadcastReceiver,
                     filter: IntentFilter )

fun registerReceiver(receiver: BroadcastReceiver,
                     filter: IntentFilter,
                     broadcastPermission: String,
                     scheduler: Handler )
```

- From API 33 onwards, it is mandatory to use the `registerReceiver` method with an extra flag parameter to specify whether the BR is exported (`RECEIVER_EXPORTED`) or not (`RECEIVER_NOT_EXPORTED`)

Context-registered receivers

- In this type of register, one of the following objects can be used:
  - An instance of a class derived from the `BroadcastReceiver` base class
  - An instance created from an anonymous class

Context-registered receivers (derived class)

```
class MainActivity : AppCompatActivity() {

    ...

    var exampleBR02 : ExampleBR02? = null

    fun registerExampleBR02() {
        exampleBR02 = ExampleBR02()
        registerReceiver(exampleBR02, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
    }

    override fun onPause() {
        super.onPause()
        if (exampleBR02 != null)
            unregisterReceiver(exampleBR02)
        ...
    }

    class ExampleBR02 : BroadcastReceiver() {
        override fun onReceive(context: Context?, intent: Intent?) {
            Log.i("ExampleBR02","onReceive: ")
        }
    }

    ...
}
```

Context-registered receivers (anonymous class)

```
class MainActivity : AppCompatActivity() {

    ...

    var exampleBR03 : BroadcastReceiver? = null

    fun registerExampleBR03() {
        exampleBR03 = object : BroadcastReceiver() {
            override fun onReceive(context: Context?, intent: Intent?) {
                Log.i("ExampleBR03","onReceive: ")
            }
        }
        registerReceiver(exampleBR03, IntentFilter(Intent.ACTION_BATTERY_CHANGED))
    }

    override fun onPause() {
        super.onPause()
        if (exampleBR03 != null)
            unregisterReceiver(exampleBR03)
        ...
    }

    ...
}
```

BR lifecycle

- As mentioned earlier, in the most recent versions, the application must be run at least once for the BRs to be activated
  - Applications must include at least one Activity
- A BR runs only for as long as necessary to execute the `onReceive` method
- If the process in which the BR can run is not active, the system will start it
- When a BR is dynamically registered, the application must cancel the BR registration before finishing

BR lifecycle

- The processing carried out in the `onReceive` method must not exceed 10 seconds
  - After this period, the system can terminate the BR process in question
    - Any threads that have been launched will be terminated if the process is finished

- BR runs in the context of the *main thread*
  - Attention: it may affect the user interaction within the current activity

https://developer.android.com/guide/components/activities/process-lifecycle

BR lifecycle

- If a BR needs more time to carry out the processing, then it can:
  - If some time is needed (< 10 seconds)
    - The `goAsync()` method should be used to signal this need

      ```
      val pr: PendingResult = goAsync()
      ```
    - Launch a *thread* or *coroutine* to perform the necessary operations
    - When the task ends, it must be signaled with

      ```
      pr.finish ()
      ```
  - If much more time is needed (> 10 sec)
    - Launch a `JobService` or `WorkManager` to perform necessary operations

Messages

- Messages that are sent to the BR are represented through instances of `Intent` objects
  - In the context of BR, an *intent* represents an event/action
    - It has information about the type of action
    - May have additional data with information about the action in question

Examples of Broadcasts

Example of additional information

```
exampleBR04 = object : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {

        val level = (100.0f * intent.getIntExtra(BatteryManager.EXTRA_LEVEL, 0)
                / intent.getIntExtra(BatteryManager.EXTRA_SCALE, 0))

        val on_ac = intent.getIntExtra(BatteryManager.EXTRA_PLUGGED, 0)

        val status = intent.getIntExtra(BatteryManager.EXTRA_STATUS,
                BatteryManager.BATTERY_STATUS_UNKNOWN)

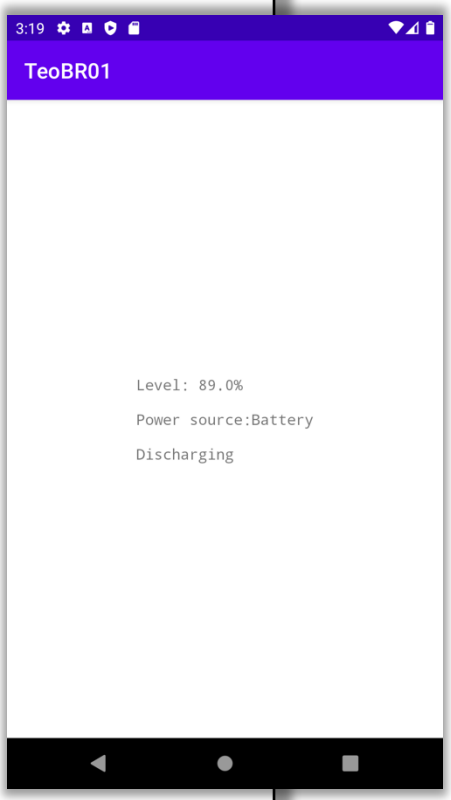
        var strStatus = "Level: $level%\n\n"

        strStatus += "Power source:${if (on_ac == 0) "Battery" else "AC"}\n\n"

        strStatus += when (status) {
            BatteryManager.BATTERY_STATUS_CHARGING -> "Charging"
            BatteryManager.BATTERY_STATUS_DISCHARGING -> "Discharging"
            BatteryManager.BATTERY_STATUS_FULL -> "Full charged"
            BatteryManager.BATTERY_STATUS_NOT_CHARGING -> "Not charging"
            else -> "Unknown state"
        }

        this@MainActivity.findViewById<TextView>(R.id.tvState).text = strStatus
    }
}

registerReceiver(exampleBR04,IntentFilter(Intent.ACTION_BATTERY_CHANGED))
```



Broadcast types

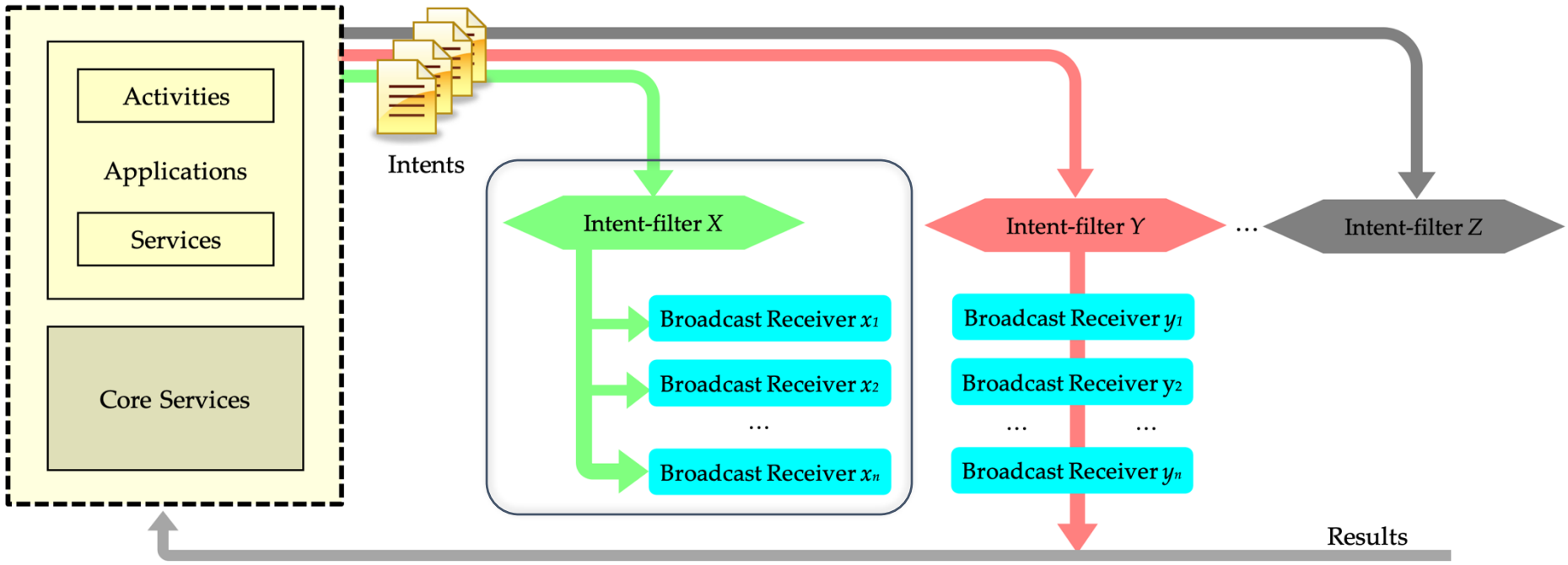
- Regarding their persistence and the way they are processed, broadcasts can be classified as:
  - "Normal" broadcasts
  - Ordered broadcasts
  - ~~*Sticky broadcasts*~~

API Level >= 21 → deprecated !!!!

“Normal” broadcasts

- These broadcasts are sent "simultaneously" to the different BRs that have registered interest in the broadcast in question

- Each BR carries out individual treatment according to the objectives they want to achieve



“Normal” broadcasts

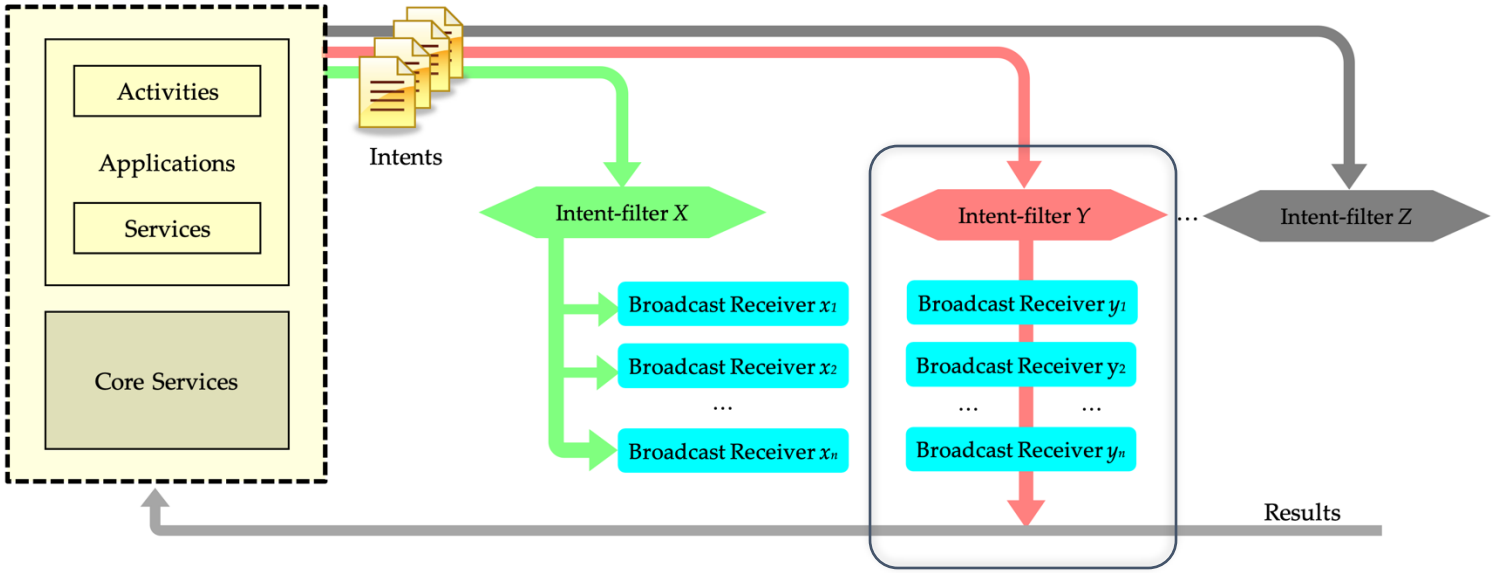
- An application can generate “normal” (unordered) broadcasts using the `sendBroadcast method`
- An *intent* can be defined using a *string* representing the event you want to announce

```
fun sendBroadcast(intent: Intent)

fun sendBroadcast(intent: Intent, receiverPermission: String)
```

Ordered broadcasts

- These broadcasts are sent to the various BRs in sequence
  - The broadcasts are sent following the descending order of each BR priority
    - Priorities are defined through an attribute of the structure `<intent-filter android:priority =“<xx>”... />` or through the respective property in the `IntentFilter class`
  - Results are sent between BRs and, in the end, a result is returned to another BR, which was defined when the broadcast was originated



Ordered broadcasts

- To generate ordered broadcasts, an application must use the `sendOrderedBroadcast` method

```
fun sendOrderedBroadcast( intent: Intent,
                receiverPermission: String)

fun sendOrderedBroadcast( intent: Intent,
                receiverPermission: String,
                resultReceiver: BroadcastReceiver,
                scheduler: Handler,
                initialCode: Int,
                initialData: String,
                initialExtras: Bundle)
```

Ordered broadcasts

- The processing sequence can be interrupted using the `abortBroadcast()` method

- Querying results generated by previous BRs or generating new results for subsequent processing can be carried out using the methods and properties made available for this purpose.

```
resultCode : Int
resultData : String
fun getResultExtras(makeMap : Boolean) : Bundle
fun setResultExtras(extras : Bundle)
fun setResult(code : Int, data : String, extras : Bundle)
```

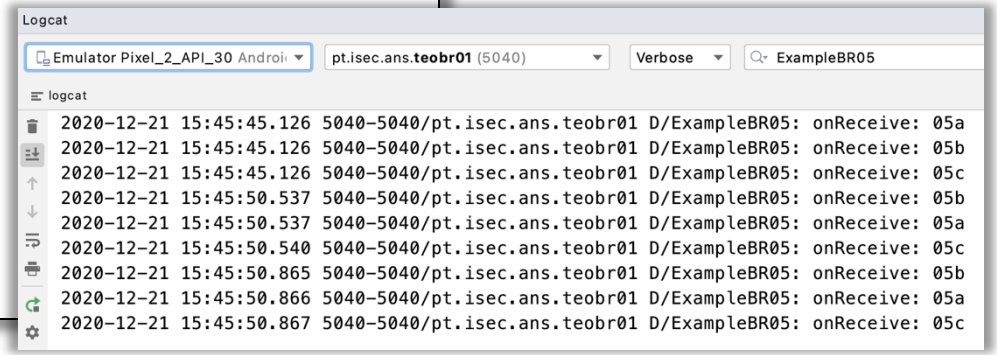
Example: ordered broadcasts

```
var exampleBR05a : BroadcastReceiver? = null
var exampleBR05b : BroadcastReceiver? = null
var exampleBR05c : BroadcastReceiver? = null

fun registerExampleBR05() {
    exampleBR05a = object : BroadcastReceiver() {
        override fun onReceive(context: Context, intent: Intent) {
            Log.d("ExampleBR05", "onReceive: 05a")
        }
    }
    exampleBR05b = object : BroadcastReceiver() {
        override fun onReceive(context: Context, intent: Intent) {
            Log.d("ExampleBR05", "onReceive: 05b")
        }
    }
    exampleBR05c = object : BroadcastReceiver() {
        override fun onReceive(context: Context, intent: Intent) {
            Log.d("ExampleBR05", "onReceive: 05c")
        }
    }
    val intFil1 = IntentFilter(Intent.ACTION_BATTERY_CHANGED)
    val intFil2 = IntentFilter(Intent.ACTION_BATTERY_CHANGED)
    val intFil3 = IntentFilter(Intent.ACTION_BATTERY_CHANGED)

    intFil1.priority = 100
    registerReceiver(exampleBR05a, intFil1)
    intFil2.priority = 200
    registerReceiver(exampleBR05b, intFil2)
    intFil3.priority =   0
    registerReceiver(exampleBR05c, intFil3)

}
```



~~Sticky broadcasts~~

API Level >= 21 → deprecated !!!!

- ~~They allow a BR to receive information about the latest broadcast even if the BR registered its interest at a later time~~
- ~~These are broadcasts whose *Intent objects* are temporarily stored in the system~~
- ~~The occurrence of a new broadcast of the same type leads to the overlapping of the previous *Intent*~~
- ~~For an application to send *sticky broadcasts* it must request the `BROADCAST_STICKY` permission~~

~~*Note: they should be avoided due to the additional weight they cause in the system*~~

~~Sticky~~

API Level >= 21 → deprecated !!!!

- ~~To generate *sticky broadcasts*, an application must use the methods~~

```
fun sendStickyBroadcast(intent : Intent)
fun sendStickyOrderedBroadcast(
            intent : Intent,
            resultReceiver :
BroadcastReceiver,
            scheduler : Handler,
            initialCode : Int,
            initialData : String,
            initialExtras : Bundle)
```

- ~~The information that is stored in the system is the one that was initially defined and not the result of changes by a BR~~

Defining custom broadcasts

- An application can generate their own broadcasts
  - A broadcast (action) is defined through a string of characters that identifies it

```
fun exampleBR06send() {
    val intent = Intent("pt.isec.ans.amov.mybroadcast")
    intent.putExtra("value",1234)
    sendBroadcast(intent)
}
```

Defining custom broadcast

- Processing a custom broadcast is done in the same way as other broadcasts
  - For the example in the previous slide…

```
class ExampleBR06a : BroadcastReceiver() {
    override fun onReceive(context: Context,intent: Intent) {
        Log.i("ExampleBR06a", "onReceive: ")
    }
}
```

```
<receiver
    android:name=".ExampleBR06a"
    android:enabled="true"
    android:exported="true">
    <intent-filter android:priority="100">
        <action android:name="pt.isec.ans.amov.mybroadcast" />
    </intent-filter>
</receiver>
```

  - Or… **(only way from API 26)**

```
fun registerExampleBR06b() {
    val intentFilter = IntentFilter("pt.isec.ans.amov.mybroadcast")
    intentFilter.priority = 200
    registerReceiver(ExampleBR06b(),intentFilter)
}
class ExampleBR06b : BroadcastReceiver() {
    override fun onReceive(context: Context, intent: Intent) {
        val value = intent.getIntExtra("value",-1)
        Log.i("ExampleBR06b", "onReceive: $value")
    }
}
```

Permissions

- Permissions may be required to...
  - Restrict the processing of broadcast received from certain sources
    - BR only processes the broadcast if it was sent with certain permissions
  - Restrict the processing only to authorized BRs
    - Broadcasts will only be processed if BR has permission to do so

Permission managing

- Define a new permission

```
<permission android:name ="pt.isec.ans.amov.myperm" />
```

- Request for a permission

```
<uses-permission android:name="pt.isec.ans.amov.myperm"/>
```

- Register a BR and setting a permission

```
registerReceiver(ExampleBR07(), intentFilter,
                          "pt.isec.ans.amov.myperm" ,null)
```

- Send a broadcast and restrict it with a permission

```
sendBroadcast(intent,"pt.isec.ans.amov.myperm")
```

setPackage

- To limit access to information present in a *broadcast* (`Intent`), the *Intent* that will be sent can be configured to be accessible only in a certain *package* (usually the application)
  - Use the method

```
Intent.setPackage(<package> : String )
```

Local Broadcast Manager

- The *Support Library* provides a class, `LocalBroadcastManager`, that allows:
  - To register local BR
  - To send local *Broadcasts* (for the same process)

  - Advantages:
    - Security
      - Does not allow other processes to access information sent in the context of *Intents*
    - Efficiency
      - The messages with only local purposes will not overload the system and other processes

Usage examples

- In addition to uses within applications themselves (custom broadcasts), there are many situations in which BRs can be used
- Exercise examples
  - Detect external power supply connection
    - There are some features of an application that may be delayed until the device is connected to an external power source
      - As already studied, this can also be achieved with `WorkManager` or `JobScheduler`
  - SMS processing and geolocation
    - Developing an anti-theft system
      - Allows to locate cell phones or other objects to which these cell phones are associated
      - Allows remote consultation of the object's location by sending an SMS to the “lost” cell phone
        - The SMS must contain text that does not induce suspicion
      - It is assumed that the cell phone owner has previously installed the application and has run it at least once
      - Note: the obtaining of geographic locations is done using other ways that will be taught in practical lessons, they are not related with broadcasts or BR

Example: SMS processing

```
class SMSReceiver : BroadcastReceiver() {
    companion object {
        private const val TAG = "SMSReceiver"
        private const val SEARCH = "ISEC"
        private const val ANSWER = "Thanks"
    }
    @Suppress("DEPRECATION")
    override fun onReceive(context: Context, intent: Intent) {
        Toast.makeText(context, "$TAG: SMS received", Toast.LENGTH_LONG).show()

        val messages = Telephony.Sms.Intents.getMessagesFromIntent(intent)
        messages?.forEach { message ->
            val origin = message.displayOriginatingAddress
            val msg = message.displayMessageBody
            Log.i(TAG, "Origin: $origin; Msg: $msg")
            if (msg.contains(SEARCH)) {
                abortBroadcast()
                Log.i(TAG, "Contains $SEARCH")
                val smsManager: SmsManager =  SmsManager.getDefault()
                smsManager.sendTextMessage(origin, null, ANSWER, null, null)
                Log.i(TAG, "Message \"$ANSWER\" sent")
            }
        }
    }
}
```

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest ... >

    <uses-permission android:name="android.permission.RECEIVE_SMS" />
    <uses-permission android:name="android.permission.SEND_SMS" />

    <application ... >

        <receiver
            android:name=".SMSReceiver" android:enabled="true" android:exported="true"
            android:permission="android.permission.BROADCAST_SMS">
            <intent-filter>
                <action android:name="android.provider.Telephony.SMS_RECEIVED" />
            </intent-filter>
        </receiver>

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

MainActivity

```
class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        if (ActivityCompat.checkSelfPermission( this, RECEIVE_SMS)
            != PERMISSION_GRANTED ||
            ActivityCompat.checkSelfPermission( this, SEND_SMS)
            != PERMISSION_GRANTED ) {
            requestPermissionsResult.launch(
                arrayOf(RECEIVE_SMS, SEND_SMS)
            )
        }
    }

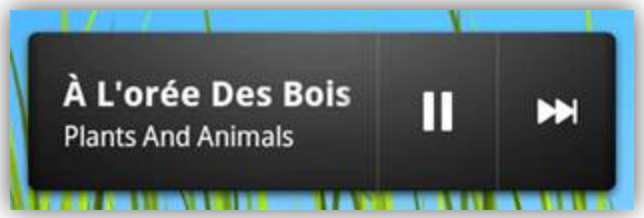
    private val requestPermissionsResult = registerForActivityResult(
        ActivityResultContracts.RequestMultiplePermissions()
    ) { permissions ->
          permissions.values.forEach { result -> if (!result) finish() }
    }
}
```

Android Widgets

Creating widgets for an application

Widgets

- Graphic components that can be included in the context of other applications
  - Usually widgets are related to the *Android Launcher* (*Home Screen*)

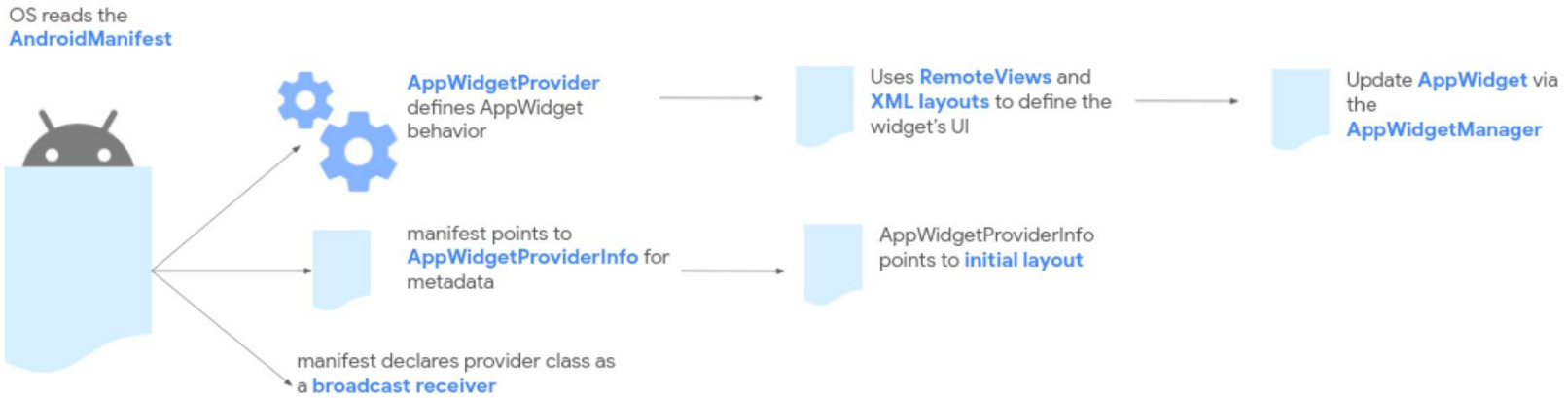




- Widgets allow:
  - Quick access to applications
  - Change or display the state of a component
  - Customize the interface

Widgets

- Widgets are implemented as a *Broadcast Receiver* extension
  - Declared with the tag `<receiver ... />` in the manifest file
- Consisting of the following elements
  - `AppWidgetProviderInfo`
    - Structure defined in XML
  - *Layout*
    - XML structure that defines the appearance of the *widget*
    - Defined in a similar way to activity *layouts*
  - `AppWidgetProvider`
    - Class that extends the `BroadcastReceiver` class
  - (Optional) Activity for configuring the *widget*



Widgets (Jetpack Compose)

- To develop widgets using Jetpack Compose, the *Jetpack Glance* framework can be used
  - Note: "*Jetpack Glance is in active development*"
  - https://developer.android.com/develop/ui/compose/glance

AppWidgetProviderInfo

- Structure defined through an XML file, placed in `<dir_proj>/res/xml`, including:
  - Minimum widget width and height
    - Formula used to calculate dimensions: 70 x *nr_cells* - 30
  - Interval between updates
  - Layout for the *widget*
  - Declaration of an activity for configuration (optional)

```
<appwidget-provider xmlns:android="http://schemas.android.com/apk/res/android"
    android:configure="pt.isec.amov.widgetapp.AppWidgetConfigureActivity"
    android:description="@string/app_widget_description"
    android:initialKeyguardLayout="@layout/app_widget"
    android:initialLayout="@layout/app_widget"
    android:minWidth="180dp"
    android:minHeight="110dp"
    android:previewImage="@drawable/appwidget_preview"
    android:previewLayout="@layout/app_widget"
    android:resizeMode="horizontal|vertical"
    android:targetCellWidth="3"
    android:targetCellHeight="2"
    android:updatePeriodMillis="86400000"
    android:widgetCategory="home_screen" />
```

XML Layout – Allowed objects

- Organization
  - `AdapterViewFlipper`
  - `FrameLayout`
  - `GridLayout`
  - `GridView`
  - `ListView`
  - `LinearLayout`
  - `RelativeLayout`
  - `StackView`
  - `ViewFlipper`
- Others
  - `AnalogClock`
  - `Button`
  - `Chronometer`
  - `ImageButton`
  - `ImageView`
  - `ProgressBar`
  - `TextClock`
  - `TextView`
  - `CheckBox`
  - `RadioButton`
  - `RadioGroup`
  - `Switch`

AppWidgetProvider

- Methods:
  - `fun onReceive(context: Context?, intent: Intent?)`
    - Usually this method is not overridden
    - Allows broadcasts to be forwarded to one of the following methods
  - `fun onUpdate(context:Context, appWidgetManager: AppWidgetManager,appWidgetIds: IntArray)`
    - Broadcast : `ACTION_APPWIDGET_UPDATE`
    - Method where the main configurations of the widget are done
  - `fun onDeleted(context: Context, appWidgetIds:IntArray)`
    - Broadcast : `ACTION_APPWIDGET_DELETED`
    - Method called when one or more widgets have been removed

AppWidgetProvider (cont.)

- Methods (cont.):
  - `fun onEnabled(context: Context)`
    - Broadcast : `ACTION_APPWIDGET_ENABLED`
    - Method called when the first widget is instantiated
  - `fun onDisabled(context: Context)`
    - Broadcast : `ACTION_APPWIDGET_DISABLED`
    - Method called when the last widget is removed
  - `fun onAppWidgetOptionsChanged(context: Context?, appWidgetManager: AppWidgetManager?, appWidgetId: Int, newOptions: Bundle?)`
    - Broadcast : `ACTION_APPWIDGET_OPTIONS_CHANGED` (API>=16)
    - Method that is called when the widget basic information changes (e.g. dimension)

AppWidgetProvider

- Implementation example

```
class AppWidget : AppWidgetProvider() {
    override fun onUpdate(context: Context, appWidgetManager: AppWidgetManager,
                                                                appWidgetIds: IntArray) {
        // There may be multiple widgets active, so update all of them
        for (appWidgetId in appWidgetIds)
            updateAppWidget(context, appWidgetManager, appWidgetId)
    }
    // When the user deletes the widget, delete the preference associated with it.
    override fun onDeleted(context: Context, appWidgetIds: IntArray) {
        for (appWidgetId in appWidgetIds)
            deleteTitlePref(context, appWidgetId) //optional.. assuming SharedPreferences
    }
    // When the first widget is created
    override fun onEnabled(context: Context) { }
    // When the last widget is disabled
    override fun onDisabled(context: Context) { }
}
internal fun updateAppWidget(context: Context, appWidgetManager: AppWidgetManager,
                                                                appWidgetId: Int) {
    val widgetText = loadTitlePref(context, appWidgetId) //optional.. SharedPreferences
    // Construct the RemoteViews object
    val views = RemoteViews(context.packageName, R.layout.app_widget)
    views.setTextViewText(R.id.appwidget_text, widgetText)

    // Instruct the widget manager to update the widget
    appWidgetManager.updateAppWidget(appWidgetId, views)
}
```

Widget

- It must be ensured that the *widget* is properly declared in the manifest file

```
<receiver
    android:name=".AppWidget"
    android:exported="true">
    <intent-filter>
        <action android:name="android.appwidget.action.APPWIDGET_UPDATE" />
    </intent-filter>

    <meta-data
        android:name="android.appwidget.provider"
        android:resource="@xml/appwidget_info" />
</receiver>
```

Widget

- The activity must be associated with the `ACTION_APPWIDGET_CONFIGURE` action in the manifest file

```
<activity android:name=".AppWidgetConfigureActivity" android:exported="true">
    <intent-filter>
        <action android:name="android.appwidget.action.APPWIDGET_CONFIGURE" />
    </intent-filter>
</activity>
```

... and, as mentioned, it must be associated with the *widget* through the `AppWidgetProviderInfo` structure

```
<appwidget-provider xmlns:android="http://schemas.android.com/apk/res/android"
    android:configure="pt.isec.amov.widgetapp.AppWidgetConfigureActivity"
    ... />
```

Activity: Widget

- If an activity is defined to configure the *widget*, then the first *update* is not sent
  - It is necessary to force the *update*
- Steps to follow
  - Get the Widget ID

```
val extras = intent.extras
if (extras != null) {
    mAppWidgetId = extras.getInt(AppWidgetManager.EXTRA_APPWIDGET_ID,
        AppWidgetManager.INVALID_APPWIDGET_ID)
}
```

  - Carry out the desired configuration processes
    - Normal operation of the activity

Activity: Widget

- After the configuration is finished (e. g., pressing the button "*Add Widget*" or similar)
  - Obtain the reference to the widget manager

```
val appWidgetManager = AppWidgetManager.getInstance(context)
```

  - Force the update of the remote views associated with the widget

```
val views = RemoteViews(context.packageName,R.layout.appwidget_provider_layout)
appWidgetManager.updateAppWidget(mAppWidgetId, views);
```

  - Create an intent to return values among which the widget ID must be sent, and end the activity

```
val resultValue = Intent()
resultValue.putExtra(AppWidgetManager.EXTRA_APPWIDGET_ID, appWidgetId)
setResult(RESULT_OK, resultValue)
finish()
```

  - If the widget should not be created (e. g., back button was pressed) the result `RESULT_CANCELED` must be returned

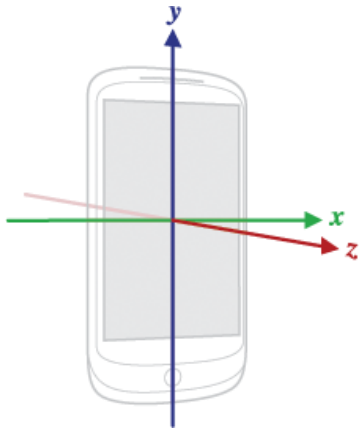
Android

Use of Sensors

Example of sensors

Sensors

- An Android application can take advantage of the various sensors available on the device, for example, to improve interaction with the user or to adapt to the environment (contextualize)



- Various types of sensors:
  - Movement
    - Gravitational forces and rotational forces
    - Examples: accelerometer, gyroscope, rotation, gravity
  - Position
    - Physical position of the device
    - Examples: orientation, magnetometers
  - Environmental
    - Measurement of environmental values
    - Examples: temperature, pressure, luminosity, humidity

Sensors

Sensor	Type	Description	Common Uses
TYPE_ACCELEROMETER	Hardware	Measures the acceleration force in m/s^2 that is applied to a device on all three physical axes (x, y, and z), including the force of gravity.	Motion detection (shake, tilt, etc.).
TYPE_AMBIENT_TEMPERATURE	Hardware	Measures the ambient room temperature in degrees Celsius (°C). See note below.	Monitoring air temperatures.
TYPE_GRAVITY	Software or Hardware	Measures the force of gravity in m/s^2 that is applied to a device on all three physical axes (x, y, z).	Motion detection (shake, tilt, etc.).
TYPE_GYROSCOPE	Hardware	Measures a device's rate of rotation in rad/s around each of the three physical axes (x, y, and z).	Rotation detection (spin, turn, etc.).
TYPE_LIGHT	Hardware	Measures the ambient light level (illumination) in lx.	Controlling screen brightness.
TYPE_LINEAR_ACCELERATION	Software or Hardware	Measures the acceleration force in m/s^2 that is applied to a device on all three physical axes (x, y, and z), excluding the force of gravity.	Monitoring acceleration along a single axis.
TYPE_MAGNETIC_FIELD	Hardware	Measures the ambient geomagnetic field for all three physical axes (x, y, z) in μT.	Creating a compass.

Sensors

`TYPE_ORIENTATION`	Software	Measures degrees of rotation that a device makes around all three physical axes (x, y, z). As of API level 3 you can obtain the inclination matrix and rotation matrix for a device by using the gravity sensor and the geomagnetic field sensor in conjunction with the `getRotationMatrix()` method.	Determining device position.
`TYPE_PRESSURE`	Hardware	Measures the ambient air pressure in hPa or mbar.	Monitoring air pressure changes.
`TYPE_PROXIMITY`	Hardware	Measures the proximity of an object in cm relative to the view screen of a device. This sensor is typically used to determine whether a handset is being held up to a person's ear.	Phone position during a call.
`TYPE_RELATIVE_HUMIDITY`	Hardware	Measures the relative ambient humidity in percent (%).	Monitoring dewpoint, absolute, and relative humidity.
`TYPE_ROTATION_VECTOR`	Software or Hardware	Measures the orientation of a device by providing the three elements of the device's rotation vector.	Motion detection and rotation detection.
`TYPE_TEMPERATURE`	Hardware	Measures the temperature of the device in degrees Celsius (°C). This sensor implementation varies across devices and this sensor was replaced with the `TYPE_AMBIENT_TEMPERATURE` sensor in API Level 14	Monitoring temperatures.

Sensors

- Other sensors
  - *Step detector*
    - `Sensor.TYPE_STEP_DETECTOR`
    - Allows the detection of every step that is taken
  - *Step counter*
    - `Sensor.TYPE_STEP_COUNTER`
    - Allows to obtain the total steps since the last reboot
    - This sensor may have a delay in providing information, but it provides more accurate information than the previous one
  - To be used, both previous sensors need to request permission:

    `android.permission.ACTIVITY_RECOGNITION` (API29+)

Relevant classes and interfaces

- `SensorManager`
  - Provides basic methods for listing sensors, registering interest in sensors, obtaining values/constants, …
- `Sensor`
  - Represents a sensor
  - Provides methods to check sensor capabilities
- `SensorEvent`
  - Represents the information generated by a given sensor when an event occurs
  - Includes time stamp when the information is obtained
- `SensorEventListener`
  - Interface implemented to define callbacks to receive events

Sensors

- The various sensors available are accessed as follows
  - Get reference to the `SensorManager` through the method `getSystemService(SENSOR_SERVICE)`
  - Get the desired sensor using the method `getDefaultSensor(sensorType: int)`
  - Register a `SensorEventListener` for the sensor in question, in order to update the values
    - Activate a listener's `onSensorChanged` method to obtain the values of the sensor in question through the '`values`' array passed in the event
  - When the sensor becomes unnecessary, the listener must be unregistered using the method:
    - *`SensorManager`*`.unregisterListener(…)`

Sensors

- Example of listing all sensors and listener registration

```
fun showSensors() {
    val sm = getSystemService(SENSOR_SERVICE) as SensorManager

    val lstSensors = sm.getSensorList(Sensor.TYPE_ALL)

    lstSensors.forEach {
        Log.i("Sensor list", "Sensor: " +
                "${it.stringType}, ${it.type}, ${it.name}, " +
                "0-${it.maximumRange}, ${it.power} mA, " +
                "${it.resolution}, ${it.vendor}, ${it.version}")
        sm.registerListener(sensorListener,it,
                                    SensorManager.SENSOR_DELAY_NORMAL)
        //Don't forget to do sm.unregisterListener(sensorListener)
    }
}
```

SensorEventListener

```
val sensorListener = object : SensorEventListener {
    override fun onSensorChanged(event: SensorEvent?) {
        if (event == null)
            return
        val sensor = event.sensor
        var str = "${sensor.stringType} "
        event.values.forEach { str += String.format(" %.4f",it)
}
        Log.i("Sensor", str)
    }
    override fun onAccuracyChanged(sensor: Sensor?, accuracy:
Int) { }
}
```

Homework exercise

- Make an application that allows to show the number of (walking) steps taken
  - Use one of the sensors...
    - `Sensor.TYPE_STEP_COUNTER`
    - `Sensor.TYPE_STEP_DETECTOR`

Example with accelerometer

- Development of an application that allows to control a ball on the screen using the accelerometer

- Other features to test
  - Lock screen orientation (in this case, landscape)
  - Keep the screen on while using the application

Register application requirements

- Applications can register resource needs so that they can run correctly
  - If a device does not have the characteristics considered necessary, the application in question will not appear as an option in the *Play Store* by *Google* if a user searches for it

- A `uses-feature` XML structure must be added to the manifest file :

```
<uses-feature android:name ="<feature>" />
```

- In this example, when using the accelerometer:

```
<uses-feature android:name="android.hardware.sensor.accelerometer"/>
```

Fullscreen

- Use the `setSystemUIVisibility` function with the following *flags*

```
window.decorView.systemUiVisibility = (
            View.SYSTEM_UI_FLAG_FULLSCREEN or
            View.SYSTEM_UI_FLAG_HIDE_NAVIGATION or
            View.SYSTEM_UI_FLAG_IMMERSIVE_STICKY or
            View.SYSTEM_UI_FLAG_LAYOUT_FULLSCREEN or
            View.SYSTEM_UI_FLAG_LAYOUT_HIDE_NAVIGATION or
            View.SYSTEM_UI_FLAG_LAYOUT_STABLE )
```

- If the `ActionBar` is not to be shown, applications must choose themes that do not include it
  - Can only be applied to the intended activity

```
<style name="Theme.AMovSensores"
       parent="Theme.MaterialComponents.Light.NoActionBar">
</style>
```

- More information at
  - https://developer.android.com/training/system-ui/immersive

- From API 30 onwards, `WindowInsetsController` and/or `window.setDecorFitsSystemWindows(...)` must be used

Force screen orientation

- One of the ways that can be used to force and/or fix the screen orientation of an activity is to specify the `screenOrientation` attribute when registering it in the manifest file

```
<activity android:name=".MainActivity”
      android:screenOrientation="landscape"
      android:exported="true">
    ...
</activity>
```

Keep the screen on

- To request that the screen stays on, the `keepScreenOn` attribute must be added to the base element of the activity layout

```
<FrameLayout ... android:keepScreenOn="true" > ...
```

- Alternatively, it can be managed at *runtime*

```
window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
```

... and ...

```
window.clearFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
```

Board

- Create a *Custom View* for the game board on which the ball will be controlled
  - The view must be added to the activity's base *layout (XML)*
    - In order for it to be added directly to the *layout*, the constructor must receive the set of attributes
  - The `Board` object will be responsible for processing the information received from the accelerometer

```
class Board(context: Context, attrs: AttributeSet) :
        View(context,attrs), SensorEventListener { ... }
```

Next steps

- Test existence and obtain reference for the accelerometer sensor
  - Activate and deactivate the sensor in the `onResume` and `onPause` methods, respectively
  - Process events, with a `SensorEventListener`, in the `Board` class

`onResume:`

```
sm.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)?.also {
 sensor ->
        sm.registerListener(
              b.board,
              sensor,
              SensorManager.SENSOR_DELAY_GAME)
}
```

`onPause:`

```
sm.unregisterListener(b.board)
```

Defining sprites

- *Sprite* is a concept normally used in 2D games that corresponds to objects present on the game board ("world")
- In this example...
  - An abstract `Sprite` class was defined to encapsulate the concepts of the elements belonging to the game
    - Position, limits and dimension (*x*, *y*, *width*, *height*, …)
    - Motion (*speedX*, *speedY*, …)
      - Methods to apply forces to objects
      - Methods for detecting collisions
    - Design
      - `onDraw` abstract method
  - A `Ball` class was derived from the `Sprite` class to implement the ball to be controlled on the screen
    - Draw the ball with `drawCircle` or…
    - Draw a bitmap with `drawBitmap`

Sprite Class

```
const val WALL_FORCE = 0f
open abstract class Sprite(var x: Float, var y: Float,
                           var width: Float, var height: Float,
                           var speedX: Float, var speedY: Float,
                           var isStatic: Boolean) {
    var minX = 0f
    var maxX = 1000f
    var minY = 0f
    var maxY = 1000f
    var wallForce = WALL_FORCE

    abstract fun onDraw(c: Canvas?)

    fun setLimits(minX: Float, maxX: Float,
                  minY: Float, maxY: Float) {
        this.minX = minX
        this.maxX = maxX
        this.minY = minY
        this.maxY = maxY
    }
...
```



Sprite Class (cont .)

```
...
    fun getRect(): RectF {
        return RectF(x - width / 2, y - height / 2,
                x + width / 2, y + height / 2)
    }
    fun collision(x: Float, y: Float): Boolean {
        return getRect().contains(x, y)
    }
    fun collision(r: RectF?): Boolean {
        return getRect().intersect(r!!)
    }
    fun collision(other: Sprite): Boolean {
        return getRect().intersect(other.getRect())
    }
    open fun applyAcceleration(accX: Float, accY: Float,
                               elapsedTime: Float) {
        if (isStatic) return
        speedX += elapsedTime * accX
        speedY += elapsedTime * accY
        animate(elapsedTime)
    }
...
```



Sprite Class (cont .)

```
 open fun animate(elapsedTime: Float) {
        if (speedX == 0f && speedY == 0f) return

        val tempX = x + speedX * elapsedTime
        if (x < minX + width / 2) {
            x = minX + width / 2
            speedX = if (isStatic) -speedX else -wallForce * speedX
        } else if (x > maxX - width / 2) {
            x = maxX - width / 2
            speedX = if (isStatic) -speedX else -wallForce * speedX
        } else
            x = tempX

        val tempY = y + speedY * elapsedTime
        if (y < minY + height / 2) {
            y = minY + height / 2
            speedY = if (isStatic) -speedY else -wallForce * speedY
        } else if (y > maxY - height / 2) {
            y = maxY - height / 2
            speedY = if (isStatic) -speedY else -wallForce * speedY
        } else
            y = tempY
    }
}
```



Ball

```
private const val BALL_RADIUS = 20f

class Ball(x : Float, y : Float, var color : Int = Color.BLUE) :
        Sprite(x,y,2*BALL_RADIUS,2*BALL_RADIUS,0f,0f,false) {

    var paint = Paint(DITHER_FLAG).apply {
        style = Paint.Style.FILL_AND_STROKE
        color = this@Ball.color
        strokeWidth = 2f
    }

    init {
        wallForce = 0.7f
    }

    override fun onDraw(c: Canvas?) {
        if (c==null) return
        c.drawCircle(x, y, BALL_RADIUS, paint)
    }
}
```

Board

```
private const val SPEEDFACTOR = 5f
class Board(context: Context?, var backColor: Int = Color.WHITE) :
        View(context), SensorEventListener {
    var lstSprites: ArrayList<Sprite> = ArrayList()

    init {
        setBackgroundColor(backColor)
        lstSprites.add(Ball(100f, 100f, Color.rgb(0, 0, 128)))
    }

    override fun onDraw(canvas: Canvas?) {
        super.onDraw(canvas)
        for (s in lstSprites) { s.onDraw(canvas) }
    }
    override fun onSizeChanged(w: Int, h: Int, oldw: Int, oldh: Int) {
        super.onSizeChanged(w, h, oldw, oldh)
        for (s in lstSprites) {
            s.setLimits(0f, w.toFloat(), 0f, h.toFloat())
        }
    }
    fun applyAcceleration(accX: Float, accY: Float,
                          elapsedTime: Float) {
        for (s in lstSprites) {
            s.applyAcceleration(accX, accY, elapsedTime)
        }
        invalidate()
    }
    ...
}
```

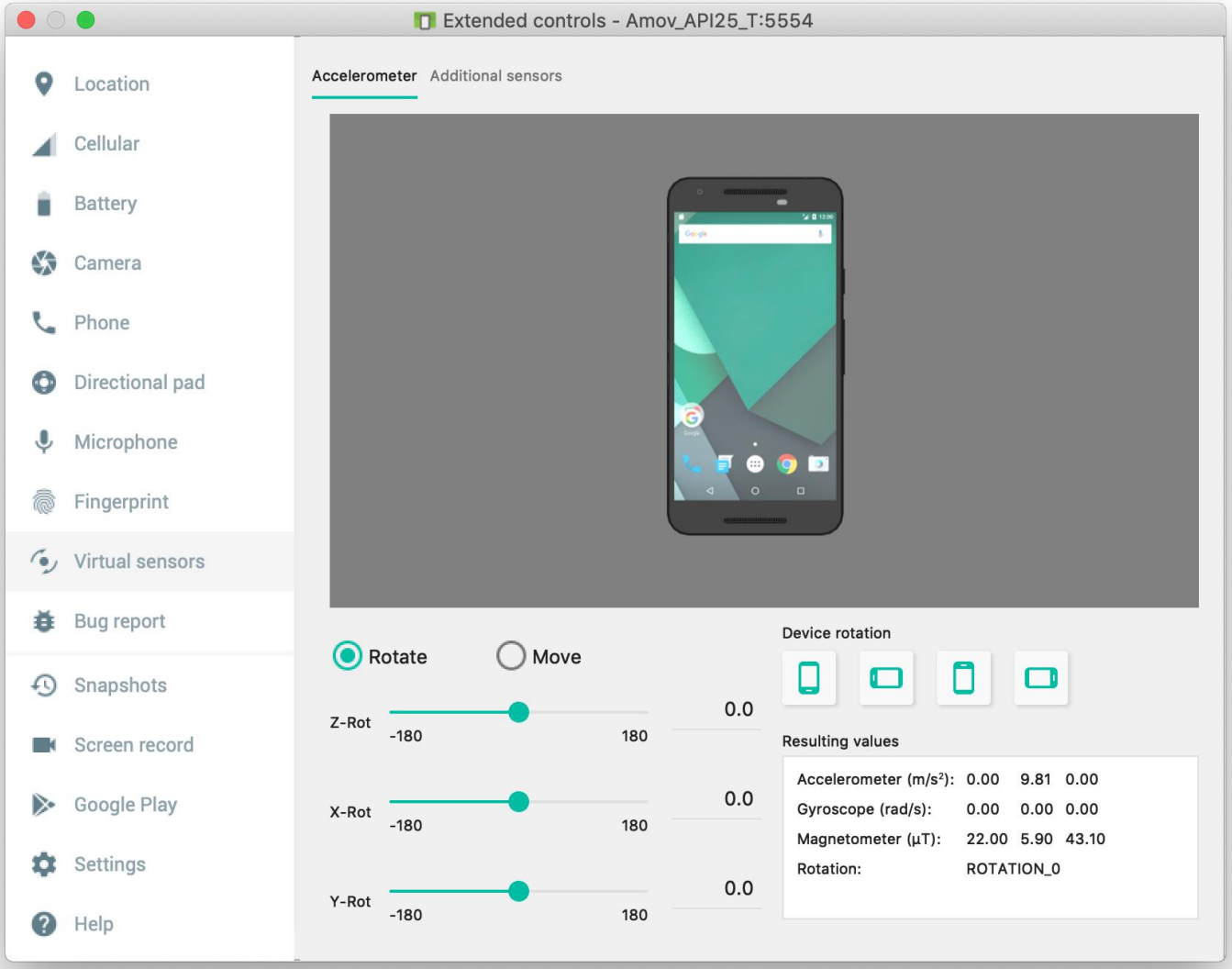
Board (accelerometer proc.)

```
var lasttime: Long = -1

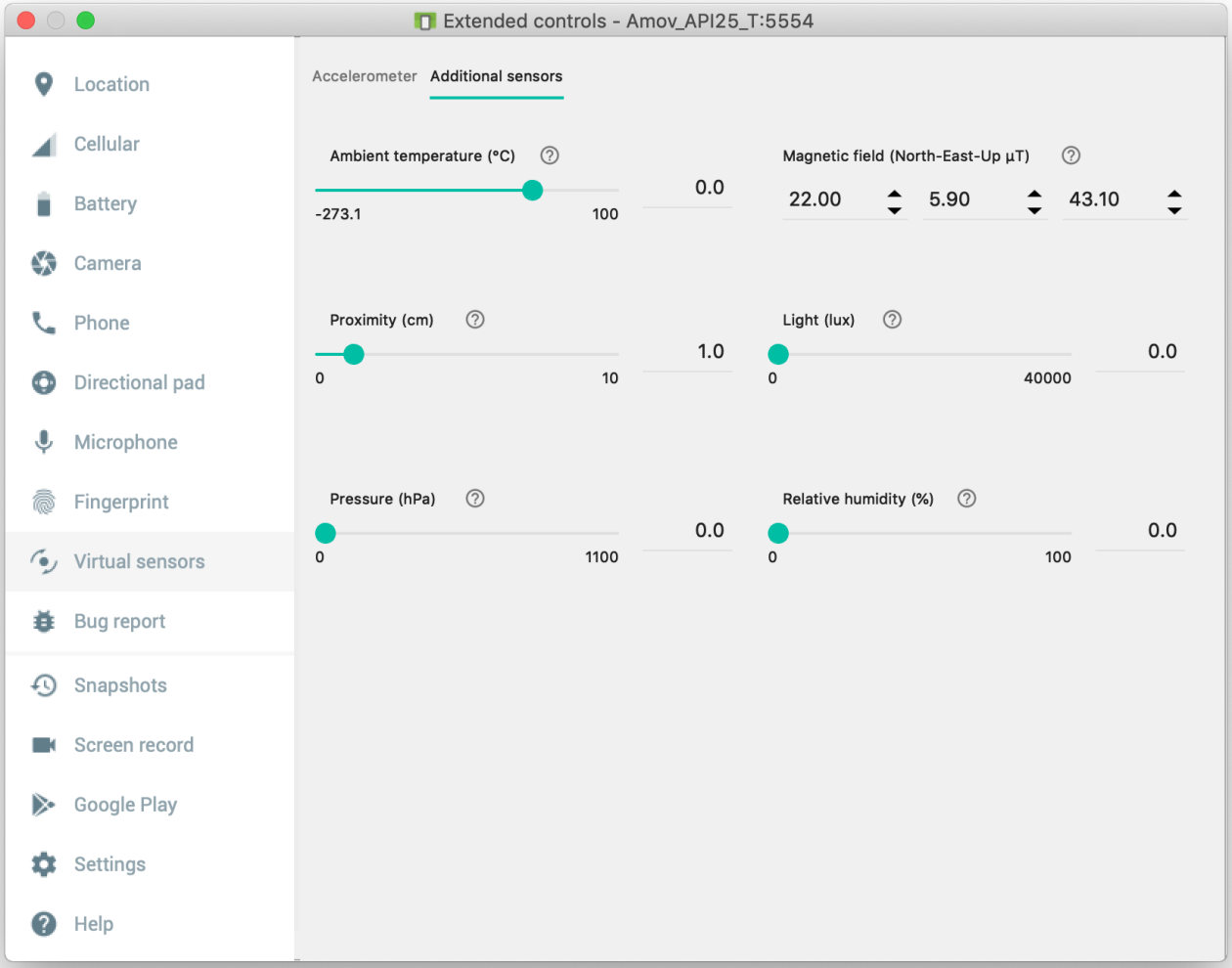
override fun onSensorChanged(event: SensorEvent) {
    if (event.sensor.type == Sensor.TYPE_ACCELEROMETER && display != null) {
        if (lasttime > 0) {
            var sx = 0f
            var sy = 0f
            when (display!!.rotation) {
                Surface.ROTATION_0   ->{ sx = -event.values[0]; sy = event.values[1]  }
                Surface.ROTATION_90  ->{ sx = event.values[1];  sy = event.values[0]  }
                Surface.ROTATION_180 ->{ sx = event.values[0];  sy = -event.values[1] }
                Surface.ROTATION_270 ->{ sx = -event.values[1]; sy = -event.values[0] }
            }
            val deltatime = SPEEDFACTOR * (event.timestamp - lasttime) / 1000000000.0f
            applyAcceleration(sx, sy, deltatime)
        }
        lasttime = event.timestamp
    }
}

override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {}
```

Test with emulators



Test with emulators



Improvements

- Improvements...
  - Change the ball's circle to a suitable bitmap
  - Change the elements present on the board
  - Take into account collisions between elements

- For example...
  - Add a component, *target*, which will have to be reached by the ball
  - Similar to the previous but the *target* has a decreasing seconds counter (10..0) and must be reached before getting to 0
  - When it is reached, it changes place on the screen and starts the time again
  - When the user hits the target ten times they win the game
  - If the countdown timer reaches 0, the game ends



Android developing

Access Web Pages

Access Web Services (JSON)

Web Access

- For accessing a web page:
  - Include internet access permission in the manifest file

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
```

  - Use a client object for the HTTP protocol
    - HttpURLConnection or HttpsURLConnection
      - To use clear text requests, from API 28 onwards it is necessary to add the `android:usesCleartextTraffic="true"` attribute to the application structure of the manifest file
  - The request must be made in the context of a secondary thread

getData

```
const val LIMIT = 4096
...
private fun getData(strUrl: String?): String? {
    val resp = StringBuilder()
    var conn: HttpURLConnection? = null
    try {
        val url = URL(strUrl)
        conn = url.openConnection() as HttpURLConnection
        conn.readTimeout = 10000
        conn.connectTimeout = 15000
        conn.requestMethod = "GET"
        conn.doInput = true
        conn.connect()
        val code: Int = conn.responseCode
        if (code == HttpURLConnection.HTTP_OK /*200*/) {
            var count = 0
            conn.inputStream.bufferedReader().forEachLine {
                count++
                if (count >= LIMIT)
                    return@forEachLine
                resp.append(it + "\n")
            }
        } else {
            resp.append("Error: $code")
        }
    } catch (_: Exception) {
        return null
    } finally {
        conn?.inputStream?.close()
        conn?.disconnect()
    }
    return resp.toString()
}
```

getDataAsync

- Call the getData function in the context of a secondary *thread*

```
fun getDataAsync(strURL: String, onResult : (String?) -> Unit) {
    thread {
        onResult(getData(strURL))
    }
}
```

- Example of use

```
val webContent : MutableLiveData<String?> = MutableLiveData()
getDataAsync("https://www.isec.pt") { result ->
    webContent.postValue(result)
}
```

Check network status

```
fun verifyNetworkState(context: Context): Boolean {
    val connMgr = context.getSystemService(Context.CONNECTIVITY_SERVICE)
                                                    as ConnectivityManager?
    val networkInfo = connMgr?.activeNetworkInfo
    if (networkInfo != null && networkInfo.isConnected)
        return true
    return false
}

...
    if (!NetUtils.verifyNetworkState(this)) {
        Toast.makeText(this,"No network available",Toast.LENGTH_LONG).show()
        finish()
        return
    }
...
```

Check network status

```
fun verifyNetworkStateV2(context: Context):Boolean {
    val connMgr = context.getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
    connMgr.allNetworks.forEach { network ->
        connMgr.getNetworkCapabilities(network).apply {
            if (this?.hasTransport(NetworkCapabilities.TRANSPORT_WIFI)     == true ||
                this?.hasTransport(NetworkCapabilities.TRANSPORT_CELLULAR) == true)
                    return true
        }
    }
    return false
}

...
    if (!NetUtils.verifyNetworkStateV2(this)) {
        Toast.makeText(this,"No network available",Toast.LENGTH_LONG).show()
        finish()
        return
    }
...
```

Check network status

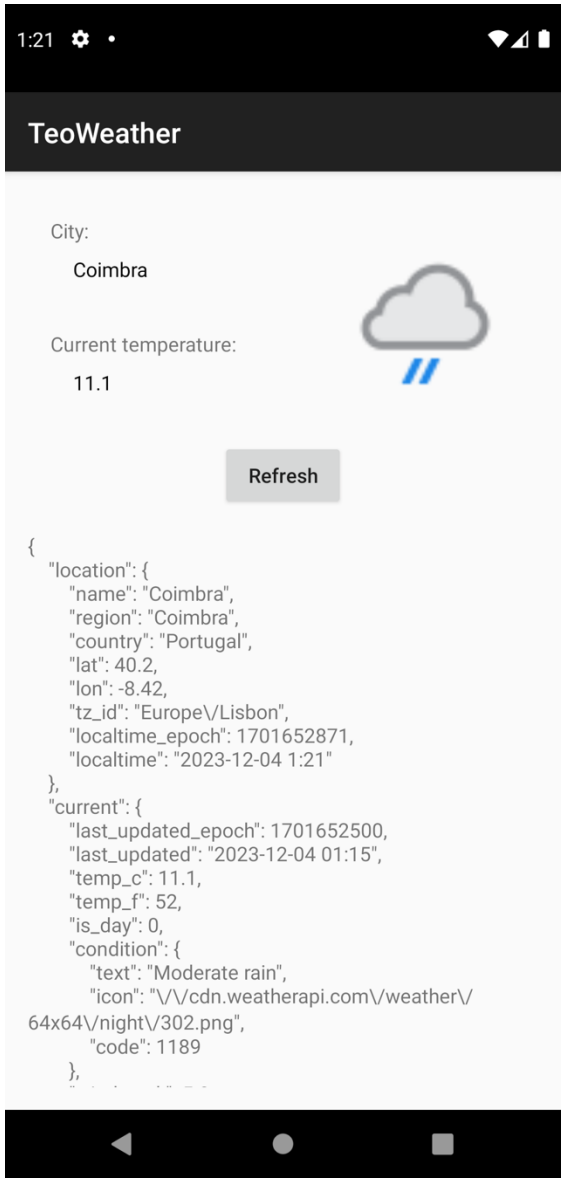
```
fun verifyNetworkStateV3(context: Context):Boolean {
    val connMgr =
              context.getSystemService(Context.CONNECTIVITY_SERVICE) as ConnectivityManager
    with(connMgr.activeNetwork) {
        with(connMgr.getNetworkCapabilities(this)) {
            if (this?.hasTransport(NetworkCapabilities.TRANSPORT_WIFI) == true ||
                this?.hasTransport(NetworkCapabilities.TRANSPORT_CELLULAR) == true)
                return true
        }
    }
    return false
}

...
    if (!NetUtils.verifyNetworkStateV3(this)) {
        Toast.makeText(this,"No network available",Toast.LENGTH_LONG).show()
        finish()
        return
    }
...
```

Exercise with WebServices

- Develop an application to present current weather information for Coimbra
  - Use the service provided by *WeatherAPI*
    - https://www.weatherapi.com
    - It is necessary to register to have an API Key
  - Example of a request to get information in JSON format

```
https://api.weatherapi.com/v1/forecast.json?
      key=<APIKey>&q=Coimbra&days=2&aqi=no&alerts=no
```



Weather Example : JSON

- Simple format for representing data structures, alternative to *XML* but with similar objectives
  - Example:

```
{
    "name": "Zeferino Pancracio",
    "age": 57,
    "emails" : [
          "zefe@world.com",
          "zepa@gmail.com"
    ]
}
```

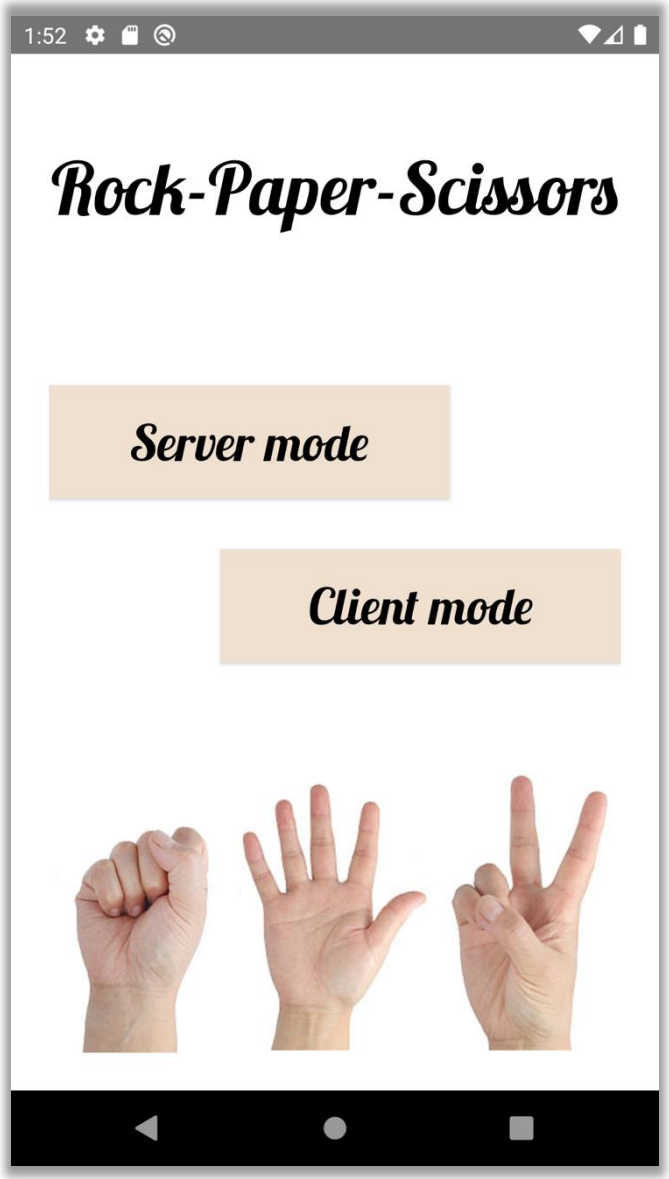
- Two types of objects
  - `JSONObject ( { … } )`
    - Data stored according to the model `<attribute>: <value>`
  - `JSONArray ( [ … ] )`
    - Data stored as a sequence of values separated by ','
    - Values can be objects `JSONObject` or `JSONArray`

Multiplayer game

Rock-Paper-Scissors Game Development

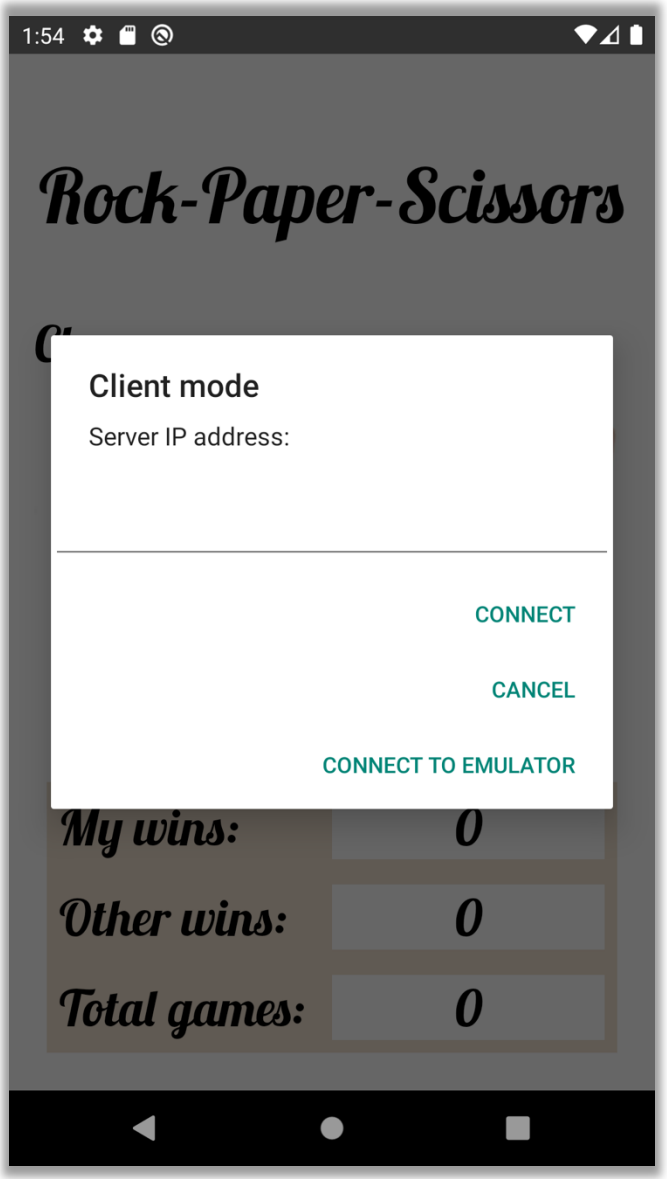
- Multiplayer game
- Communication via *sockets*
  - `java.net, java.io`
- One device will be designated as the server
  - It waits for another device to connect
- The other device will be designated as the client
  - The server's IP must be defined
    - As an option, a search for other devices on the network can be done
      - *Broadcast*
      - *Wifi peer-to-peer (P2P)*
      - *Network Service Directory (NSD)*
      - *…*

Main screen

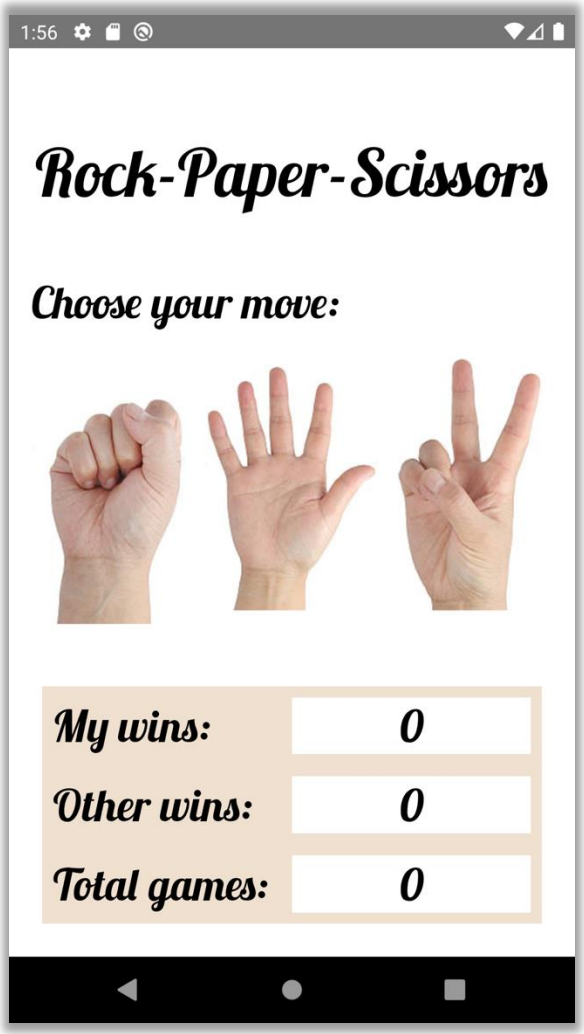


Game mode configuration

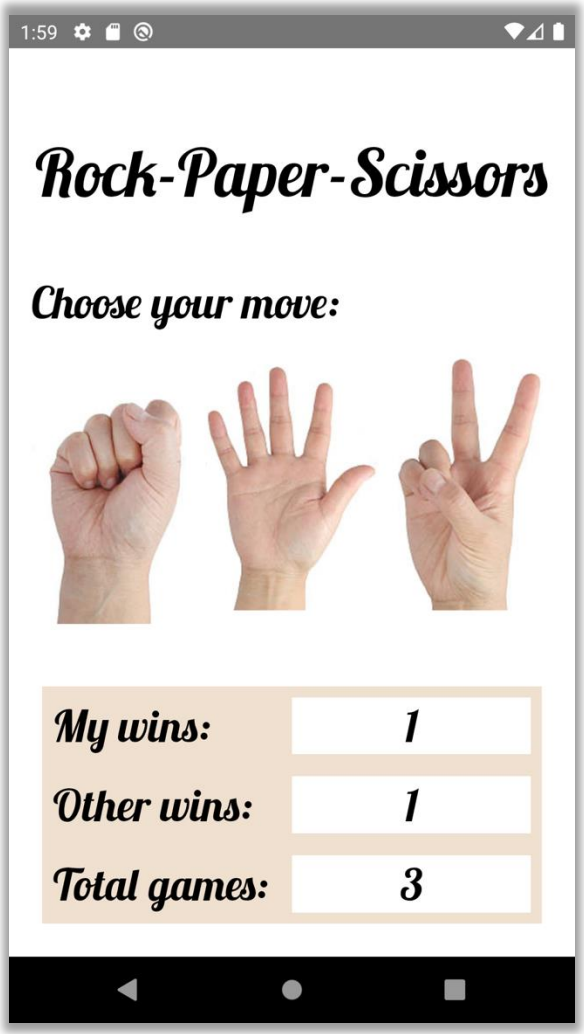




Playing


1:56
Rock-Paper-Scissors
Choose your move:
My wins: 0
Other wins: 0
Total games: 0


1:57
Rock-Paper-Scissors
My wins: 0
Other wins: 0
Total games: 0


1:59
Rock-Paper-Scissors
Choose your move:
My wins: 1
Other wins: 1
Total games: 3

Application testing

- *Device (Server/Client) ⇔ Device (Server/Client)*
  - No issues
- *Device (Server) ⇔ Emulator (Client)*
  - The physical device has to act as the server since there is no way to connect to the server running on an emulator
  - The emulator can connect to an external server
- *Emulator (Server) ⇔ Emulator (Client)*
  - Configuration of different ports and port forwarding is necessary (see next slide)
  - Some emulators have internal implementation issues with the IP stack, they can be used as a client but not as a server

Testing with emulators

- To enable testing with emulators, a different listening port to the one used for the client to connect must be specified for the server
- It is necessary to enable TCP/IP traffic redirection between the ports defined for the server and client
  - Port redirection configuration is only required on the emulator acting as the server
    - `telnet localhost 5554`
      - *macOS*: `nc localhost 5554`
    - `auth <authentication_token>`
    - `redir add tcp:9998:9999`
    - `exit`
- In the client emulator, the connection must be made to the IP 10.0.2.2

Bluetooth Communication

https://developer.android.com/guide/topics/connectivity/bluetooth

Bluetooth Communication

- One of the most popular ways to implement communication between devices is through radio communication via *Bluetooth*
- Android supports the typical *Bluetooth* protocol stack allowing:
  - Searching for other devices
  - Device pairing
  - Establishment of communication channels
  - Data transfer
  - Management of multiple connections

Permissions

- The use of some *Bluetooth subservices* requires a permission request
  - For example, to perform search operations from other devices
- API permissions < 31
  - `Manifest.permission.BLUETOOTH`
  - `Manifest.permission.ACCESS_FINE_LOCATION`
  - `Manifest.permission.ACCESS_COARSE_LOCATION`
- API permissions >= 31
  - `Manifest.permission.BLUETOOTH_CONNECT`
  - `Manifest.permission.BLUETOOTH_SCAN`
  - `Manifest.permission.BLUETOOTH_ADVERTISE`

Important Classes

- `BluetoothManager`
  - API to obtain `BluetoothAdapter` objects
- `BluetoothAdapter`
  - API to interact with the entire *Bluetooth* stack, scan the network, obtain `BluetoothDevice` object, ...
- `BluetoothDevice`
  - Represents a device with which a connection can be established
- `BluetoothServerSocket`
  - Represents a socket over *Bluetooth* to wait for client connections
  - Works similarly to TCP/IP `ServerSocket`
- `BluetoothSocket`
  - Represents a connection established between two devices
  - It works in a similar way to the TCP/IP `Socket`

Bluetooth preparation

- Obtaining the `BluetoothAdapter` object

```
val bluetoothManager: BluetoothManager =
                                getSystemService(BluetoothManager::class.java)
val bluetoothAdapter: BluetoothAdapter? = bluetoothManager.getAdapter()
if (bluetoothAdapter == null) {
  // Device doesn't support Bluetooth
}
```

- Activate *Bluetooth* if it is turned off

```
if (bluetoothAdapter?.isEnabled == false) {
  val enableBtIntent = Intent(BluetoothAdapter.ACTION_REQUEST_ENABLE)
  startActivityForResult(enableBtIntent, REQUEST_ENABLE_BT)
}
```

Scan for Bluetooth devices

- Get previously paired devices

```
val pairedDevices: Set<BluetoothDevice>? =
                            bluetoothAdapter?.bondedDevices
pairedDevices?.forEach { device ->
    val deviceName = device.name
    val deviceHardwareAddress = device.address
                                        // MAC address
}
```

Scan for Bluetooth devices

- Search for other nearby devices using the following methods of `BluetoothAdapter`:
  - `startDiscovery()`
  - `stopDiscovery()`

```
override fun onCreate(savedInstanceState: Bundle?) {
   ...
   // Register for broadcasts when a device is discovered.
   val filter = IntentFilter(BluetoothDevice.ACTION_FOUND)
   registerReceiver(receiver, filter)
}
```

```
// Create a BroadcastReceiver for ACTION_FOUND.
private val receiver = object : BroadcastReceiver() {
   override fun onReceive(context: Context, intent: Intent) {
       when(intent.action) {
           BluetoothDevice.ACTION_FOUND -> {
               // Discovery has found a device. Get the BluetoothDevice
               // object and its info from the Intent.
               val device: BluetoothDevice =
                    intent.getParcelableExtra(BluetoothDevice.EXTRA_DEVICE)
               val deviceName = device.name
               val deviceHardwareAddress = device.address // MAC address
           }
       }
   }
}
```

```
override fun onDestroy() {
   super.onDestroy()
   ...

   // Don't forget to unregister the ACTION_FOUND receiver
   unregisterReceiver(receiver)
}
```

Turn on Bluetooth visibility

- To make a device visible to others

```
val requestCode = 1
val discoverableIntent: Intent =
   Intent(BluetoothAdapter.ACTION_REQUEST_DISCOVERABLE).apply {
      putExtra(BluetoothAdapter.EXTRA_DISCOVERABLE_DURATION, 300)
   }

startActivityForResult(discoverableIntent, requestCode)
```

Server side

- One of the ways used may involve all clients performing the roles of server and client (accepting connections or establishing a connection)
  - When two devices are not previously paired, the pairing mechanism will start automatically

- A device can obtain a `BluetoothSocketServer` when registering to listen to a service, identified by a UUID
  - `listenUsingRfcommWithServiceRecord(String,UUID)`
    - UUID – *Universally Unique Identifier* (`java.util.UUID`)
      - Example: `71b12727-9c5b-4ed2-a749-cb18f9768eb9`
      - It can be obtained through online generators, calling `UUID.randomUUID()`, …

- By using `BluetoothSocketServer`, it is possible to wait for client connections using the `accept` function
  - After accepting a client (`BluetoothSocket`), the `BluetoothServerSocket` object can be released if new clients are not expected or allowed

Server side

```
private inner class AcceptThread : Thread() {

    private val mmServerSocket: BluetoothServerSocket? by lazy(LazyThreadSafetyMode.NONE) {
        bluetoothAdapter?.listenUsingInsecureRfcommWithServiceRecord(NAME, MY_UUID)
    }

    override fun run() {
        // Keep listening until exception occurs or a socket is returned.
        var shouldLoop = true
        while (shouldLoop) {
            val socket: BluetoothSocket? = try {
                mmServerSocket?.accept()
            } catch (e: IOException) {
                Log.e(TAG, "Socket's accept() method failed", e)
                shouldLoop = false
                null
            }
            socket?.also {
                manageMyConnectedSocket(it)
                mmServerSocket?.close()
                shouldLoop = false
            }
        }
    }

    // Closes the connect socket and causes the thread to finish.
    fun cancel() {
        try {
            mmServerSocket?.close()
        } catch (e: IOException) { Log.e(TAG, "Could not close the connect socket", e) }
    }
}
```

Client side

- For a client to connect to a Bluetooth server
  - Get the `BluetoothDevice` to connect to, by
    - Getting the paired devices, or
    - Doing a network-to-network search
  - Get the `BluetoothSocket` object

    `device``.createRfcommSocketToServiceRecord(UUID)`
  - Establish the connection

    `bluetoothSocket``.connect()`

Client side

```
private inner class ConnectThread(device: BluetoothDevice) : Thread() {

    private val mmSocket: BluetoothSocket? by lazy(LazyThreadSafetyMode.NONE) {
        device.createRfcommSocketToServiceRecord(MY_UUID)
    }

    public override fun run() {
        // Cancel discovery because it otherwise slows down the connection.
        bluetoothAdapter?.cancelDiscovery()

        mmSocket?.let { socket ->
            // Connect to the remote device through the socket. This call blocks
            // until it succeeds or throws an exception.
            socket.connect()

            // The connection attempt succeeded. Perform work associated with
            // the connection in a separate thread.
            manageMyConnectedSocket(socket)
        }
    }

    // Closes the client socket and causes the thread to finish.
    fun cancel() {
        try {
            mmSocket?.close()
        } catch (e: IOException) { Log.e(TAG, "Could not close the client socket", e) }
    }
}
```

Data transfer

- Having the connection established, represented through a `BluetoothSocket`:
  - To receive messages via *Bluetooth,* use the `inputStream` provided by `BluetoothSocket`
  - To send messages via *Bluetooth,* use the `outputStream` provided by `BluetoothSocket`
  - Message readings and writings must be carried out in the context of a *thread* other than the *main thread*

Other forms of connection

- In this context, there are other forms of communication that may be more suitable for certain applications
  - *Bluetooth Low Energy* (BLE)
    - Communication with low-power systems used, for example, in the context of *wearables* and *beacons*
    - https://developer.android.com/guide/topics/connectivity/bluetooth/ble-overview
  - *Near Field Communication* (NFC)
    - Very short distance communication, usually 4 cm or less
    - Based on RFID technology
    - https://developer.android.com/guide/topics/connectivity/nfc

Inclusion of advertising

AdMob

Application Monetization

Advertising insertion – AdMob

- To obtain profits from an application, several ways can be explored, including the insertion of advertising in them
- There are several ways to insert advertising, but in this lesson we will cover inserting advertising through the *Google AdMob* service
- Go to https://admob.google.com to create an *AdMob account*
- Register the application

Advertising insertion – AdMob

- Add the following line to the section `dependencies` in the `build.gradle.kts` file

```
implementation("com.google.android.gms:play-services-ads:23.6.0")
```

- *Obtain the AdMob App ID* key from the *AdMob* website in the "*App settings*" page in the application configuration
  - Place the *App ID* assigned to the application in the manifest file using the following meta information:

```
<manifest>
    <application>
        <!-- Sample AdMob app ID: ca-app-pub-3940256099942544~3347511713 -->
        <meta-data
            android:name="com.google.android.gms.ads.APPLICATION_ID"
            android:value="ca-app-pub-xxxxxxxxxxxxxxx~yyyyyyyyyy"/>
    </application>
</manifest>
```

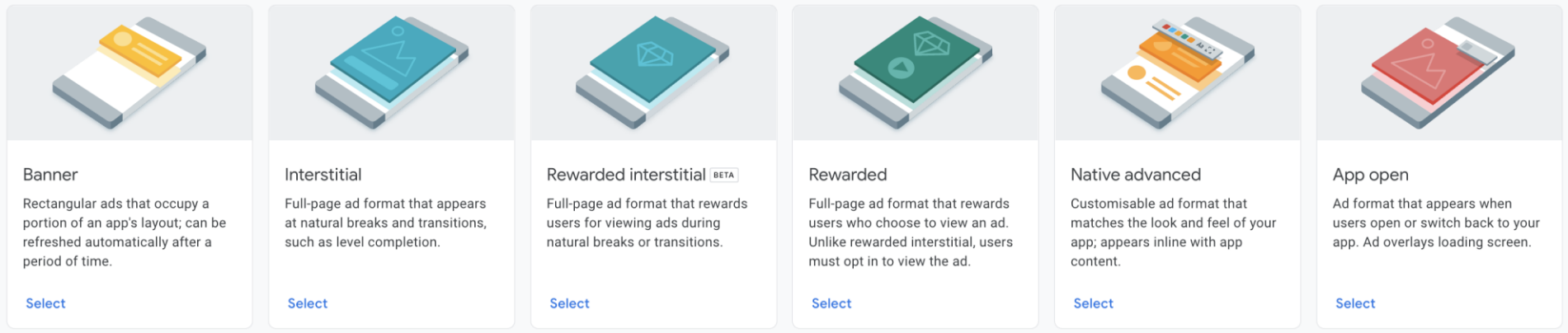
Advertising insertion – AdMob

- Start *AdMod* as soon as possible in the application, for example, in the `onCreate` of the `Application` object, `MainActivity` or similar

```
MobileAds.initialize(this) { }
```

Advertising insertion – AdMob

- Choose and insert the desired type of advertising for the different screens/moments of the application
  - The type of advertising to be inserted must be registered on the application website in *AdMod* using the "*Add ad unit*" option



Advertising insertion – AdMob

- When a new "*ad unit*" is created, an appropriate *ID* is generated for it, which must be associated with the elements in the application

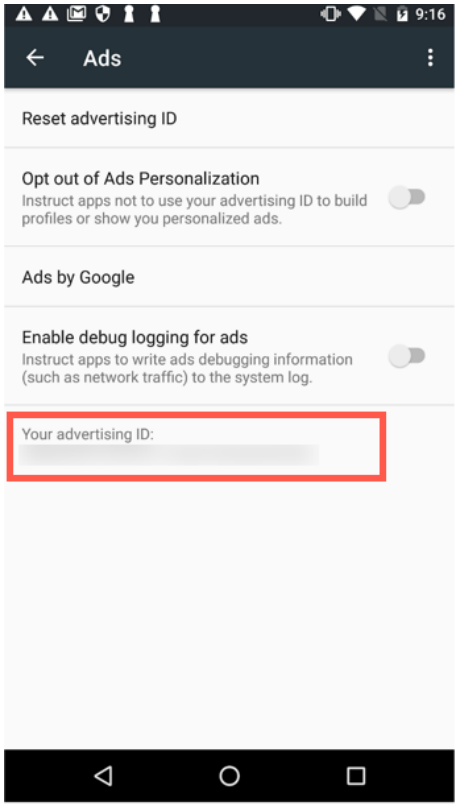
- In the development phase, the following keys must be used to prevent touches on test advertisements from being interpreted as fraud attempts

Ad format	Sample ad unit ID
AppOpen	ca-app-pub-3940256099942544/3419835294
Banner	ca-app-pub-3940256099942544/6300978111
Interstitial	ca-app-pub-3940256099942544/1033173712
Interstitial Video	ca-app-pub-3940256099942544/8691691433
Rewarded	ca-app-pub-3940256099942544/5224354917
Rewarded Interstitial	ca-app-pub-3940256099942544/5354046379
Native Advanced	ca-app-pub-3940256099942544/2247696110
Native Advanced Video	ca-app-pub-3940256099942544/1044960115

Advertising insertion – AdMob

- In the development phase, instead of using test codes, test devices can be registered (whose use will not be detected as fraud)

```
val testDeviceIds = listOf(
                AdRequest.DEVICE_ID_EMULATOR,
                "33BE2250B43518CCDA7DE426D04EE231")
val configuration = RequestConfiguration.Builder()
                .setTestDeviceIds(testDeviceIds)
                .build()
MobileAds.setRequestConfiguration(configuration)
```



- The *id* of a device can be obtained by querying *Logcat* for a device that is running the application in *debug mode*

```
I/Ads: Use RequestConfiguration.Builder().setTestDeviceIds
(Arrays.asList("33BE2250B43518CCDA7DE426D04EE231")) to get
test ads on this device
```

Advertising insertion – AdMob

- Example of a component to be inserted in a XML layout file for an *ad* in *Banner* format

```
<com.google.android.gms.ads.AdView
        xmlns:ads="http://schemas.android.com/apk/res-auto"
        android:id="@+id/adView"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        ads:adSize="BANNER"
        ads:adUnitId="ca-app-pub-3940256099942544/6300978111"/>
```

- To insert an *ad* programmatically:

```
val adView = AdView(this)
adView.adSize = AdSize.BANNER
adView.adUnitId = "ca-app-pub-3940256099942544/6300978111"
// TODO: Add adView to your view hierarchy
```

Advertising insertion – AdMob

- Example that shows how to load an *ad* in the `onCreate` method of an `Activity`:

```
class MainActivity : AppCompatActivity() {

    lateinit var mAdView : AdView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        MobileAds.initialize(this) {}

        mAdView = findViewById(R.id.adView)
        val adRequest = AdRequest.Builder().build()
        mAdView.loadAd(adRequest)
    }
}
```

Advertising insertion – AdMob

- Example that shows how to load an *ad* in *Jetpack Compose*:

```
@Composable
fun MyAdComposable(modifier: Modifier = Modifier) {
    if (LocalInspectionMode.current) {
        Text(
            modifier = modifier
                .fillMaxWidth()
                .background(Color.Red)
                .padding(horizontal = 2.dp, vertical = 6.dp),
            textAlign = TextAlign.Center,
            color = Color.White,
            text = "Advert Here",
        )
    } else {
        AndroidView(
            modifier = modifier.fillMaxWidth(),
            factory = { context ->
                AdView(context).apply {
                    adSize = AdSize.BANNER
                    adUnitId = context.getString(R.string.ad_id_banner)
                    loadAd(AdRequest.Builder().build())
                }
            }
        )
    }
}

@Preview(showBackground = true)
@Composable
fun AdvertPreview() {
    AdvertView()
}
```

Play Store & Deployment

- Process and information required to publish an application on *Play Store* from *Google* using the *Google Play Developer Console*

Demonstrated in class with the help of an example